3X-UI Panel User Manual

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский · Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

3X-UI version: 3.4.2. This manual is based on and current for this version. A summary of changes in 3.4.2 relative to 3.4.1 is in the What's new in 3.4.2 section.

> A detailed English-language manual for the **3X-UI** web panel (Xray-core management): features, configuration and operation, with an explanation of every field and toggle in the interface.
>
> Names and labels correspond to the panel interface. The words *inbound* / *outbound* are not translated.

Table of Contents

What's new in 3.4.2

Version 3.4.2 is a major update: WireGuard has been moved to a multi-client model, REALITY gained a live target scanner, balancers got Observatory / Burst Observatory tabs, and confirmation of sensitive settings with a 2FA code was added. Below are the changes relative to 3.4.1, grouped by manual section.

Changes in section 1 — Introduction, requirements and installation

- A **"Documentation"** button (book icon) has appeared in the sidebar (and in the mobile drawer) — it opens the official documentation at `https://docs.sanaei.dev/`.
- The minimum Xray version the panel updates to has been raised to **26.6.27** (the bundled core is Xray 26.6.27).

Changes in section 2 — Panel login and access security

- When 2FA is enabled, changing the administrator login/password and disabling 2FA now require **entering the current code** from the authenticator app (confirmation of sensitive changes).
- LDAP: a new **"Skip TLS certificate verification"** toggle (`ldapInsecureSkipVerify`) — disables certificate verification under LDAPS; available only when "Use TLS (LDAPS)" is enabled.

Changes in section 3 — Overview / Dashboard

- The panel version button now always opens the update window (see section 16 — dev channel).
- A cross-cutting **accessibility** improvement: aria labels for icon buttons and activation of elements via Enter/Space (for screen readers and keyboard navigation).

Changes in section 4 — Inbounds: creation and common parameters

- The **"Export all links"** action now builds links through the subscription engine — it applies the remark template to each client and prefers managed Host endpoints (previously a fixed `inbound-email` remark was used).

Changes in section 5 — Protocols

- **WireGuard has been moved to a multi-client model.** Peers are now regular clients (with automatic in-tunnel address assignment, subscription support, traffic/expiry limits and groups); the inline "Peers" list has been removed from the inbound form.
- A configurable **DNS** field (default `1.1.1.1, 1.0.0.1`) and a **client configuration card** have been added to the WireGuard inbound — copy/download/QR of the full `.conf` and a `wireguard://` / `wg://` link.

Changes in section 6 — Transport (Stream Settings)

- For new XHTTP inbounds, the `maxConnections` parameter in **xmux** now defaults to **6** (it was `0` — no limit). Existing inbounds keep their value.

Changes in section 7 — Connection security: TLS, XTLS and REALITY

- A **live REALITY target scanner** has been added: the **"Scan"** button (check the current target "live") and the **"Find targets"** button (scan a domain or an **IP/CIDR** range and pick suitable targets by their certificates). The "Target" and SNI fields are now empty when REALITY is first selected.

Changes in section 8 — Clients

- Extending the expiry/quota via `bulkAdjust` now **automatically enables** a client that was disabled solely because of exhaustion (expired term or exceeded quota), if the extension brings it back within the limits. Clients disabled manually or still exhausted remain disabled.

Changes in section 9 — Client groups

- **"Reset traffic"** for a group now zeroes **only the group's own counter**; the counters, quotas and state of individual clients are not affected, and no Xray restart is required. This is a change from the previous behavior (where the traffic of all the group's clients used to be reset).

Changes in section 10 — Subscriptions

- In **managed hosts**, the **VLESS route** field has been redefined: it is now a single value `0–65535` (rather than a port list), and it is actually "baked" into the UUID of every subscription (raw/JSON/Clash).
- The `{{EMAIL}}` variable (and its synonym `{{USERNAME}}`) in the remark template is now output only on the client's **first link** — just like the traffic/expiry block.

Changes in section 11 — Xray: routing, outbounds, DNS and extensions

- **Balancers**: the page has been split into **"Balancer settings"** and **"Observatory"** tabs; instead of raw JSON there are now Observatory and Burst Observatory forms (Burst gains an **"HTTP method"** field). A Random/Round-robin balancer with a `fallbackTag` now automatically creates a Burst Observatory.
- When an outbound or balancer is deleted, the panel cleans up the related references in routing itself and shows a **preview of the consequences** in the confirmation dialog.
- In routing rules, the **L4** network criterion is written to the config in lowercase (`tcp` / `udp`), while the table displays it in uppercase.
- Errors in the add/edit balancer form are now deferred until the first time a field is touched or a save is attempted.

Changes in section 12 — Nodes (multi-panel, master/slave)

- The "saved locally, node offline — will sync later" notification is now shown only when the node is actually offline or disabled (previously — on every save to an online node).

Changes in section 16 — Operations: backups, logs, updates, CLI

- Backup file names now contain the server address and a **date-time**: `{host}_YYYY-MM-DD_HHMMSS.db` (`.dump` for PostgreSQL), for example `panel.example.com_2026-06-27_000000.db` — both when downloading from the panel and in backups sent by the Telegram bot.
- The **dev channel** of updates can now be enabled from a stable build: the version button always opens the update window, and a **"Dev channel"** toggle has appeared with a warning about instability and the absence of automatic rollback.

1. Introduction, Requirements, and Installation

1.1. What Is 3X-UI

3X-UI is an open-source web management panel for Xray-core servers. The panel provides a unified multilingual web interface for deploying, configuring, and monitoring a wide range of proxy and VPN protocols — from a single VPS to distributed multi-node configurations.

3X-UI is an extended fork of the original X-UI project. Compared to it, support for more protocols has been added, stability has been improved, per-client traffic accounting has been introduced, and many convenient features have been included.

Key features:

- **Inbound for various protocols** — VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN, and **MTProto** (Telegram proxy, added in 3.3.0).
- **Modern transports and encryption** — TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade, and XHTTP, secured with TLS, XTLS, and REALITY.
- **Fallback** — serving multiple protocols on a single port (e.g., VLESS and Trojan on 443) using Xray's fallback mechanism.
- **Per-client management** — traffic quotas, expiration dates, IP limits, online status display, one-click invite links, QR codes, and subscriptions.
- **Traffic statistics** — per inbound, client, and outbound, with reset capability.
- **Multi-node support** — managing and scaling across multiple servers from a single panel.
- **Outbound and routing** — WARP, NordVPN, custom routing rules, load balancers, proxy chains.
- **Built-in subscription server** with multiple output formats.
- **Telegram bot** for remote monitoring and management.
- **REST API** with built-in Swagger documentation.
- **Flexible storage** — SQLite (default) or PostgreSQL.
- **13 interface languages**, dark and light themes.
- **Fail2ban integration** for enforcing per-client IP limits.

> Important: the project is intended for personal use only. It is not recommended for use in illegal activities or production environments.

1.2. Supported Operating Systems and Architectures

Operating Systems

The installation script detects the distribution from the `ID` field in `/etc/os-release` (or `/usr/lib/os-release`). Officially supported systems are:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine, and Windows.

On Alpine-family systems the OpenRC service manager is used (`rc-service` / `rc-update`); on all others — systemd. For CentOS 7, packages are installed via `yum` ; for newer releases — via `dnf` . If the distribution is not recognized, the script falls back to `apt-get` by default.

CPU Architectures

The architecture is determined from the output of `uname -m` and mapped to one of the supported values:

`uname -m` value	3X-UI architecture
`x86_64` , `x64` , `amd64`	`amd64`
`i*86` , `x86`	`386`
`armv8*` , `arm64` , `aarch64`	`arm64`
`armv7*` , `arm`	`armv7`
`armv6*`	`armv6`
`armv5*`	`armv5`
`s390x`	`s390x`

If the architecture is not in this list, the script prints "Unsupported CPU architecture!" and aborts installation.

Base Dependencies

Before installing the panel, the script automatically installs a base set of packages (names vary by distribution): `cron` / `cronie` / `dcron` , `curl` , `tar` , `tzdata` / `timezone` , `socat` , `ca-certificates` , `openssl` .

1.3. Installation Methods

Method 1. Installation Script (Recommended)

Installation is performed with a single command as root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

The script requires root privileges: if run as a non-root user, it prints "Please run this script with root privilege" and exits with an error.

What the installer does, step by step:

1. Detects the OS and architecture.
2. Installs base dependencies.
3. Downloads the release archive `x-ui-linux-<arch>.tar.gz` and extracts it to `/usr/local/x-ui` .
4. Downloads the management script `x-ui.sh` and installs it as the `/usr/bin/x-ui` command.
5. Creates the log directory `/var/log/x-ui` .
6. Runs initial setup: database selection, credential generation, port selection, optional SSL configuration.
7. Installs and starts the autostart service (systemd unit `x-ui.service` or an OpenRC init script for Alpine).

Database selection during installation. The installer offers:

- `1) SQLite` (default, recommended for fewer than 500 clients) — a single file at `/etc/x-ui/x-ui.db`, no configuration required.
- `2) PostgreSQL` (recommended for a large number of clients or multiple nodes). PostgreSQL can be installed locally (a dedicated user and database named `xui` are created) or you can provide a DSN for an existing server. The connection parameters are written to the service environment file (`/etc/default/x-ui`, `/etc/conf.d/x-ui`, or `/etc/sysconfig/x-ui` depending on the distribution) as the `XUI_DB_TYPE` and `XUI_DB_DSN` variables.

Example: writing PostgreSQL parameters to the service environment file. After selecting PostgreSQL and providing the DSN, the installer will add approximately the following lines to the environment file:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Here `xui` is the username and database name, `127.0.0.1:5432` is the server address and port, and `sslmode=disable` is suitable for a local connection (for a remote server, `require` is typically used).

Installing a specific (older) version. You can explicitly specify a version tag — the installer will download the corresponding release:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

The minimum supported version for this type of installation is `v2.3.5`; specifying an older version prints "Please use a newer version (at least v2.3.5)".

Installing a dev build. In addition to a version tag, the installer accepts the argument `dev-latest` (alias `dev`) — this installs the rolling dev build from the latest commit on the `main` branch:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

The dev build is a per-commit pre-release (tag `dev-latest`), not a stable version, so the minimum version check is skipped for it. When run, it prints the warning "Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version." Without an argument, the installer installs the latest stable release. Use the dev build only to test fixes that have not yet been released; for normal operation, install stable versions.

Method 2. Docker

Run with the default SQLite database:

```
docker compose up -d
```

To run with the built-in PostgreSQL service, uncomment the `XUI_DB_*` lines in `docker-compose.yml` and start with the profile:

```
docker compose --profile postgres up -d
```

The image includes Fail2ban (enabled by default) for enforcing per-client IP limits. Fail2ban blocks violators via `iptables`, which requires the `NET_ADMIN` capability. In `docker-compose.yml` it is already provided via `cap_add`. When running manually with `docker run`, you must add the capabilities yourself; otherwise blocks will only be logged but not applied:

Example: full `docker run` command. A minimal variant with the panel port exposed, network capabilities, and a persistent volume for the database:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

The `/etc/x-ui` volume preserves the `x-ui.db` file across container restarts; without it, settings and accounts will be lost.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

In Docker, the panel is the container's main process: autostart is controlled by the container's restart policy (e.g., `restart: unless-stopped`), not by a service inside the container.

1.4. First Launch and Default Credentials

On the first installation (when default credentials are still in use), the installer **generates random values** for the username, password, web path, and port:

Parameter	How it is set during installation	Note
Username	random 10-character string	generated automatically
Password	random 10-character string	generated automatically
Panel web path (WebBasePath)	random 18-character string	protects the panel from discovery at the root URL
Panel port (Port)	random port in the range 1024–62000 by default; can be set manually if desired	the factory `webPort` value is `2053`, but the installer overwrites it

At the end of installation the script prints a summary: username, password, port, web path, API token, and the ready-to-use Access URL of the form:

```
https://<domain-or-IP>:<port>/<web-path>
```

If an SSL certificate has not been configured, the URL will use `http://`, and the script will print a warning about the need to configure SSL (menu item 19).

> Mandatory credential change. Since the login and password are randomly generated, you should **save them immediately after installation**. They can be changed at any time using the "Reset Username & Password" menu item (see below) or from the web interface in the panel settings. After the reset, the script reminds you: "Please use the new login username and password to access the X-UI panel. Also remember them!".

After installation, use the `x-ui` command to open the management menu (see section 1.6).

1.5. File Locations

Path	Purpose
`/usr/local/x-ui/`	panel installation directory (binary `x-ui`, script `x-ui.sh`)
`/usr/local/x-ui/bin/xray-linux-<arch>`	Xray-core binary (on armv5/armv6/armv7 renamed to `xray-linux-arm`)
`/usr/bin/x-ui`	management script (the `x-ui` command)
`/etc/x-ui/x-ui.db`	SQLite database file (default)
`/var/log/x-ui/`	panel log directory
`/etc/systemd/system/x-ui.service`	systemd service unit (not for Alpine)
`/etc/init.d/x-ui`	OpenRC init script (Alpine only)
`/etc/default/x-ui` · `/etc/conf.d/x-ui` · `/etc/sysconfig/x-ui`	service environment variable file (path depends on distribution); `XUI_DB_TYPE` / `XUI_DB_DSN` are written here

The database directory can be overridden with the `XUI_DB_FOLDER` environment variable (default `/etc/x-ui`), and the Xray binary directory with `XUI_BIN_FOLDER` (default `bin` relative to the panel directory). The database file name is `x-ui.db`.

Example: moving the database to a separate disk. To store `x-ui.db` not in `/etc/x-ui` but, for example, on a mounted disk at `/data`, set the variable in the service environment file and restart the panel:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

The full database path will become `/data/x-ui/x-ui.db`.

Main Environment Variables

Variable	Purpose	Default
`XUI_DB_TYPE`	database backend: `sqlite` or `postgres`	`sqlite`

Variable	Purpose	Default
`XUI_DB_DSN`	PostgreSQL connection string (when `XUI_DB_TYPE=postgres`)	—
`XUI_DB_FOLDER`	SQLite database file directory	`/etc/x-ui`
`XUI_INIT_WEB_BASE_PATH`	initial web panel URI path (only on first initialization)	`/`
`XUI_DB_MAX_OPEN_CONNS`	maximum open connections (PostgreSQL pool)	—
`XUI_DB_MAX_IDLE_CONNS`	maximum idle connections (PostgreSQL pool)	—
`XUI_ENABLE_FAIL2BAN`	enable IP limit enforcement via Fail2ban	`true`
`XUI_LOG_LEVEL`	logging level (`debug`, `info`, `warning`, `error`)	`info`
`XUI_DEBUG`	debug mode	`false`

Example: temporarily enabling verbose logging. To diagnose an issue, raise the log level to `debug` and restart the service:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log    # view the debug log
```

After diagnostics, restore the value to `info` to prevent the log from growing excessively.

Initial web panel path via environment. The `XUI_INIT_WEB_BASE_PATH` variable sets the web panel URI path (`webBasePath`) during initial settings initialization. This is useful when deploying via Docker or systemd to fix the panel's login path from the start. The value is normalized automatically — leading and trailing slashes are added as needed, and an empty or whitespace-only value is ignored (in which case the default path `/` is applied). The variable affects **only the initial initialization**: if settings already exist, the path is changed in the web interface or via the "Reset Web Base Path" menu item.

1.6. The `x-ui` Management Command (Script Menu)

After installation, the `x-ui` command (run as root) opens the "3X-UI Panel Management Script" interactive menu. A menu item is selected by entering its number (range 0–27). Many items are also available as subcommands for use in scripts (see section 1.7).

The menu is divided into thematic sections.

Installation and Updates

- **1. Install** — install the panel (runs `install.sh`). Checks that the panel is not already installed before proceeding.
- **2. Update** — update all x-ui components to the latest version. Data is preserved; the panel restarts automatically after the update. Requires confirmation.
- **3. Update Menu** — update only the management script (`x-ui.sh` / the `x-ui` command) to the current version without reinstalling the panel.

- **4. Legacy Version** — install a specified (older) version of the panel. The script prompts for a version number (e.g., `2.4.0`) and downloads the corresponding release.
- **5. Uninstall** — completely remove the panel **along with Xray**. The service is stopped and disabled, the directories `/etc/x-ui/` and `/usr/local/x-ui/`, the service environment file, and the management script itself are all deleted. Requires confirmation (default is "no").

Credentials and Settings

- **6. Reset Username & Password** — reset the panel username and password. You can enter your own values or leave them empty for random generation (random username — 10 characters, random password — 18 characters). Additionally offers to disable two-factor authentication (2FA) if it is configured. The panel restarts after the reset.
- **7. Reset Web Base Path** — reset the panel web path: a new random path (18 characters) is generated and the panel restarts. Use this if the previous path was compromised or forgotten.
- **8. Reset Settings** — reset all panel settings to their default values. **Credentials (username and password) and account data are preserved.** Requires confirmation; the panel restarts after the reset.
- **9. Change Port** — change the web panel port. Prompts for a port number (1–65535); a restart is required after setting it for the change to take effect.
- **10. View Current Settings** — view current settings (`x-ui setting -show`). Shows, among other things, the database backend in use (SQLite or PostgreSQL with the password masked in the DSN) and the ready-to-use Access URL. If SSL is not configured, offers to issue a Let's Encrypt certificate for an IP address.

Service Management

- **11. Start** — start the panel service. If the panel is already running, a message is displayed indicating that a restart is not needed.
- **12. Stop** — stop the panel service.
- **13. Restart** — restart the panel service.
- **14. Restart Xray** — restart only the Xray-core engine without restarting the panel itself (via `systemctl reload x-ui`; in Docker — by sending the `USR1` signal to the panel process).
- **15. Check Status** — check the service status (`systemctl status x-ui` or `rc-service x-ui status`).
- **16. Logs Management** — log management: view the debug log (Debug Log, via `journalctl`) and, except on Alpine, clear all logs (Clear All logs).

Autostart

- **17. Enable Autostart** — enable panel autostart on OS boot (`systemctl enable x-ui` or `rc-update add`).
- **18. Disable Autostart** — disable autostart on OS boot.

In Docker, autostart is controlled by the container's restart policy, so these items only display a corresponding hint.

Security and Networking

- **19. SSL Certificate Management** — manage SSL certificates via acme.sh: issue a certificate for a domain, revoke, force-renew, view existing domains, specify certificate paths for the panel, and issue a short-lived (~6 days, with auto-renewal) certificate for an IP address.
- **20. Cloudflare SSL Certificate** — issue an SSL certificate via Cloudflare DNS validation.
- **21. IP Limit Management** — manage per-client IP limits (based on Fail2ban): view and remove blocks, etc.
- **22. Firewall Management** — manage the firewall (open/close ports and view rules).
- **23. SSH Port Forwarding Management** — configure SSH port forwarding to access the panel from a local machine via an SSH tunnel.

Performance and Maintenance

- **24. Enable BBR** — enable/disable the BBR TCP congestion control algorithm (submenu with Enable BBR / Disable BBR items).
- **25. Update Geo Files** — update geo databases (`.dat` files) with a choice of source: Loyalsoldier (`geoip.dat` , `geosite.dat`), chocolate4u (`geoip_IR.dat` , `geosite_IR.dat`), runetfreedom (`geoip_RU.dat` , `geosite_RU.dat`), or All (all at once). The panel restarts after the update.
- **26. Speedtest by Ookla** — run a network speed test via Speedtest by Ookla.
- **27. PostgreSQL Management** — manage the built-in/linked PostgreSQL instance (enabling and related operations).
- **0. Exit Script** — exit the menu.

1.7. `x-ui` Subcommands (Without the Interactive Menu)

For use in scripts, the `x-ui` command supports direct subcommands (running `x-ui` without arguments opens the menu):

Command	Action
`x-ui`	open the management menu
`x-ui start`	start the panel
`x-ui stop`	stop the panel
`x-ui restart`	restart the panel
`x-ui restart-xray`	restart Xray
`x-ui status`	current service status
`x-ui settings`	current settings
`x-ui enable`	enable autostart on OS boot
`x-ui disable`	disable autostart
`x-ui log`	view logs
`x-ui banlog`	view Fail2ban block logs
`x-ui update`	update the panel

Command	Action
`x-ui update-all-geofiles`	update all geo files
`x-ui migrateDB [file]`	convert `.db` ⍰ `.dump` (SQLite)
`x-ui legacy`	install an older version
`x-ui install`	install the panel
`x-ui uninstall`	remove the panel

1.8. SQLite → PostgreSQL Migration

An existing SQLite installation can be migrated to PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# then set XUI_DB_TYPE and XUI_DB_DSN in /etc/default/x-ui and restart:
systemctl restart x-ui
```

The original SQLite file is left untouched — delete it manually only after verifying that the new backend is working.

Example: verifying the switch to PostgreSQL. After migration, confirm that the panel is actually running on the new backend using the settings view command — the output should indicate PostgreSQL (the password in the DSN is masked):

```
x-ui settings | grep -i -E 'db|dsn'
```

If the panel opens and accounts are in place, the original `x-ui.db` can be deleted.

---

2. Panel login and access security

This section covers everything related to authenticating the administrator of the 3X-UI panel: the login form, two-factor authentication (TOTP), brute-force protection, changing credentials, changing the panel's secret path and port, session lifetime, and synchronization/authentication via LDAP.

2.1. Login form

The login page is served at the root of the panel's secret path (`webBasePath`). If the user is already authenticated, they are automatically redirected to `…/panel/` . The page has a theme switcher, an interface language selector, and the form itself.

Form fields:

Field	Hint/label (RU)	Required	Description
Username	"Username"	Yes	The administrator's login. An empty value is rejected on the client side, and on the server side with the message "Enter username".
Password	"Password"	Yes	The administrator's password. An empty value is rejected with the message "Enter password".
2FA code	"2FA code"	Only when 2FA is enabled	The field appears **only** if two-factor authentication is enabled for the panel. A 6-digit code from the authenticator app.

The **"Login"** button submits the form to `POST /login` .

Behavior and messages:

- On successful login, "Login successful" is shown and the user is taken to `…/panel/` .
- On any credentials error or an incorrect 2FA code, the server returns a **single** message: "Invalid account data." (English: *Invalid username or password or two-factor code.*). This is intentional — the panel does not reveal exactly what is wrong (login, password, or code) so as not to make brute-forcing easier.
- The panel shows or hides the "2FA code" field based on the `POST /getTwoFactorEnable` request, which returns the current 2FA status even before authorization.
- If the server-side session has expired, the next request shows "Session expired. Please log in again", and the user is redirected to the login page.

> Note about CSRF: before submitting the form, the client obtains a CSRF token (`GET /csrf-token`); the `/login` and `/logout` requests are protected by a CSRF check.

Example: logging in via the API. When 2FA is off, the login and password are enough; when 2FA is on, the `twoFactorCode` field is added:

```
# Without 2FA
curl -i -X POST https://panel.example.com:2053/my-secret/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=YourPassword'

# With 2FA enabled — a 6-digit code is added
curl -i -X POST https://panel.example.com:2053/my-secret/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=YourPassword&twoFactorCode=123456'
```

On success the server returns a `Set-Cookie` header with the session cookie — pass it in subsequent requests to `/panel/api/…`.

2.2. Two-factor authentication (2FA / TOTP)

2FA in 3X-UI is implemented according to the **TOTP** standard and is compatible with any authenticator app (Google Authenticator, Aegis, FreeOTP, etc.). The parameters are hard-coded: algorithm **SHA1**, **6** digits, period **30** seconds, issuer `3x-ui`, label `Administrator`.

Example: the otpauth URI encoded by the QR code. If the authenticator app cannot use the camera, the token can be added manually via a link like this (substitute your own Base32 secret instead of `JBSWY3DPEHPK3PXP`):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

The parameters `algorithm=SHA1`, `digits=6`, `period=30` match the panel's hard-coded values — there is no need to change them.

The settings are located in **Settings** → **Account**, on the **"Two-factor authentication"** tab.

Element	Text (RU)	Description
Toggle	"Enable 2FA"	Enables/disables two-factor authentication.
Description	"Adds an additional layer of authentication to improve security."	The hint below the toggle.

How to enable 2FA

When the toggle is turned on, the panel **generates a new secret locally** — a random string in Base32 encoding (alphabet `A–Z` and `2–7`). The "Enable two-factor authentication" window opens with a step-by-step guide:

1. **"Scan this QR code in your authenticator app, or copy the token next to the QR code and paste it into the app"**. Below the QR code, the secret itself is displayed in text form — clicking the QR code copies the secret to the clipboard (a "Copied" notification pops up).
2. **"Enter the code from the app"** — you must enter the 6-digit code generated by the app. The code is verified **on the browser side**: the panel itself computes the current TOTP from the just-generated secret and compares it with the entered one. If the code is incorrect — "Invalid code"; the field accepts only exactly 6 digits.

Only after a successful confirmation are the secret and the enable flag saved. On saving, "Two-factor authentication has been set up successfully" is shown.

Important: changes in the settings section are applied with the common **"Save"** button, after which a panel restart is usually required ("Save the changes and restart the panel to apply them"). When 2FA is enabled for the first time, the server additionally **invalidates all active sessions** (increments the "login epoch"), so after applying the setting a new login will be required — now with the 2FA code.

How to disable 2FA

Toggling the switch again opens the "Disable two-factor authentication" window with the hint "Enter the code from the app to disable two-factor authentication.". After entering a valid code (as of 3.4.2 it is verified **on the server**), the flag and the secret are cleared, and "Two-factor authentication has been removed successfully" is shown.

Code verification at login

At login, the server takes the stored secret and compares the current TOTP with the submitted 2FA code. A mismatch is treated as a failed login, but the user is shown the same combined message "Invalid account data.".

Recovery of access

There is **no** separate "recovery codes" mechanism in 3X-UI. If access to the authenticator app is lost, login cannot be recovered through the panel interface. The only way is to disable 2FA directly in the database on the server: reset the `twoFactorEnable` key to `false` (and, if necessary, clear `twoFactorToken`) in the settings table, then restart the panel. For this reason, it is recommended to store the secret (the Base32 token) in a safe place when enabling 2FA.

Example: emergency 2FA disable on the server. After getting SSH access to the server, stop the panel, reset the keys in the settings table, and start the panel again:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

After this, login works with the username and password only, and 2FA can be set up again if desired.

> Relation to changing credentials: when the login/password is changed (see 2.4), 2FA is **automatically disabled** on the server so that the old secret does not block access under the new account.

2.3. Login attempt limiting (login limiter / brute-force protection)

The panel includes a built-in failed-login limiter (an application-level analog of fail2ban). The parameters are set in code and are **not configurable** through the interface:

Parameter	Value	Purpose
Maximum failures	**5**	How many failed attempts are allowed within the window.
Counting window	**5 minutes**	The sliding window in which failures accumulate (older ones are discarded).
Lockout (cooldown)	**15 minutes**	How long the key is blocked after exceeding the threshold.

How it works:

- The lockout key is built from the **"IP + login" pair** (the login is lowercased, whitespace is trimmed). That is, the lockout applies to a specific "address + username" pair, not to the entire panel.
- On each failed attempt (incorrect login/password or an incorrect 2FA code) the counter grows. After reaching **5** failures within **5 minutes**, the key is blocked for **15 minutes**. During the lockout, any attempts by this pair are immediately rejected with the same message "Invalid account data.", even if the data is correct.
- A **successful login immediately resets** the counter and lifts the lockout for that pair.
- The client's IP address is determined taking trusted proxies into account (see `trustedProxyCIDRs`): the `X-Real-IP` and `X-Forwarded-For` headers are accepted only if the request came from a trusted address. Otherwise the real connection address is used, and if it cannot be extracted — the string `unknown`.

All attempts are logged. For failed ones, a warning is written to the server log with the username, IP, reason and, on lockout, the `blocked_until` time. If login notifications via the Telegram bot are enabled (`tgNotifyLogin` — "Login notification"), the administrator additionally receives the username, IP, and time of both successful and failed and blocked attempts.

Example: login notification in Telegram. With `tgNotifyLogin` enabled, after each attempt the administrator receives a message roughly like this:

```
Login notification
User: admin
IP: 203.0.113.45
Time: 2026-06-10 14:32:07
Status: success
```

For a blocked "IP + login" pair, the status will indicate that the attempt was rejected by the limiter.

2.4. Changing the administrator login and password

The **Settings** → **Account** section, the **"Administrator credentials"** tab. Fields:

Field	Text (RU)	Description
Current login	"Current login"	The current username. It must match the current login, otherwise the change is rejected.
Current password	"Current password"	The current password, for identity confirmation.
New login	"New login"	The new username. Cannot be empty.

Field	Text (RU)	Description
New password	"New password"	The new password. Cannot be empty.

The change is applied with the **"Confirm"** button and submitted to `POST /panel/setting/updateUser`.

Server logic and messages:

- If "Current login" does not match the actual one or "Current password" is incorrect — "An error occurred while changing the administrator credentials." with the explanation "Incorrect username or password".
- If the new login or new password is empty — the explanation "The new username and new password must not be empty".
- On success — "You have successfully changed the administrator credentials.". The password is stored as a bcrypt hash.

Confirmation with a 2FA code (as of 3.4.2). If two-factor authentication is enabled on the panel, changing the login/password additionally requires **entering the current code** from the authenticator app. A **"Change credentials"** window opens with the hint "Enter the code from the app to change the administrator credentials."; the code (`twoFactorCode`) is verified on the server, and with an incorrect or empty code the change is not applied. The same confirmation is now also required when disabling 2FA (see 2.2).

Example: changing credentials via the API. The request requires a valid session cookie (obtained at login) and confirmation of the current login/password:

```
curl -X POST https://panel.example.com:2053/my-secret/panel/setting/updateUser \
  -b 'session=YOUR_SESSION_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=OldPassword&newUsername=root&newPassword=NewStro
ngPassword'
```

After success the current session is invalidated — you will need to log in again with the new credentials.

Important effects of changing credentials:

- **All existing sessions are invalidated** (the user's `login_epoch` counter is incremented), so after the change the panel automatically logs out and redirects to the login page — you need to log in again.
- If **2FA was enabled** at the time of the change, **it is automatically disabled** (the flag and the secret are reset). Two-factor authentication will have to be set up again after changing the login/password.

If 2FA is enabled, before the form is submitted the "Change credentials" window opens with the hint "Enter the code from the app to change the administrator credentials." — credentials can be changed only by confirming the current 2FA code.

2.5. Secret path (URI path / webBasePath) and panel port

These parameters are located in the **Settings** → **Panel** section and directly affect the panel's "stealth" and accessibility. They take effect after saving and **restarting the panel**.

Field	Text (RU)	Default value	Description
Panel port	"Panel port" (`panelPort`), hint "The port the panel runs on"	**2053**	The TCP port of the web interface.
URI path	"URI path" (`panelUrlPath`), hint "Must start with '/' and end with '/'"	**/**	The secret base path (`webBasePath`). The panel is accessible only at it (for example, `/my-secret/`).
Panel management IP address	"Panel management IP address" (`panelListeningIP`), hint "Leave empty to connect from any IP"	empty	The address the panel listens on. Empty = all interfaces.
Panel domain	"Panel domain" (`panelListeningDomain`), hint "Leave empty to connect from any domains and IPs."	empty	Restricts access by domain (Host).
Path to the panel certificate's public key	`publicKeyPath` , hint "Enter the full path starting with '/'"	empty	The TLS certificate for HTTPS access to the panel.
Path to the panel certificate's private key	`privateKeyPath` , same hint	empty	The TLS private key.

Behavior of the base path (`webBasePath`):

- The value is normalized automatically: if it does not start with `/` , the character is added at the beginning; if it does not end with `/` , one is added at the end. So in practice the path is always of the form `/…/` .
- The base path applies to the panel itself, to the assets, and to the session cookie (the cookie is issued only for this path).

> Security recommendations (the "Security warnings" section): the panel itself shows warnings if the configuration is "too public":
>
> - "The panel runs over plain HTTP — set up TLS for production."
> - "The default port 2053 is widely known — change it to a random one."
> - "The default base path "/" is widely known — change it to a random one."
>
> In other words, for a production server you should set a **non-standard port**, a **non-trivial URI path**, and a **TLS certificate**.

Example: a "stealth" panel configuration for production. In the **Settings** → **Panel** section set roughly these values:

Field	Value
Panel port	`34571` (random, instead of 2053)
URI path	`/aXf9Qm2/` (non-trivial, starts and ends with `/`)

Field	Value
Path to the panel certificate's public key	`/etc/letsencrypt/live/panel.example.com/fullchain.pem`
Path to the panel certificate's private key	`/etc/letsencrypt/live/panel.example.com/privkey.pem`

After saving and restarting, the panel will be reachable only at `https://panel.example.com:34571/aXf9Qm2/`, and the security warnings will disappear.

2.6. Session lifetime (timeout)

The **"Session duration"** field (`sessionMaxAge`) is located among the panel/interval settings.

Field	Text (RU)	Default value	Unit	Description
Session duration	"Session duration", hint "The session duration in the system (value: minute)"	**360**	minutes	The lifetime of the administrator session cookie.

Behavior:

- The value is specified in **minutes** (default 360 minutes = 6 hours) and is converted to seconds when configuring the cookie.
- If the value is **greater than 0**, the session cookie is given a corresponding `MaxAge`. After this period expires, the cookie stops being valid and on the next request the user gets "Session expired. Please log in again".
- The session also becomes invalid prematurely when credentials are changed or 2FA is enabled for the first time (via the `login_epoch` mechanism, see 2.4 and 2.2) and on an explicit logout (`POST /logout`).
- The session cookie is marked `HttpOnly`, with the `SameSite=Lax` policy; the `Secure` flag is set on direct HTTPS access to the panel.

In addition to the timeout itself, there is a related notification: **"Session expiry notification lead time"** (`expireTimeDiff`, hint "Receive a notification about session expiry before the threshold is reached (value: day)", default `0`) — it allows receiving a warning in advance.

2.7. LDAP (synchronization and authentication)

The LDAP section provides two capabilities: (1) authenticating the administrator's login via LDAP if the local password did not match, and (2) periodically synchronizing the state of clients (the enabled/disabled VLESS flag) from the directory.

How it is used at login: the server first checks the local bcrypt password hash. If it **did not match** and LDAP is enabled, the panel attempts to authenticate the user in the directory: when a `Bind DN` is set, a service bind is performed, then the user record is searched by the filter and attribute, and a bind under the found DN with the entered password is attempted. Success means a login. (After a successful LDAP authentication, if 2FA is enabled, the TOTP code is still verified.)

Section fields:

Field	Text (RU)	Default value	Description
Enable LDAP synchronization	"Enable LDAP synchronization" (`enable`)	**false**	The master switch for the LDAP integration.
LDAP host	"LDAP host" (`host`)	empty	The address of the LDAP server.
LDAP port	"LDAP port" (`port`)	**389**	The port. For LDAPS usually 636.
Use TLS (LDAPS)	"Use TLS (LDAPS)" (`useTls`)	**false**	When enabled, the `ldaps://` scheme is used (by default — with server certificate verification).
Skip TLS certificate verification	"Skip TLS certificate verification" (`ldapInsecureSkipVerify`)	**false**	Added in 3.4.2. Disables server certificate verification under LDAPS — for internal/self-signed CAs. Hint: "Insecure — disables server certificate verification. Use only with internal/untrusted CAs". The toggle is available **only** when "Use TLS (LDAPS)" is enabled.
Bind DN	"Bind DN" (`bindDn`)	empty	The DN of the service account for the initial bind/search. If empty — no bind is performed (anonymous search).
Bind password	hints: "Configured; leave empty to keep the current password." / "Not configured." / "Configured — enter a new value to replace"	empty	The password for the `Bind DN` . Stored separately; to keep the previous one, the field is left empty.
Base DN	"Base DN" (`baseDn`)	empty	The root of the subtree in which the search is performed (the search is recursive, over the entire subtree).
User filter	"User filter" (`userFilter`)	`(objectClass=person)`	The LDAP filter for selecting accounts. During authentication, the login is substituted into the filter with escaping.
User attribute (username/email)	"User attribute (username/email)" (`userAttr`)	`mail`	The attribute matched against the login/client identifier (for example, `mail` or `uid`).
VLESS flag attribute	"VLESS flag attribute" (`vlessField`)	`vless_enabled`	The attribute that determines whether the client's VLESS access should be enabled.
General flag attribute (opt.)	"General flag attribute (opt.)" (`flagField`), hint "If set, overrides the VLESS flag — e.g. shadowInactive."	empty	If set, it is used instead of `vless_enabled` .

Field	Text (RU)	Default value	Description
Truthy values	"Truthy values" (`truthyValues`), hint "Comma-separated; default: true,1,yes,on"	`true,1,yes,on`	The list of flag-attribute values treated as "enabled".
Invert flag	"Invert flag" (`invertFlag`), hint "Enable when the attribute means "disabled" (e.g. shadowInactive)."	**false**	Inverts the meaning of the flag.
Sync schedule	"Sync schedule" (`syncSchedule`), hint "A cron-like string, e.g. @every 1m"	`@every 1m`	The synchronization frequency in a cron-like format.
Inbound tags	"Inbound tags" (`inboundTags`), hint "The inbounds on which LDAP synchronization may auto-create or auto-delete clients."	empty	Restricts which inbounds allow auto-operations. If there are no inbounds: "No inbounds found. Create an inbound first."
Auto-create clients	"Auto-create clients" (`autoCreate`)	**false**	Create a client in the specified inbounds if it appeared in the directory.
Auto-delete clients	"Auto-delete clients" (`autoDelete`)	**false**	Delete a client if it disappeared from the directory.
Default volume (GB)	"Default volume (GB)" (`defaultTotalGb`)	**0**	The traffic limit for auto-created clients (0 = no limit).
Default term (days)	"Default term (days)" (`defaultExpiryDays`)	**0**	The validity period for auto-created clients (0 = unlimited).
Default IP limit	"Default IP limit" (`defaultIpLimit`)	**0**	The limit on the number of simultaneous IPs (0 = no limit).

Specifics of the synchronization flag logic: when reading the flag attribute (`flagField` , default `vless_enabled`), the value is considered "enabled" if it is in the list of truthy values; when inversion is enabled, the result is flipped to the opposite. The user attribute (`userAttr`) is used as the matching key (email/name) — records without a value for this attribute are skipped.

> Security: it is recommended to enable **TLS (LDAPS)** so that bind passwords and verified passwords are not transmitted in plain text, and to use an account with the minimum necessary read permissions for the `Bind DN` .

Example: a typical LDAP synchronization configuration (Active Directory). Filling in the section fields for a directory where the access status is stored in a flag attribute and matching is done by email:

Field	Value
LDAP host	`ldap.example.com`
LDAP port	`636`
Use TLS (LDAPS)	enabled

Field	Value
Bind DN	`CN=svc-3xui,OU=Service,DC=example,DC=com`
Base DN	`OU=Users,DC=example,DC=com`
User filter	`(objectClass=person)`
User attribute (username/email)	`mail`
VLESS flag attribute	`vless_enabled`
Truthy values	`true,1,yes,on`
Sync schedule	`@every 5m`

With this setup, every 5 minutes the panel walks the `OU=Users` subtree, matches clients by `mail`, and enables/disables VLESS access based on the `vless_enabled` value.

---

3. Overview / Dashboard

The Dashboard (*Overview*) is the panel's start page. It shows the server and Xray process state in real time. All metrics come from the server side. A background scheduler rebuilds the snapshot **every 2 seconds** and broadcasts it to all open tabs via WebSocket; once a minute the accumulated metric rows are flushed to disk. The HTTP endpoint `GET /status` returns the last cached snapshot.

Every metric and every control on the page is described below.

As of version 3.4.2, the panel sidebar (and the mobile drawer) has a **"Documentation"** button (book icon) — it opens the official documentation at `https://docs.sanaei.dev/`. Also in 3.4.2 a cross-cutting **accessibility** improvement was made: icon buttons and clickable elements received aria labels and activation via Enter/Space (for screen readers and keyboard navigation).

3.1. General data-collection principles

- The snapshot is collected by the `gopsutil` library. If a particular measurement fails, the field remains zero and a warning is written to the log (`get cpu percent failed`, `get uptime failed`, etc.) — this does not crash the whole dashboard; the corresponding tile simply shows 0/N-A.
- "Instantaneous" speeds (CPU %, network, disk I/O) are computed as the difference between the current and the previous snapshot divided by the interval in seconds. Therefore, on the first page load speed values may be zero until a second measurement is available.
- History can be viewed in the *System History* section — charts are built from the same data rows described below (see §3.12).

3.2. CPU

The *CPU* tile shows the current processor utilization in percent, as well as processor parameters.

Metric	Description
CPU usage, %	Share of processor time used during the last interval. Smoothed with an exponential moving average (EMA, coefficient `alpha = 0.3`) so that spikes do not jitter the indicator. Value is always clamped to 0–100 %. On the very first measurement 0 is returned (baseline initialization).
Logical processors	Number of logical cores, i.e. including Hyper-Threading.
Physical cores	Number of physical cores.
Frequency	Base processor frequency in MHz. Queried lazily and cached: the first successful reading is saved, a retry is made no more than once every 5 minutes, and the request itself is limited to a 1.5 s timeout (frequency queries respond slowly on some systems).

CPU utilization is calculated as follows: if a native platform implementation is available it is used; otherwise the calculation is based on processor-time counter deltas (busy / total). Guest and GuestNice time are excluded to avoid counting them twice.

3.3. RAM

The *RAM* tile shows used and total memory. Displayed as "used / total" and/or fill percentage. The percentage is written to history.

3.4. Swap

The *Swap* tile shows used and total swap. If no swap file/partition is configured (total = 0), the metric is zero; when swap is absent, 0 is written to the history row.

3.5. Storage

The *Storage* tile shows used and total, considering **only the root partition** `/`. The fill percentage is written to the *Disk Usage* history. Disk I/O (read / write, bytes/s) is collected separately as a counter delta over the interval — it is shown on the *Disk I/O* history tab.

3.6. System Uptime

The *Uptime* metric is the time since **the entire server** was booted (in seconds), not the uptime of the panel or Xray. The Xray process uptime is stored separately (see §3.9), as is the panel thread count (*Threads*).

Panel memory usage

Next to the panel process metrics the amount of RAM occupied by the 3X-UI process itself is displayed. This value is taken from the process's actual RSS (as the operating system sees it) and matches what system utilities show. The number decreases as memory is freed. Previously the panel showed an internal Go counter that overstated memory consumption (e.g. ~300 MB on an idle server with one client) and never decreased — that artifact is gone. Additionally, a periodic background process returns unused memory to the operating system so that the metric reflects actual consumption.

3.7. Load Average

The *System Load* block is an array of three numbers `[Load1, Load5, Load15]`. Tooltip label: "System load average for the past 1, 5, and 15 minutes". The history chart is titled "System load average (1 / 5 / 15 min)". Values are written to history rows separately: `load1`, `load5`, `load15`.

This is the standard Unix metric: the average number of processes waiting in the run queue. As a reference point, compare it against the number of cores — a load that consistently exceeds the number of physical cores indicates overload.

3.8. Network: Speed and Total Traffic

Only physical interfaces are counted. Virtual and tunnel interfaces are excluded: `lo` / `lo0` and everything starting with `loopback`, `docker`, `br-`, `veth`, `virbr`, `tun`, `tap`, `wg`, `tailscale`, `zt`. Values are summed across all remaining interfaces.

Overall Speed — instantaneous speed, counter delta over the interval:

Metric	Description
Upload (*Upload*)	Outgoing speed, bytes/s.
Download (*Download*)	Incoming speed, bytes/s.

Total Data — accumulated counters since system start:

Metric	Description
Sent (*Sent*)	Total bytes sent.
Received (*Received*)	Total bytes received.

Packet rates (packets/s) and total packet counters are also collected — they are shown on the *Network Packets* history tab. Network history rows: `netUp`, `netDown`, `pktUp`, `pktDown`.

3.9. Server IP Addresses

The *IP Addresses* block shows `IPv4` and `IPv6`. External addresses are determined via third-party services (`api4.ipify.org`, `ipv4.icanhazip.com`, `v4.api.ipinfo.io/ip`, `ipv4.myexternalip.com/raw`, `4.ident.me`, `check-host.net/ip` for IPv4 and equivalent services for IPv6). The list is tried in order until the first successful response; timeout per request is 3 s.

Details:

- The result is **cached** for the lifetime of the process: a successfully resolved address is not re-requested.
- If no service responds, the field remains `N/A`. For IPv6, after the first `N/A` all IPv6 requests are disabled to avoid wasting time on networks without IPv6.
- Nearby there is an eye button to hide/show addresses — tooltip: "Toggle visibility of the IP". This is purely a visual toggle in the interface (e.g. for screenshots) and has no effect on the addresses themselves.

3.10. TCP/UDP Connections

The *Connection Stats* block shows the total number of active TCP and UDP connections on the server (system-wide, not just Xray). The history chart is *Active Connections (TCP / UDP)*, rows `tcpCount`, `udpCount`.

3.11. Xray Status and Process Controls

The *Xray* card shows the state of the Xray-core process and provides controls for it.

States

Value	Label	When set
`running`	*Running*	The Xray process is running.
`stop`	*Stopped*	The process is not running and no startup error has been recorded.
`error`	*Error*	The process is not running and a startup error has been recorded. The error text is shown in a pop-up with the title "An error occurred while running Xray".

Value	Label	When set
—	*Unknown*	Displayed while status has not yet been received.

The **Xray version** is shown next to the status.

Control buttons

- **Stop** (*Stop*). Calls `POST /stopXrayService`. On success, the panel broadcasts the new `stop` state and a notification "Xray service has been stopped" over WebSocket; on error — the `error` state with the error text. Important: if the panel is accessed *through* Xray itself, stopping Xray may break the connection to the panel — there is no issue when connecting to the panel directly.
- **Restart** (*Restart*). Calls `POST /restartXrayService`. A confirmation dialog "Restart xray?" with the note "Reloads the xray service with the saved configuration" is shown before the action. On success — `running` state and notification "Xray service has been restarted successfully". Restart applies the current saved configuration — use it after changing settings.

> Note. In this fork full Start / Stop / Restart controls for all authorization types have been added to the dashboard; the original 3x-ui UI has no separate Start button — starting is performed via Restart.

Xray log viewer button

The Xray card has a *Logs* button for viewing Xray logs. It appears only when an access log is configured in the Xray configuration: the built-in viewer reads exactly that file, so without an access log the button is hidden. Button visibility is tied to a separate flag `accessLogEnable` and no longer depends on the IP limit — the online list and IP-address limit continue to work even without an access log (see §8).

Xray version selection

The *Version* section lets you switch Xray-core to a different release. The version list is loaded via `GET /getXrayVersion`:

- The source is the GitHub API of the `XTLS/Xray-core` repository (`/releases`). Requests are cached for **15 minutes**; on a GitHub failure the last successfully fetched list is returned so that the picker is not empty.
- Only releases of the form `X.Y.Z` and **no older than 26.4.25** are included in the list.

Tooltips: "Choose the version you want to switch to." and the warning "Choose carefully, as older versions may not be compatible with current configurations."

Switching: `POST /installXray/:version`. Scenario:

Example. Switch to a specific Xray-core version (the session cookie must already be obtained via authentication):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Here `v25.6.8` is a tag from the list returned by `GET /getXrayVersion`. The version must be present in that list; otherwise the panel will reject the request. As of version 3.4.2 the minimum Xray version allowed for installation has been raised to **26.6.27** (the bundled core is Xray 26.6.27), so older builds are not available for updating.

1. The selected version is verified against the current release list (otherwise — rejected).
2. Xray is stopped.
3. The archive `Xray-<os>-<arch>.zip` for the current OS and architecture is downloaded from GitHub (supported: amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; for Windows — `xray.exe`). The archive and binary size limit is 200 MB.
4. The binary is replaced atomically (via a temporary file + rename) and marked as executable.
5. Xray is started again.

A dialog "Do you really want to change the Xray version?" with the description "This will change the Xray version to #version#" is shown before switching. On success — notification "Xray updated successfully".

3.12. Panel Update (3X-UI)

The panel update check block. Data comes via `GET /getPanelUpdateInfo`:

Field	Description
Current panel version	Version of the installed panel.
Latest panel version	The latest 3x-ui release fetched from GitHub.
Update available	Indicates that the latest version is newer than the current one. If no update is needed, "Panel is up to date" is shown.

The **Update Panel** button (*Update Panel*) triggers `POST /updatePanel`. Tooltip: "This will update 3X-UI to the latest release and restart the panel service". A confirmation "Do you really want to update the panel?" with the text "This will update 3X-UI to version #version# and restart the panel service" is shown before running.

Specifics and limitations:

- Self-update is supported **on Linux only** (other OSes return an error).
- The updater script is downloaded from the official repository (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`, 2 MB limit) and executed via `bash`, where possible in isolation via `systemd-run`.
- On successful launch, "Panel update started" is shown; if the update check failed — "Panel update check failed". During installation a warning "Installation in progress. Do not refresh the page" is displayed.

3.13. Geo-file Update (GeoIP / GeoSite)

The geo-database update button/dialog calls `POST /updateGeofile` (all files) or `POST /updateGeofile/:fileName` (a single file). Updates work against a strict allowlist of names and sources:

File	Source
`geoip.dat`, `geosite.dat`	`Loyalsoldier/v2ray-rules-dat` (latest)
`geoip_IR.dat`, `geosite_IR.dat`	`chocolate4u/Iran-v2ray-rules` (latest)
`geoip_RU.dat`, `geosite_RU.dat`	`runetfreedom/russia-v2ray-rules-dat` (latest)

Behavior:

- The file name is validated: `..`, slashes, and absolute paths are forbidden; only `[a-zA-Z0-9._-]+.dat` is allowed. Files not on the allowlist are not downloaded.
- Conditional requests use `If-Modified-Since`: if the file has not changed on the source server (HTTP 304), it is not re-downloaded — only its timestamp is updated.
- After download, Xray is **restarted** to pick up the new databases.

Example. Update only the Russian geo-databases without touching the other files:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b
cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b
cookie.txt
```

To update all files on the allowlist at once, call `POST /updateGeofile` without a file name.

- Dialogs: "Do you really want to update the geo file?" with "This will update the #filename# file" for a single file and "This will update all geo files" for the "Update All" button. Success — "Geo files updated successfully".

3.14. Database Backup and Restore

The *Backup & Restore* block. Behavior depends on the database engine in use (SQLite by default or PostgreSQL).

Database export (Backup)

The *Back Up* button calls `GET /getDb`. The file is delivered as an attachment:

- **SQLite**: a checkpoint (WAL flush) is performed first, then the `x-ui.db` file is downloaded. Tooltip: "Click to download the .db file containing the backup of your current database...".
- **PostgreSQL**: an `x-ui.dump` file in custom format is downloaded (`pg_dump --format=custom --no-owner --no-privileges`). PostgreSQL client tools must be installed on the server; otherwise an error about missing `pg_dump` is returned.

Database import (Restore)

The *Restore* button uploads a file via `POST /importDB` (form field `db`). Tooltip: "Click to select and upload the .db file... to restore the database from backup".

The **SQLite** scenario is safe, with rollback:

1. The file is checked for SQLite format and saved to a temporary file, then its integrity is verified.
2. Xray is stopped, the current database is closed and renamed to `*.backup` (fallback).
3. The new file takes the place of the working database; initialization and migration are performed. If something goes wrong, the fallback is restored.
4. Xray is started again.

For **PostgreSQL** a `.dump` file is uploaded (the `PGDMP` signature is checked) and applied via `pg_restore --clean --if-exists --single-transaction …`. The tooltip explicitly warns: "This will replace all current data".

Messages: "Database imported successfully", "An error occurred while importing the database", "...while reading the database", "...while getting the database".

Migration file (between SQLite and PostgreSQL)

The *Download Migration* button calls `GET /getMigration` and produces a portable export for running the panel on a different database engine:

- On **SQLite** an `x-ui.dump` (plain SQL dump) is downloaded.
- On **PostgreSQL** an `x-ui.db` — a ready-made SQLite database assembled from PostgreSQL data — is downloaded.

3.15. Additional Interface Elements

- **Online clients indicator.** The dashboard maintains an `online` row (*Online Clients*) — the number of clients with an active connection. Counted when Xray is running (otherwise 0) and written to history on the same 2-second tick. Chart — the *Online* tab.
- **System History (charts).** The *Charts → System History* button/section with tabs: *Bandwidth*, *Packets*, *Disk I/O*, *Online*, *Load*, *Connections*, *Disk Usage*. Data is fetched via `GET /history/:metric/:bucket`; allowed aggregation intervals (bucket, seconds): **2, 30, 60, 180, 360, 720, 1440, 2880, 10080**, up to 60 points per tab. The range selector on the page offers buttons **2m, 1h, 3h, 6h, 12h, 24h, 2d, 7d** (buckets `2, 60, 180, 360, 720, 1440, 2880, 10080` respectively). On the longer ranges **2d** and **7d** axis time labels include the date in `MM-DD HH:MM` format. Storage uses three-level downsampling (rollup): fresh data is kept at a 2 s step for the last **hour**, then averaged to 1 min steps for **48 hours**, and to 10 min steps for **7 days**. Charts (CPU, RAM, traffic, packets, connections, disk, online, load) can therefore be viewed for a period of **up to 7 days** (previously up to 48 hours), with coarser detail the further back in time. Allowed metrics: `cpu, mem, swap, netUp, netDown, pktUp, pktDown, diskRead, diskWrite, diskUsage, tcpCount, udpCount, online, load1, load5, load15`. The label "Last 2 minutes" corresponds to bucket = 2 (real-time mode).

Example. Fetch the CPU load series for approximately the last 2 minutes (bucket = 2 s, up to 60 points) and the same series aggregated over 5 minutes (bucket = 300 s):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

The metric can be replaced with any allowed value (`mem`, `netUp`, `tcpCount`, `load1`, etc.). A bucket not on the allowlist `2, 30, 60, 180, 360, 720, 1440, 2880, 10080` will be rejected.

- **Xray metrics** — a separate block with Xray memory consumption and garbage collection (rows `xrAlloc, xrSys, xrHeapObjects, xrNumGC, xrPauseNs`) and an *Observatory* (state of outbound connections). Works only when a `metrics` block is configured in the Xray configuration (`listen 127.0.0.1:11111`, tag `metrics_out`); otherwise "Xray metrics endpoint is not configured" is shown. The Xray metrics window has its own range selector with buttons **2m, 1h, 3h, 6h, 12h** (buckets `2, 60, 180, 360, 720`).

Example of a block that enables the Xray metrics tile. The Xray settings section must contain both `metrics` (with a tag) and an inbound that listens on that tag simultaneously:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

The address `127.0.0.1:11111` is intentionally not exposed externally — the panel queries it locally.

- **Dark theme toggle.** Located in the general menu/header, not in the dashboard itself. Options: *Theme* with choices *Dark* and *Ultra Dark*. This is a purely visual appearance setting and has no effect on panel operation.
- **Other links** in the dashboard surroundings (from menu/bottom bar): *Logs*, *Configuration* — view the final Xray JSON (`GET /getConfigJson`), *Documentation*.

---

4. Inbounds: creation and common parameters

The **Inbounds** section is the list of all Xray entry points through which clients connect. Each inbound stores both "panel" fields (remark, traffic limit, reset schedule) and raw JSON blocks of the Xray configuration (`settings`, `streamSettings`, `sniffing`).

Creation is done via the **Add Inbound** button, editing via **Modify Inbound**. Both operations are sent to the API endpoints `POST /add` and `POST /update/:id`.

Below are all form fields that are **not** related to the settings of a specific protocol (clients, encryption, REALITY/TLS) and **not** related to transport/stream (the **Stream** and **Security** tabs) — those are the subjects of separate sections.

4.1. Common form fields

Remark

Parameter	Value
Field	`remark`
Type	string
Default	empty

A human-readable name for the inbound, shown in the list and in dialog titles ("Delete inbound "{remark}"?", etc.). The field label is **"Remark"**. It does not affect Xray operation and is only for administrative convenience; it is recommended to set unique, meaningful names, since they are inserted into the names of exported files and into confirmations of bulk operations.

Protocol

Parameter	Value
Field	`protocol`
Label	**"Protocol"**
Validation	`required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun`

A drop-down list of the inbound's protocol. The allowed values are:

Value	Note
`vmess`	
`vless`	
`trojan`	

Value	Note
`shadowsocks`	
`wireguard`	
`hysteria`	Hysteria v2 is `hysteria` with `streamSettings.version = 2`; there is no separate protocol
`http`	
`mixed`	socks/http on a single port
`tunnel`	
`tun`	accepted by the validator, no separate protocol constant

The field is required (`required`). The choice of protocol determines which client settings fields and which transport will be available (see the protocol-specific sections).

> Important: when saving, the service normalizes `streamSettings`. Transport settings are kept only for the protocols `vmess`, `vless`, `trojan`, `shadowsocks`, `hysteria`; for the others (`http`, `mixed`, `tunnel`, `wireguard`, `tun`) the `streamSettings` field is **forcibly cleared**.

For a `tunnel`/TProxy inbound whose `streamSettings` block carries no `security` key (the transportless variant), the form now opens and saves without the `streamSettings.security Invalid input` validation error.

Listen IP

Parameter	Value
Field	`listen`
Type	string
Default	empty → Xray listens on `0.0.0.0` (all IPs)

The IP address on which the inbound accepts connections. The field hint:

> "Leave blank to listen on all IP addresses."

When generating the Xray configuration, an empty value is replaced with `0.0.0.0`. In addition to an IP, the field accepts a **Unix socket path** — hint:

> "You can also enter a Unix socket path (e.g. /run/xray/in.sock), or an abstract socket name prefixed with @ (e.g. @xray/in.sock), to listen on a socket instead of a TCP port — set Port to 0 in that case."

Thus the field accepts two Unix socket forms: a filesystem path (`/run/xray/in.sock`) and an abstract socket name prefixed with `@` (`@xray/in.sock`). In both cases set `Port` to `0`.

You change this field when you need to restrict the inbound to a single interface (for example, `127.0.0.1` for an inbound that works only as a fallback target behind Nginx), or when the inbound listens on a Unix socket.

Example. An inbound that listens only on the local interface (a typical fallback target behind Nginx), and one that listens on a Unix socket:

```
listen = 127.0.0.1   port = 8443
listen = /run/xray/in.sock   port = 0
```

Port

Parameter	Value
Field	`port`
Label	**"Port"**
Validation	`gte=0,lte=65535`
Default	— (set by the user)

The TCP/UDP listening port. Values from `0` to `65535` are allowed. The value `0` is used only in combination with listening on a Unix socket (see above).

When saving, the service checks for a port conflict: two inbounds cannot simultaneously occupy overlapping `listen:port` for the same transport (TCP/UDP). The transport is derived from the protocol and `streamSettings` / `settings`: for example, `hysteria` and `wireguard` always occupy UDP, `kcp` / `quic` — UDP, and most others — TCP. On a conflict, saving is rejected with an error.

Separately, the panel does not allow occupying the **reserved internal Xray API port** (tag `api`, default `62789` on `127.0.0.1`): a local TCP inbound whose listen address overlaps that port on loopback is rejected with the same port-conflict error. The actual API port is read from the Xray config template (with a fallback of `62789`). On nodes this restriction does not apply — they run their own Xray.

> The Xray tag (`Tag`, unique) is generated automatically from the port and transport in the format `in-<port>-<tcp|udp|tcpudp|any>`; for an inbound deployed on a node, the prefix `n<nodeId>-` is added. On a collision, `-2`, `-3`, etc. is appended to the tag. The user usually does not edit the tag.

Total traffic (GB)

Parameter	Value
Field	`total` (in **bytes**)
Label	**"Total flow"**
Default	`0`

The total traffic limit of the inbound. In the form, the value is entered in gigabytes; in the database it is stored in bytes. The field hint:

> "= Unlimited. (unit: GB)."

That is, **`0` means unlimited**. This is a limit at the level of the entire inbound (not individual clients); the actual consumed traffic is stored in the `up` (sent) and `down` (received) fields and compared against `total`.

Expiry date / Duration

Parameter	Value
Field	`expiryTime` (Unix timestamp)
Label	**"Expiry date"** (Duration)
Default	empty / `0`

The validity period of the inbound. The hint:

> "Leave blank to never expire."

An empty value (`0`) means the inbound never expires. The value is stored as a Unix timestamp; the form allows you to specify either a specific date or a period in days (a relative countdown from the current moment — the English field label is *Duration*).

Enabled

Parameter	Value
Field	`enable`
Label	**"Enable"** (Enabled)
Default	set at creation

The inbound's active flag. Toggling this flag in the list is handled by a separate "lightweight" endpoint `POST /setEnable/:id`, rather than by a full update — this is done deliberately to avoid re-serializing the entire `settings` block (all clients) on every click of the toggle on an inbound with thousands of clients. When an inbound is disabled, it is removed from the running Xray; when enabled, it is added back.

Node / Deploy to

Parameter	Value
Field	`nodeId`
Label	**"Deploy to"**, **"Local panel"**

Parameter	Value
Default	empty (local panel)

A choice of where the inbound physically runs: on the local panel or on one of the registered nodes. An implementation detail: `nodeId = 0` is normalized to `nil`, since `0` is not a valid node id but an artifact of form binding; `nil` / `0` means the local panel. When saving an inbound on an offline node, a toast is possible — the change is synchronized when the node reconnects. As of version 3.4.2 this toast is shown only when the node is actually offline or disabled (previously it could also appear when saving to an online node).

Share address strategy

Parameter	Value
Field	strategy + (optional) custom address
Label	**"Share address strategy"**
Default	**"Inbound listen"**

A drop-down list that controls which address is inserted into this inbound's **exported share links and QR codes**. The values:

Value	Label	What is inserted
`node`	**"Node address"**	the address of the node on which the inbound runs
`listen`	**"Inbound listen"**	the listen address of the inbound itself
`custom`	**"Custom"**	a custom address from the **"Custom share address"** field

When **"Custom"** is selected, the **"Custom share address"** field appears; enter a host or IP **without a scheme or port** (the value is validated). The **"Node address"** option is shown in the list only if there is an enabled node on which this inbound can run; otherwise it is hidden and the value is coerced to **"Inbound listen"**.

This strategy affects **only** the direct share links and QR codes. It does **not** affect subscription output — there the address is still resolved by the usual panel logic.

4.2. Sniffing

The **Sniffing** tab edits the `sniffing` block of the Xray configuration, which is stored as raw JSON. Sniffing allows Xray to "peek" at the real domain name/protocol inside a connection for routing purposes.

Subfield	Label	Purpose
`enabled`	(tab toggle)	Enables/disables sniffing for the inbound
`destOverride`	—	The list of protocols for which the destination address is intercepted: `http`, `tls`, `quic`, `fakedns`
`metadataOnly`	**"Metadata only"**	Use only the connection's metadata, without reading the payload

Subfield	Label	Purpose
`routeOnly`	**"Route only"**	Apply the sniffing result only for routing, without rewriting the destination address
`domainsExcluded`	**"Excluded domains"**	Domains excluded from sniffing
(excluded IPs)	**"Excluded IPs"**	IP addresses excluded from sniffing

- **`destOverride`** — the set of sniffers: `http` (determines the domain from the HTTP Host header), `tls` (from the SNI), `quic` (from the QUIC ClientHello), `fakedns` (matching against the FakeDNS pool). Usually `http` and `tls` are enabled to determine the domain.

Example of a `sniffing` block (determine the domain from HTTP and TLS, use the result for routing only, without rewriting the destination):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** — when enabled, Xray does not read the contents of the first packet and relies only on metadata; useful to avoid breaking protocols whose data cannot be "peeked" at.
- **`routeOnly`** — the sniffing result is used only by routing rules; the connection address in the outbound is not rewritten to the recognized domain.

> Note: the panel stores `sniffing` as an opaque JSON block and adds nothing to it when saving — all default values for these checkboxes are formed on the client-application side. The raw block can be edited via the "Inbound JSON" section (see below).

4.3. Allocate (port allocation strategy)

The `allocate` block in `streamSettings` controls how Xray allocates listening ports. This is part of the Xray configuration; the panel stores and passes it as part of the inbound's `streamSettings` /JSON. Parameters (per Xray-core terminology):

Subfield	Purpose	Values / default
`strategy`	Port allocation strategy	`always` — always listen on the specified port (default); `random` — periodically change the listened ports within a range
`refresh`	Port change interval (minutes) for `random`	an integer number of minutes (5 is recommended; minimum — 2)
`concurrency`	How many ports to keep open simultaneously for `random`	an integer (default 3; no more than a third of the width of the port range)

`strategy: always` keeps the inbound on a single port (the standard mode). `strategy: random` is needed for anti-blocking scenarios, where the inbound periodically "hops" across a port range; in that case `refresh` and `concurrency` make sense. You should change these values only when deliberately using the random-port mode.

Example of an `allocate` block in `streamSettings` (random-port mode: keep 3 ports open, change them every 5 minutes):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

For this to work, the inbound's `port` is set as a range (for example, `20000–20100`).

4.4. External Proxy

The **External Proxy** field relates to the settings for generating invitation links and is stored in the inbound's `streamSettings`. It specifies a list of alternative external addresses (host/port, optionally with forced TLS — "**Force TLS**") that are inserted into client links instead of the inbound's real `listen:port`.

It is used when clients should connect not directly to the server but through an external proxy/reverse/CDN: in that case the public address of such a frontend is specified in the shared links. This does not affect Xray's connection-accepting process itself — it is "cosmetics" of the generated links. Related form fields: "**Force TLS**", "**Fingerprint**", and a label for each entry.

4.5. Fallbacks

The **Fallbacks** section defines rules for redirecting connections that did not match any of the inbound's clients. It is available for a master inbound on a TLS transport (VLESS/Trojan TCP-TLS). It is managed via the endpoints `GET /:id/fallbacks` / `POST /:id/fallbacks`.

The section hint:

> "When a connection on this inbound does not match any client, it is redirected elsewhere. Select a child inbound below to auto-fill the routing fields (SNI / ALPN / Path / xver) from its transport, or leave the selection empty and set Dest directly (for example, 8080 or 127.0.0.1:8080) to redirect to an external server such as Nginx. Each child inbound must listen on 127.0.0.1 with security=none."

The fallbacks section is shown only for a VLESS/Trojan inbound over RAW (TCP) with TLS or REALITY security. A new inbound starts at `security=none`, so the section may at first look as if it is missing. In that state (VLESS/Trojan, RAW/TCP, security not yet configured), an inline hint is shown instead of the section: fallbacks become available once TLS or Reality is selected on the **Security** tab.

Fallback row fields

Field	Default	Description
(child inbound)	—	Selection of the child inbound (label **"Pick inbound"**). If selected, the Name/Alpn/Path/Dest fields may be auto-filled from its transport
Name	empty (= any)	Match condition on the name (SNI/name). The "any" label — **"any"**
Alpn	empty	Match condition on ALPN
Path	empty	Match condition on path (for the WS/HTTP transports of the child inbound)
Dest	auto	Where to redirect. Placeholder **"auto (child's listen:port)"**. You can specify a port (`8080`) or `host:port` (`127.0.0.1:8080`)
Xver	`0`	PROXY protocol version (**"Xver"**): `0` — disabled, `1` or `2` — the corresponding PROXY protocol version
(order)	by position	The order in which the rules are applied; set with the **Up**/**Down** buttons

Save logic: the entire fallback list of the master is replaced atomically. A row that has neither a selected child inbound (`childId <= 0`) nor a specified `Dest` is **skipped**. If the selected child inbound equals the id of the master itself, it is zeroed out. When generating the final JSON: if `Dest` is empty, it is computed from the child inbound as `listen:port` , with `0.0.0.0` / `::` / `::0` replaced with `127.0.0.1` ; empty `name` / `alpn` / `path` fields do not end up in the output JSON; `xver` is added only if it is greater than 0.

Example of the resulting `settings.fallbacks` (traffic with `alpn=h2` goes to a WS target at path `/ws` , everything else to a local Nginx on port 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

The last row, without `name` / `alpn` / `path` , is the "default" rule that catches everything else.

Buttons and hints of the fallbacks section

- **"Add fallback"** — add a row; **"No fallbacks yet"** — the empty state.
- **"Quick add all matching"** / **"Add all"** — adds a fallback row for every matching inbound that is not yet connected. Result: "Added {n} fallback(s)" or "No new matching inbounds".
- **"Fill from child"** — re-pull the routing fields (SNI/ALPN/Path/xver) from the transport of the selected child inbound; after execution — "Filled from child".
- **"Edit routing fields"** / **"Hide advanced"** — show/hide the fine-grained fields of the row.
- The labels **"Routes when"** and **"Default — catches everything else"** explain the trigger condition of each row.

After fallbacks are saved, the server triggers an Xray restart so that the new `settings.fallbacks` take effect.

4.6. Periodic traffic reset

The **Traffic reset** block configures automatic reset of the inbound's traffic counters on a schedule. Description:

> "Automatically reset the traffic counter at the specified intervals."

Parameter	Value
Field	`trafficReset`
Validation	`omitempty,oneof=never hourly daily weekly monthly`
Default	`never`
Companion field	`lastTrafficResetTime` — the timestamp of the last reset (label **"Last reset"**)

Drop-down list:

Value	Label
`never`	**"Never"**
`hourly`	**"Hourly"**
`daily`	**"Daily"**
`weekly`	**"Weekly"**
`monthly`	**"Monthly"**

For each period a cron job is registered that runs on the corresponding schedule (`@hourly` , `@daily` , `@weekly` , `@monthly`). The job selects all inbounds with the given `trafficReset` and, for each, resets the counters of the inbound itself (`up=0` , `down=0`) **and** the traffic of all its clients. That is, a periodic reset affects both the inbound and its clients.

Example field value. To have the counters zeroed on the first day of each month, select **"Monthly"** in the form, which is stored as:

```
{ "trafficReset": "monthly" }
```

The value `never` (the default) disables auto-reset entirely.

4.7. Inbound JSON (advanced)

The **Inbound JSON sections** section gives direct access to the inbound's raw JSON blocks. Description:

> "The full inbound JSON and individual editors for settings, sniffing and streamSettings."

The following editors are available:

Tab	Label	What it edits
All	"The full inbound object with all fields in a single editor"	the entire Inbound object
Settings	"Wrapper for the Xray settings block"	the `settings` field
Sniffing	"Wrapper for the Xray sniffing block"	the `sniffing` field
Stream	"Wrapper for the Xray stream block"	the `streamSettings` field

These fields are serialized as nested JSON objects: empty blocks are returned as `null`, and text that is not valid JSON is wrapped in a string so that data is not lost. Parsing errors on save are shown with the prefix "**Advanced JSON**".

The "Inbound JSON" viewer, like the inbound import dialog, uses a full code editor with JSON syntax highlighting (instead of a plain text area): the configuration view is a highlighted read-only mode, while import is editable — which makes reading and editing the configuration easier.

4.8. Inbound actions: QR / Edit / Reset / Delete and statistics

The following actions are available in the list and in the inbound card (the "**Menu**" menu):

Traffic statistics

The inbound's aggregated traffic is displayed: "**Sent/received**" (the `up` / `down` fields), "**Total traffic**", "**Total connections**". The card also shows "**Created**", "**Updated**", "**Expiry date**".

The inbounds list has a **Speed** column showing each inbound's current traffic speed (upload/download), computed from the counter deltas between polls; the same live speed is shown in the inbound stats modal. When a poll returns no delta, the speed value is cleared.

In the client summary on the inbounds page, status is determined depleted-first: clients that have expired or exhausted their traffic (and that the auto-job cleared `enable` for) fall into the "**Depleted/Ended**" status rather than the grey "**Disabled**" one, and are not counted twice. The classification matches the one shown in the client's own card and correctly accounts for clients attached to several inbounds.

QR code and copying links

- "**Details**" — expands the connection and subscription links.
- Client QR code: hint "**Click the QR code to copy**".
- "**Copy link**" (*Copy URL*), "**Export links**". As of version 3.4.2 the page-level "**Export all links**" action (`GET /panel/api/inbounds/allLinks` ; the "Export all inbound links" window, file "All-Inbounds") builds links through the subscription engine — it applies the remark template to each client and prefers managed Host endpoints (previously a fixed `inbound-email` remark was used).

Edit

"Modify inbound" — opens the editing form (`POST /update/:id`). On update, the service re-reads the existing row, carries over the changed fields, regenerates the tag if necessary (if the old tag was auto-generated) and synchronizes the Xray runtime. Success — toast **"Inbound updated successfully"**.

Reset Traffic

"Reset traffic" — zeroes the `up` / `down` counters of this specific inbound (`POST /:id/resetTraffic` , sets `up=0, down=0`). Confirmation:

> "Reset traffic for "{remark}"?" / "Resets the sent/received counters of this inbound to 0."

Resetting an inbound's traffic does **not** touch its clients' counters (there are separate "Reset clients' traffic" actions for those). After the reset, an Xray restart is initiated. Success — toast **"Inbound traffic reset"**. There is also a bulk variant — **"Reset traffic of all inbounds"** (`POST /resetAllTraffics`).

Delete

"Delete inbound" (`POST /del/:id`). Confirmation:

> "Delete inbound "{remark}"?" / "The inbound and all its clients will be deleted. This action cannot be undone."

Deletion removes the inbound from the running Xray (with a restart if necessary). Success — toast **"Inbound deleted successfully"**. Bulk deletion — `POST /bulkDel` , with per-item reporting and no more than one Xray restart.

Other actions with inbound clients

The menu also provides: **"Clone"** (a copy of the inbound with a new port and an empty client list), **"Delete all clients"** (`POST /:id/delAllClients` — deletes all clients, the inbound itself is kept), **"Delete depleted clients"**, **"Attach/Detach clients"**, **"Import"/"Export inbounds"** (`POST /import`). Details of client operations belong to the section on clients.

---

5. Protocols

When creating an inbound, the first thing you choose is the **Protocol**. The protocol determines which authentication and traffic-encryption method Xray-core applies to the inbound, which fields in `settings` need to be filled in, and which transports (`network`) and security types (TLS / REALITY) are available for it.

The protocol field is set once when the inbound is created and **cannot be changed during editing** (the drop-down list is locked in the edit form). To change the protocol, create a new inbound.

5.1. List of Supported Protocols

The server accepts the following set of values for the `Protocol` field:

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

> Starting with version **3.3.0**, the value `mtproto` (Telegram proxy) was added to the list.

Config value	Purpose	Client model
`vless`	Primary proxy protocol (default when creating an inbound)	Clients with UUID, flow and post-quantum encryption support
`vmess`	Classic Xray proxy protocol	Clients with UUID and `security` parameter
`trojan`	Proxy disguised as regular HTTPS	Clients with a password
`shadowsocks`	Shadowsocks proxy (including SIP022 / 2022-blake3)	Single user or multiple users (2022)
`wireguard`	WireGuard inbound	Peers (not clients)
`hysteria`	Hysteria inbound (version 2 by default)	Clients with an `auth` token
`http`	Classic HTTP forward proxy	User/pass accounts, no traffic accounting
`mixed`	Combined SOCKS + HTTP proxy	User/pass accounts
`tunnel`	Transparent forwarder (xray `dokodemo-door`)	No clients
`tun`	TUN interface (rendering of existing ones only)	No clients
`mtproto`	Telegram proxy (MTProto), added in 3.3.0; handled by a separate `mtg` process, not Xray	No clients (access via secret)

> Note on `tun` : the value is kept in the list for compatibility and **display** of previously saved inbounds, but in the current backend version creating new ones is not recommended — support is considered deprecated. There is no point in creating new inbounds of this type.

> Note on Hysteria 2: there is no separate "hysteria2" protocol. It is the `hysteria` protocol with `streamSettings.version = 2`. The `hysteria2://` share-link scheme is selected automatically when the stream version equals 2.

Not all protocols support deployment to nodes. Only the following can be deployed to nodes: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. The protocols `http`, `mixed`, `tunnel`, `tun`, `mtproto` work on the local panel only.

5.2. Which Protocols Support TLS / REALITY / Transport

The ability to enable a particular security layer and transport depends on the protocol and the chosen network (`streamSettings.network`):

Capability	Available for protocols	Allowed networks (`network`)
TLS	`vmess`, `vless`, `trojan`, `shadowsocks` (and always for `hysteria`)	`tcp`, `ws`, `http`, `grpc`, `httpupgrade`, `xhttp`
REALITY	`vless`, `trojan`	`tcp`, `http`, `grpc`, `xhttp`
flow (`xtls-rprx-vision`)	`vless` only	`tcp` only, with `security = tls` or `reality`
Stream / transport (Transport tab)	`vmess`, `vless`, `trojan`, `shadowsocks`, `hysteria`	—

For the protocols `http`, `mixed`, `tunnel`, `tun`, `wireguard` the transport tab is unavailable — they have no Xray stream settings.

5.3. VLESS

Purpose: the primary modern proxy protocol. Supports XTLS-Vision (`flow`), REALITY, and post-quantum encryption at the VLESS level itself (`decryption` / `encryption` fields). Used by default for new inbounds.

`settings` block fields:

Field	Default	Description
`clients`	`[]`	List of clients. Each has: `id` (UUID), `email` (required), `flow`, limits (`limitIp`, `totalGB`, `expiryTime`), `enable`, `tgId`, `subId`, `comment`, `reset`
`decryption`	`none`	Server-side decryption parameter. UI label: «Decryption»
`encryption`	`none`	Paired encryption parameter (goes into the client link). Label: «Encryption»
`fallbacks`	`[]`	List of fallbacks (see the fallbacks section); available when `network = tcp` and `security` = TLS or REALITY
`testseed`	(4 numbers: 900, 500, 900, 256)	«Vision testseed» — 4 positive integers for XTLS-Vision padding. Applied only to clients with flow `xtls-rprx-vision`, otherwise ignored

flow (`xtls-rprx-vision`)

`flow` is set **on the client**, not on the inbound, and takes one of three values:

Value	Meaning
`` (empty)	No XTLS flow (default)
`xtls-rprx-vision`	XTLS-Vision — recommended mode for VLESS over TCP+TLS/REALITY
`xtls-rprx-vision-udp443`	Same as Vision, but with UDP/443 (QUIC) handling

The `flow` field is available for selection only when all conditions are met: protocol `vless`, `network = tcp`, and `security` = `tls` or `reality`. The **Vision testseed** field in the form is shown only under the same conditions.

> Exception for XHTTP: with VLESS over `network = xhttp` and VLESS post-quantum authentication enabled (`encryption` / `decryption` , vlessenc), flow `xtls-rprx-vision` is also allowed — regardless of the security layer, including with REALITY. In this case the panel correctly passes `xtls-rprx-vision` into share links and subscriptions (including the Clash/Mihomo format), so the client receives a configuration with Vision.

Decryption / Encryption (VLESS post-quantum authentication)

The `decryption` and `encryption` fields provide authentication at the VLESS level itself (separate from the transport TLS/REALITY). By default both are `none`. In the form, below these fields there is a **Key Generation** block — a drop-down with the mode and a **Generate** button (with a **Clear** button next to it). The drop-down contains six options: **X25519 (native)**, **X25519 (xorpub)**, **X25519 (random)**, **ML-KEM-768 (native)**, **ML-KEM-768 (xorpub)**, **ML-KEM-768 (random)** — that is, two key types (classic X25519 and post-quantum ML-KEM-768), each in three modes:

- **native** — a basic key pair of the selected type;
- **xorpub** — a derived mode with additional processing of the public part;
- **random** — a derived mode with a random component.

Select the desired mode in the list and click **Generate**: the panel will fill **both** fields (`decryption` and `encryption`) with a ready-made pair of values for that mode. The **Clear** button resets both fields back to `none`.

Below the block, a status line **«Selected: ...»** is displayed; it recognises from the generated string both the key type (X25519 or ML-KEM-768) and the mode (native / xorpub / random) and shows them. Empty fields or `none` are displayed as «None».

Technically, the buttons call `GET /panel/api/server/getNewVlessEnc` (key generation via `xray vlessenc`) and fill **both** fields with paired values of the form `mlkem768x25519plus.native.<rtt>.<role>` (for example, `decryption = mlkem768x25519plus.native.600s.server-x25519` , `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). The `decryption` parameter stays on the server; `encryption` goes into the client link.

> Important: when generating the inbound configuration for Xray, the panel strips the extra field: if `encryption` (which belongs to the client side) remains in `settings`, it is **removed** from the server config. Only `decryption` remains on the server.

When to choose VLESS: this is the recommended default for a new inbound, especially combined with REALITY (no own certificate) or TLS + XTLS-Vision.

Example: `settings` block of a VLESS inbound with one client and XTLS-Vision. The `flow` field is on the client; `decryption` stays on the server:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

For the REALITY combination, the corresponding `streamSettings` block (Transport tab → Security: REALITY) looks like this:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<X25519 private key>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Purpose: classic Xray proxy protocol. Authentication by UUID; the payload encryption method (`security`) is additionally configured on the client.

`settings` block fields:

Field	Default	Description
`clients`	`[]`	List of clients

Each VMess client (in addition to the common fields `email`, limits, `enable`, `tgId`, `subId`, `comment`, `reset`):

Client field	Default	Description
`id`	—	Client UUID
`security`	`auto`	VMess payload encryption method. Allowed values: `aes-128-gcm`, `chacha20-poly1305`, `auto`, `none`, `zero`

`security` values:

- `auto` — Xray selects the cipher based on the platform (recommended);
- `aes-128-gcm`, `chacha20-poly1305` — fixed AEAD ciphers;
- `none` — no payload encryption (makes sense only over TLS);
- `zero` — no payload encryption and no payload authentication.

> Historical compatibility: old records may have stored `security: ""` — on reading, an empty string is normalised to `auto`. When generating the server config, the `security` field of VMess clients is **removed** from `settings`, as it is not required for the inbound.

When to choose VMess: for compatibility with old clients or existing configurations. For new deployments VLESS is usually preferable.

5.5. Trojan

Purpose: proxy that imitates regular HTTPS traffic. Authentication by password. Like VLESS, it supports fallbacks and (when `network = tcp`) REALITY/TLS.

`settings` block fields:

Field	Default	Description
`clients`	`[]`	List of clients
`fallbacks`	`[]`	List of fallbacks (available with `network = tcp` and TLS/REALITY)

The key field of each Trojan client:

Client field	Default	Description
`password`	—	Client password (required, minimum 1 character)
`email`	—	Unique client identifier

The remaining client fields are common (`limitIp`, `totalGB`, `expiryTime`, `enable`, `tgId`, `subId`, `comment`, `reset`).

When to choose Trojan: when you need HTTPS masquerading on port 443, including with fallbacks to a web server (Nginx) for unsolicited connections.

Example: Trojan `settings` block with a fallback to a local web server. Unsolicited connections (without a valid password) are forwarded to Nginx listening on `127.0.0.1:8080`:

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Fallbacks require `network = tcp` and Security = TLS or REALITY; otherwise the fallbacks field is not available.

5.6. Shadowsocks

Purpose: lightweight Shadowsocks proxy. Supports both legacy AEAD ciphers and the new SIP022 methods (`2022-blake3-*`). Can operate in single-user or multi-user mode.

`settings` block fields:

Field	Default	Description
`method`	`2022-blake3-aes-256-gcm`	Inbound encryption method. UI label: «Encryption method»
`password`	``	Inbound password (for 2022 methods, generated automatically to match the selected method)
`network`	`tcp,udp`	Transport. Label: «Network». Options: `tcp,udp` (TCP, UDP), `tcp`, `udp`
`clients`	`[]`	List of clients
`ivCheck`	`false` (off)	«ivCheck» toggle — protection against IV reuse

Encryption methods (`method`)

Allowed set:

Method	Category
`aes-256-gcm`	Legacy AEAD
`chacha20-poly1305`	Legacy AEAD
`chacha20-ietf-poly1305`	Legacy AEAD
`xchacha20-ietf-poly1305`	Legacy AEAD

Method	Category
`2022-blake3-aes-128-gcm`	SS 2022 (recommended)
`2022-blake3-aes-256-gcm`	SS 2022 (default)
`2022-blake3-chacha20-poly1305`	SS 2022, single-user

Panel logic by method:

- **2022 methods** (`2022-blake3-*`) are considered «SS 2022». The method `2022-blake3-chacha20-poly1305` is **single-user** (multi-user is not supported); other 2022 methods allow multiple clients. The password field (with a generation button that adjusts key length to the method) is shown in the form specifically for 2022 methods.
- **Legacy ciphers** (`aes-*` , `chacha20-*`) work via the classic «one method + one password» scheme.

> Normalisation before Xray starts: for legacy ciphers, each client must carry a `method` matching the inbound method (otherwise Xray crashes with «unsupported cipher method:»). For 2022 methods it is the opposite — the `method` field of the client must be **empty** (otherwise Xray rejects the inbound with «users must have empty method»). The panel normalises the data automatically when the method is switched.

> Client key regeneration on key-size change: for Shadowsocks-2022, when switching the encryption method to one with a different key size (for example between `2022-blake3-aes-256-gcm` and `2022-blake3-aes-128-gcm`), the panel automatically regenerates client PSKs to the new length when the inbound is saved. Otherwise the old keys would remain at the previous length, and Xray would reject them. Consequence: affected clients need to re-fetch their subscription — old links will stop connecting.

When to choose Shadowsocks: for simple deployments without TLS masquerading; the modern choice is the `2022-blake3-*` methods.

Example: Shadowsocks `settings` block for a 2022-blake3 method (multi-user mode). The inbound has its own password (a base64 key of the required length); each client has its own password; the client `method` field is **empty**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWhlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMi1ieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

For legacy ciphers (`aes-256-gcm` etc.) it is the opposite: one password for the inbound, and the client `method` must match the inbound method.

5.7. Dokodemo-door / Tunnel (Transparent Forwarder)

Purpose: transparent forwarder (in the panel — the `tunnel` protocol, implementing `dokodemo-door` behaviour). Accepts traffic and forwards it to a specified address/port, without authentication or clients.

`settings` block fields:

Field	Default	Description
`rewriteAddress`	(none)	«Rewrite address» — the destination address to which traffic is redirected
`rewritePort`	(none)	«Rewrite port» — destination port (0–65535)
`allowedNetwork`	`tcp,udp`	«Allowed network». Options: `tcp,udp` , `tcp` , `udp`
`portMap`	`{}`	«Port map» — port-to-port map (Record<string,string>)
`followRedirect`	`false` (off)	«Follow redirect» — use the original destination address from the intercepted connection

> Transport tab for Tunnel: inbounds of this type have a **Transport** tab, limited to the `sockopt` setting — this is sufficient for **TProxy** mode (transparent proxying/redirect via `sockopt.tproxy`). The transport selection drop-down (`network`) and the Security tab for Tunnel are hidden, as TLS/REALITY is not supported by this type.

When to choose: for transparent proxying / port forwarding to internal services.

The «Rewrite port» (`rewritePort`) field can be left empty: when cleared, the value is simply excluded from the inbound settings rather than causing a save error. (Previously, clearing this field caused a `settings.rewritePort` validation error that blocked saving, including through the JSON tab.)

5.8. SOCKS / HTTP (`mixed` protocol)

In this build there is no separate `socks` protocol — SOCKS and HTTP proxies are combined into the **`mixed`** protocol (combined SOCKS + HTTP). There is also a separate pure `http` proxy.

5.8.1. Mixed (SOCKS + HTTP)

`settings` block fields:

Field	Default	Description
`auth`	`password`	«Auth» — authentication mode. Options: `password` (username/password) or `noauth` (no authentication)

Field	Default	Description
`accounts`	(optional)	«Accounts» — list of user/pass pairs. With `auth = noauth` the field is not written to the config
`udp`	`false` (off)	«UDP» toggle — UDP support via SOCKS
`ip`	`127.0.0.1`	«UDP IP» — local address for UDP associations. The field is shown only when `udp` is enabled

Accounts are added via the «Add» button; a random username (8 characters) and password (12 characters) are generated on add and can be edited.

5.8.2. HTTP (pure proxy)

Purpose: classic HTTP forward proxy. At the Xray level it does not track clients as «billing» entities (no email/limits) — there is only a list of accounts.

`settings` block fields:

Field	Default	Description
`accounts`	`[]`	«Accounts» — list of user/pass pairs (both fields required)
`allowTransparent`	`false` (off)	«Allow transparent» — forward requests with the original Host header

When to choose SOCKS/HTTP: for local or service proxy access without complex masquerading. `mixed` is convenient because one port serves both SOCKS and HTTP clients.

5.9. WireGuard (inbound)

Purpose: WireGuard inbound — a WireGuard VPN tunnel, not a disguised proxy. Transport and TLS/REALITY do not apply to it.

As of version 3.4.2 WireGuard has been moved to a multi-client model (like VLESS/VMess/Trojan/Shadowsocks/Hysteria). The inbound itself stores only the server part; peer devices are now added as regular **clients** (see section 8) — with automatic in-tunnel address assignment, subscription support, traffic/expiry limits and groups. The former inline "Peers / Add peer" list has been removed from the inbound form: a new inbound is created without peers, and each client is assigned an address automatically.

Inbound fields (`settings` block):

Field	Default	Description
`secretKey`	—	Server private key (required). A generation button is next to it; the public key (`Public Key`) is derived from it automatically (read-only; the separately stored `pubKey` has been removed)
`mtu`	`1420`	Interface MTU

Field	Default	Description
`dns`	`1.1.1.1, 1.0.0.1`	**New in 3.4.2.** The DNS written into the `DNS =` line of the client `.conf`
`noKernelTun`	`false` (off)	«No-kernel TUN» — userspace TUN instead of the kernel TUN
`domainStrategy`	(optional)	«Domain Strategy»: `ForceIP`, `ForceIPv4`, `ForceIPv4v6`, `ForceIPv6`, `ForceIPv6v4`

WireGuard client parameters — the **"Credentials"** tab in the add/edit client window (shown when the bound inbound is WireGuard):

Field	Description
"WireGuard private key"	Client private key (editable, with a "Regenerate" button); when entered, the public key is derived from it automatically
"WireGuard public key"	Read-only; computed from the private key
"WireGuard pre-shared key" (PSK)	Optional additional pre-shared key
"WireGuard allowed IPs"	Read-only (in edit mode): the in-tunnel address assigned to the client, e.g. `10.0.0.2/32`

The in-tunnel address is assigned by the server automatically from the inbound's subnet (default `10.0.0.0/24`: the server takes `.1`, clients start from `.2`); if existing clients use a different `/24`, new addresses are assigned in the same subnet.

Domain Strategy and the Transport Tab

In addition to the above, the WireGuard inbound has a **Domain Strategy** field (domain resolution strategy: `ForceIP`, `ForceIPv4`, `ForceIPv4v6`, `ForceIPv6`, `ForceIPv6v4`). The field is optional and is written to the config only if set.

> The **Workers** field (`workers`, number of worker threads) has been removed from WireGuard forms (both inbound and outbound): starting with xray-core v26.6.22 the engine no longer uses it and relies on the internal wireguard-go mechanism. Previously saved configs work without changes — the field is simply discarded during parsing; no migration is needed.

The **Transport** tab is also available for WireGuard — but in a reduced form: only `sockopt` and **Finalmask** obfuscation can be configured there. The transport selection drop-down (`network`) is hidden because WireGuard always listens on UDP. In the noise records (noise), Finalmask has a separate **Rand Range** field (byte range 0–255, with validation), and **Salamander** is available as an obfuscation method for WireGuard and Hysteria.

WireGuard client configuration and sharing

In a WireGuard client's "Info"/QR windows there is a **collapsible configuration card**: the full `.conf` (`[Interface]` with `PrivateKey` / `Address` / `DNS` / `MTU` and `[Peer]` with `PublicKey` / `PresharedKey` /

`AllowedIPs = 0.0.0.0/0, ::/0` / `Endpoint` / `PersistentKeepalive`) with **Copy**, **Download** and **QR** buttons. A `wireguard://` / `wg://` link is also supported, which the client application parses into a ready `.conf`.

When to choose WireGuard: when you need a WireGuard VPN tunnel specifically, not a disguised proxy.

5.10. Hysteria (v2 by default)

Purpose: Hysteria inbound over QUIC. The panel works with version 2 by default. Each client authenticates with an `auth` token instead of a UUID/password. TLS is always available for Hysteria (see the capability table in 5.2).

`settings` block fields:

Field	Default	Description
`version`	`2`	Protocol version (minimum 1; panel default is 2)
`clients`	`[]`	List of clients

The key field of each client is `auth` (token, required) plus the common fields (`email`, limits, `enable`, `tgId`, `subId`, `comment`, `reset`).

Additional parameters are set in `streamSettings.hysteriaSettings`:

Field	Value / options	Description
`version`	fixed 2 (field locked)	«Version»
`udpIdleTimeout`	(integer ⩾ 1, seconds)	«UDP idle timeout (s)» — UDP idle timeout
`masquerade`	disabled by default	«Masquerade» — disguise as a regular web server for unsolicited requests

When `masquerade` is enabled, a type (`type`) can be selected:

- `` — default (404 page);
- `proxy` — reverse proxy (fields «Upstream URL», «Rewrite Host», «Skip TLS verify»);
- `file` — serve a directory (field «Directory», e.g. `/var/www/html`);
- `string` — fixed response (fields «Status code», «Body», «Headers»).

When to choose Hysteria: when you need QUIC transport and resilience on unstable/mobile connections; masquerading increases the stealth of the entry point.

5.11. MTProto (Telegram Proxy)

> Added in version **3.3.0**. Protocol value — `mtproto`.

MTProto is Telegram's own proxy protocol. In 3X-UI, such an inbound is **handled not by Xray but by a separate `mtg` process** managed by the panel itself. The panel periodically reconciles enabled MTProto inbounds with

running `mtg` processes: it starts missing ones, stops extra ones, and collects traffic counters from `mtg` metrics. As a result, **traffic accounting** for such an inbound works just like for regular protocols.

Official hint in the form:

> «MTProto is handled by a separate mtg process, not Xray. Transport settings and clients do not apply here — share the link below in Telegram.»

Consequences:

- The **Transport (Stream Settings) and Clients tabs do not apply to this inbound** — access is defined by a single secret, not a list of clients.
- An MTProto inbound runs **on the main panel only**; it is not deployed to child nodes (inbounds with a `NodeID` set are skipped).
- The **Sniffing** tab for MTProto is hidden — this protocol is handled by the `mtg` process, not Xray, so sniffing does not apply to it.

Fields. Stored in the inbound `settings`:

UI field	Key	Description
Remark	`remark`	Inbound label.
Listen IP	`listen`	IP to listen on (empty = all interfaces).
Port	`port`	Proxy port.
Secret	`settings.secret`	Access secret in **FakeTLS** format.
FakeTLS domain	`settings.fakeTlsDomain`	Domain whose HTTPS traffic the proxy impersonates.

Secret format (FakeTLS). The panel automatically normalises the secret to the correct form: result = `ee` + 32 hex characters + hex-encoded cover domain, i.e. `ee<hex32><hex(fakeTlsDomain)>`. The `ee` prefix enables FakeTLS mode, and the domain (for example, a well-known site) is used to disguise traffic as regular HTTPS. Simply specify the domain — the panel will build the rest automatically.

Domain Fronting and Advanced mtg Options

MTProto inbounds have additional `mtg` process parameters. The fields **Domain fronting IP**, **Domain fronting port**, and **Domain fronting PROXY protocol** specify where `mtg` forwards non-Telegram traffic (for example, to a fake NGINX site): if the IP is left empty, the FakeTLS domain is used via DNS, and the default port is `443`. Additionally available are **Accept PROXY protocol** (for the listener), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`), and **Debug logging**. Each value is written to the `mtg-<id>.toml` file only if it is set.

Routing Telegram Traffic through Xray

The **Route through Xray** toggle (off by default) and the optional **Outbound** field allow you to route Telegram egress through the Xray router. When enabled, the panel injects a local SOCKS bridge tagged with the inbound's own tag into the Xray config, and `mtg` sends Telegram traffic through it. The traffic can then be matched by rules on the «Routing» tab, or forced into a specific outbound or load balancer via the **Outbound** field (if the field is empty, routing rules decide).

How to share with users. For an MTProto inbound the panel generates an invite link:

Example: FakeTLS secret and ready-made link. If the cover domain is `www.cloudflare.com`, the secret is assembled as `ee` + 32 hex characters + hex-encoded domain, for example:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Ready-made invite link (send this and the QR code to the user in Telegram):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f75646
66c6172652e636f6d
```

```
tg://proxy?server=<address>&port=<port>&secret=<secret>
```

(equivalent — `https://t.me/proxy?server=…&port=…&secret=…`). Send this link and QR code to the Telegram user — when opened, the proxy is added to the app immediately. The link is also served via the subscription server.

When to use. The standard way to bypass Telegram blocks; FakeTLS masquerading (cover domain) makes traffic look like a regular visit to the specified site.

5.12. Quick Protocol-Selection Reference

- **VLESS** — the default choice; best with REALITY or TLS + XTLS-Vision, supports post-quantum authentication.
- **Trojan** — HTTPS masquerading with fallbacks to a web server.
- **VMess** — compatibility with old clients.
- **Shadowsocks** — simple proxy without TLS; the modern choice is `2022-blake3-*` methods.
- **Hysteria** — QUIC, resilience on poor connections.
- **mixed / http** — utility SOCKS/HTTP proxies.
- **WireGuard** — full VPN tunnel.
- **tunnel** — transparent port forwarding.
- **MTProto** — proxy for bypassing Telegram blocks (FakeTLS); separate `mtg` process.

---

6. Transport (Stream Settings)

The transport (in the panel UI — the **"Transport"** field, *Transmission*) defines how Xray-core carries data inside an inbound: which network protocol is used on top of TLS/Reality and exactly how the traffic is framed. These parameters are stored in the `streamSettings` object of the Xray configuration and are set on the transport tab of the inbound editor. Encryption (TLS / Reality) is covered in a separate section — here only the network choice and its parameters are described.

6.1. Choosing the transmission network

The network is selected from the **"Transport"** drop-down (`streamSettings.network`). The default value is `tcp` (shown in the list as **RAW**). The following options are available:

Value in the list	`network` field	Transport
RAW	`tcp`	Plain TCP (renamed to RAW in newer Xray versions), optionally with HTTP obfuscation
mKCP	`kcp`	Reliable UDP transport mKCP
WebSocket	`ws`	WebSocket over HTTP(S)
gRPC	`grpc`	gRPC tunnel (HTTP/2)
HTTPUpgrade	`httpupgrade`	HTTP Upgrade
XHTTP	`xhttp`	XHTTP / SplitHTTP — a modern multiplexable transport

When the value is changed, the panel clears the settings block of the previous network and fills the new network's block with the default values from its schema, so every field of the sub-form always has a meaningful initial value.

> **Important.** In this panel build the **HTTP/2 (`h2`) transport is absent from the list** — it was removed from the set of networks; for a bidirectional HTTP/2-like tunnel use gRPC, and for a modern HTTP-masked transport use XHTTP. The **Hysteria** transport (`hysteria`) is not selected through this list: it is rigidly tied to the Hysteria protocol and appears automatically when the inbound itself is created with the Hysteria protocol (see section 6.8).

Below, each network and each of its fields is examined separately.

6.2. RAW / TCP (`tcpSettings`)

The basic TCP transport. By default the traffic is transmitted "as is"; optionally it can be disguised as an ordinary HTTP/1.1 exchange.

Field	Default value	Description
Proxy Protocol (`acceptProxyProtocol`)	`false` (off)	Accept a PROXY protocol header from an upstream load balancer/proxy
HTTP obfuscation (`header.type`)	`none` (off)	Enables disguising the traffic as HTTP/1.1

Proxy Protocol

The "**Proxy Protocol**" toggle (`acceptProxyProtocol`). When enabled, Xray expects a PROXY protocol header on the incoming connection and extracts the client's real IP from it. Enable it only if a reverse proxy/load balancer sits in front of the panel (for example, HAProxy or nginx with `send-proxy`) that adds this header. Off by default.

HTTP obfuscation (camouflage)

The "**HTTP Obfuscation**" toggle. It controls the `header` field:

- **Off** → `header.type = "none"` (on the wire the `header` field is simply absent). Plain TCP.
- **On** → `header.type = "http"` . The traffic is framed to look like an HTTP/1.1 request and response. When enabled, the panel immediately fills the `request` and `response` sub-objects with default values.

When HTTP obfuscation is enabled, fields for configuring the simulated request and response appear.

Request header (`header.request`):

Field	Key	Default value	Description
Request version	`request.version`	`1.1`	The HTTP version in the request start-line
Request method	`request.method`	`GET`	The HTTP method of the simulated request
Request path	`request.path`	`/`	Path(s). Entered as a comma-separated list of values; on the wire this is an array of strings. If left empty, `/` is substituted
Request headers	`request.headers`	`{}` (empty)	A "Name/Value" table of HTTP headers. Stored as a map `name → [values]` (one name may have several values)

Response header (`header.response`):

Field	Key	Default value	Description
Response version	`response.version`	`1.1`	The HTTP version in the response start-line
Response status	`response.status`	`200`	The HTTP status code of the simulated response

Field	Key	Default value	Description
Response reason	`response.reason`	`OK`	Textual description of the status (reason-phrase)
Response headers	`response.headers`	`{}` (empty)	A "Name/Value" table of response headers (map `name → [values]`)

The header fields are edited line by line — each line specifies a header name (`Name`) and its value (`Value`). These parameters are used only to disguise the appearance of the traffic; they do not affect cryptography. The default values (`GET / HTTP/1.1`, response `200 OK`) suit most scenarios — change them only if you need to mimic a specific site/service.

Example `streamSettings` for RAW with HTTP obfuscation:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Note: on the wire `path` is an array of strings, and each header is an array of values (`Host → ["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP is a reliable transport over UDP. It is useful on links with packet loss and high latency, but it produces increased overhead traffic. All default values match those recommended in xray-core.

Field	Key	Default	Allowed	Description
MTU	`mtu`	`1350`	576–1460	Maximum packet size (bytes). Reduce it when there are fragmentation problems

Field	Key	Default	Allowed	Description
TTI (ms)	`tti`	`20`	10–100	Transmission interval (ms). Lower — lower latency, but higher overhead
Uplink (MB/s)	`uplinkCapacity`	`5`	⩾ 0	Estimated upload throughput (MB/s)
Downlink (MB/s)	`downlinkCapacity`	`20`	⩾ 0	Estimated download throughput (MB/s)
CWND multiplier	`cwndMultiplier`	`1`	⩾ 1	Congestion window multiplier
Max sending window	`maxSendingWindow`	`2097152`	⩾ 0	Maximum sending window size

Notes on the fields:

- **Uplink / Downlink capacity** define how aggressively mKCP occupies the channel. They are set according to the actual channel width: overstated values lead to wasted traffic, understated ones to underutilization of the channel.
- **TTI** directly affects the "latency ⍰ overhead" trade-off: smaller values reduce latency but increase the volume of overhead packets.
- **MTU** limits the size of a single mKCP packet; lowering it helps on links where large UDP packets are cut or lost.

> In this build the "seed" field (the mKCP obfuscation password) and the drop-down for the **header/obfuscation type** (`none` , `srtp` , `utp` , `wechat-video` , `dtls` , `wireguard`) in the mKCP sub-form **are absent as separate fields** — transport-layer obfuscation has been moved into the common "FinalMask" mechanism (including the `mkcp-legacy` mode), described in the corresponding section. The "congestion" parameter as a separate checkbox is also not exposed; congestion control is set via `cwndMultiplier` and `maxSendingWindow` .

6.4. WebSocket (`wsSettings`)

WebSocket transport over HTTP(S). It passes well through CDNs and reverse proxies and is disguised as ordinary web traffic.

Field	Key	Default	Description
Proxy Protocol	`acceptProxyProtocol`	`false`	Accept a PROXY protocol header from an upstream proxy (see section 6.2)
Host	`host`	`""` (empty)	The value of the HTTP `Host` header. Set it when working through a CDN/domain fronting
Path	`path`	`/`	The path in the request line of the WebSocket handshake
Heartbeat period	`heartbeatPeriod`	`0`	Interval for sending heartbeat frames (in seconds). `0` disables heartbeat

Field	Key	Default	Description
Headers	`headers`	`{}` (empty)	Additional HTTP handshake headers. A "Name → Value" map (string values only, no arrays)

Notes:

- **Path** must match on the server (inbound) and on the client. Often the entry point is disguised behind this path on the web-server side.
- **Host** is worth setting if the inbound sits behind a CDN or domain fronting is used; otherwise it can be left empty.
- **Heartbeat period** keeps the connection "alive" when passing through proxies/CDNs that aggressively drop inactive sessions. By default (`0`) heartbeat is off.
- Unlike RAW, the WebSocket header table uses a "flat" `name → value` format (one value line per header).

Example `streamSettings` for WebSocket behind a CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

The `host` and `path` values must match on the client; unlike RAW, the header value here is a plain string, not an array.

6.5. gRPC (`grpcSettings`)

The "lightest" transport in terms of the number of parameters. It tunnels traffic inside gRPC calls (over HTTP/2); it is well compatible with CDNs that support gRPC. There is no header obfuscation.

Field	Key	Default	Description
Service Name (`Service Name`)	`serviceName`	`""` (empty)	The gRPC service name (effectively the tunnel "path"). Must match on the server and the client
Authority	`authority`	`""` (empty)	The value of the `:authority` pseudo-header (the HTTP/2 equivalent of `Host`). Set it when working through a CDN/domain
Multi Mode	`multiMode`	`false` (off)	Enables multiplexing several parallel gRPC streams within a single connection

Notes:

- **Service Name** is the main identifier of the gRPC channel; it must be the same on both sides. An empty value is allowed, but a non-obvious string is usually set for disguise.
- **Authority** affects which `:authority` is sent in HTTP/2 frames; it is needed primarily when proxying through a CDN.
- **Multi Mode** allows several logical streams to go through a single physical connection; enable it to improve performance when both the server and the client support it.

Example `streamSettings` for gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

The `serviceName` (here `GunService`) acts as the tunnel "path" and must match on the server and the client.

6.6. HTTPUpgrade (`httpupgradeSettings`)

A transport based on the HTTP `Upgrade` mechanism (like WebSocket, but without the WebSocket protocol itself). It also passes well through proxies and CDNs. The set of fields repeats WebSocket, but **without** the heartbeat period.

Field	Key	Default	Description
Proxy Protocol	`acceptProxyProtocol`	`false`	Accept a PROXY protocol header from an upstream proxy
Host	`host`	`""` (empty)	The value of the HTTP `Host` header
Path	`path`	`/`	The path of the HTTP request with the `Upgrade` header
Headers	`headers`	`{}` (empty)	Additional HTTP headers. A "flat" `name → value` map (as in WebSocket)

The purpose of the **Host**, **Path** and **Headers** fields is the same as in WebSocket (section 6.4). Heartbeat is not provided for HTTPUpgrade — that is specific to WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (also known as SplitHTTP) is the modern multiplexable HTTP transport of xray-core. It splits the upstream and downstream flows into separate HTTP requests, which suits CDNs and environments with

connection-duration limits well. Not all fields are visible in the editor at once: some of them appear depending on the selected mode (`mode`) and the enabled toggles.

Basic fields (always visible)

Field	Key	Default	Description
Host	`host`	`""` (empty)	The value of the HTTP `Host` header
Path	`path`	`/`	The base path of HTTP requests
Mode (`Mode`)	`mode`	`auto`	Transmission mode (see below)
Server Max Header Bytes	`serverMaxHeaderBytes`	`0`	The limit on the request header size on the server (bytes). `0` — the xray-core default value
Padding Bytes	`xPaddingBytes`	`100-1000`	The range of random "filler" padding (in bytes, format `min-max`) to hinder size analysis
Headers	`headers`	`{}` (empty)	Additional HTTP headers. A "flat" `name → value` map
Uplink HTTP method	`uplinkHTTPMethod`	`""` (Default = POST)	The HTTP method of upstream requests. Options: empty (POST by default), `POST` , `PUT` , `GET` (the last is available only in `packet-up` mode)
Padding Obfs Mode	`xPaddingObfsMode`	`false` (off)	Enables extended padding obfuscation and opens additional fields (see below)
No SSE Header	`noSSEHeader`	`false` (off)	Do not send the `Content-Type: text/event-stream` (SSE) header. Enable it if it interferes with passage through intermediate nodes

The "Mode" field (`mode`)

A drop-down with the values:

Value	Description
`auto`	Automatic mode selection (default)
`packet-up`	The upstream flow is split into separate HTTP requests (one packet per request)
`stream-up`	The upstream flow is transmitted as a single long streaming request
`stream-one`	A single shared bidirectional streaming request

The choice of mode determines which additional fields become visible.

Fields visible only when `mode = packet-up` :

Field	Key	Default	Description
Max buffered upload	`scMaxBufferedPosts`	`30`	The maximum number of simultaneously buffered upstream POST requests

Field	Key	Default	Description
Max upload size (bytes)	`scMaxEachPostBytes`	`1000000`	The maximum size of a single upstream POST request (bytes)
Uplink Data Placement	`uplinkDataPlacement`	`""` (Default = body)	Where to place upstream data: `body`, `header`, `cookie`, `query`
Uplink Data Key	`uplinkDataKey`	`""`	The key/header name for uplink data. Appears only if `uplinkDataPlacement` is set and is not equal to `body`

Field visible only when `mode = stream-up`:

Field	Key	Default	Description
Stream-Up Server	`scStreamUpServerSecs`	`20-80`	The range of hold time for the server streaming connection (in seconds, format `min-max`)

Padding obfuscation fields (visible when `xPaddingObfsMode = on`)

Field	Key	Default	Description
Padding Key	`xPaddingKey`	`""` (placeholder `x_padding`)	The key name for the padding
Padding Header	`xPaddingHeader`	`""` (placeholder `X-Padding`)	The name of the HTTP header in which the padding is carried
Padding Placement	`xPaddingPlacement`	`""` (Default = queryInHeader)	Where to place the padding: `queryInHeader`, `header`, `cookie`, `query`
Padding Method	`xPaddingMethod`	`""` (Default = repeat-x)	The padding generation method: `repeat-x` or `tokenish`

Session and sequence placement (always visible)

Field	Key	Default	Description
Session ID Placement	`sessionIDPlacement`	`""` (Default = path)	Where to carry the session identifier: `path`, `header`, `cookie`, `query`
Session ID Key	`sessionIDKey`	`""` (placeholder `x_session`)	The session key name. Appears only if `sessionIDPlacement` is set and is not equal to `path`
Session ID Table	`sessionIDTable`	`""` (placeholder `Base62`)	The charset used to generate session identifiers. Pick a predefined one from the autocomplete drop-down (`ALPHABET`, `Alphabet`, `BASE36`, `Base62`, `HEX`, `alphabet`, `base36`, `hex`, `number`) or enter an arbitrary ASCII string. Empty — the xray-core default

Field	Key	Default	Description
Session ID Length	`sessionIDLength`	`""` (empty)	The length or range (for example `8–16`) of the generated identifiers. Shown only when `Session ID Table` is set; the lower bound must be greater than 0
Sequence Placement	`seqPlacement`	`""` (Default = path)	Where to carry the packet sequence number: `path`, `header`, `cookie`, `query`
Sequence Key	`seqKey`	`""` (placeholder `x_seq`)	The sequence key name. Appears only if `seqPlacement` is set and is not equal to `path`

The session fields were renamed for xray-core v26.6.22: they were previously called **Session Placement** / **Session Key** (`sessionPlacement` / `sessionKey`) and are now **Session ID Placement** / **Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); the core no longer understands the old names. Inbounds saved before the update are migrated to the new keys automatically — no re-save is needed.

Recommendations:

- For most setups it is enough to leave **Mode =** `auto`, set **Path**/**Host** and (when working through a CDN) coordinate them with the client.
- The placement fields (`*Placement` / `*Key`) and padding obfuscation are needed only for fine-tuning to a specific anti-DPI/CDN scenario; with empty values the xray-core default values indicated in parentheses are used.
- Parameters related to the client/outbound side (for example, retry POST intervals, chunk sizes) are not shown in the inbound form — the listener server ignores them. The XMUX multiplexer, by contrast, is available in the inbound form (see below).
- **Service defaults are no longer seeded.** The panel no longer writes the `scMaxEachPostBytes` and `scMinPostsIntervalMs` service defaults into XHTTP configs — xray-core's internal values are applied instead. This removes a stable DPI fingerprint (the literal `scMinPostsIntervalMs=30`) that traffic could previously be blocked on. For inbounds already saved, values equal to xray-core's defaults are no longer emitted into share links and the subscription (no re-save required); values you set manually are kept.

Example `streamSettings` for XHTTP (`auto` mode):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

For most setups these four fields are enough; the session/sequence placement and padding obfuscation fields are left empty — then the xray-core default values are used.

XMUX (connection multiplexing)

The **XMUX** toggle (`enableXmux`) enables a multiplexing layer that spreads parallel requests across a small pool of physical connections. When enabled, six configurable fields appear (stored under `xhttpSettings.xmux`):

Field	Key	Default	Description
Max Concurrency	`maxConcurrency`	`16–32`	Maximum concurrent requests per connection (`min–max` range)
Max Connections	`maxConnections`	`6`	Maximum physical connections (`0` — unlimited). As of version 3.4.2 new inbounds are created with the value `6` ; previously created ones keep their own.
Max Reuse Times	`cMaxReuseTimes`	`""` (empty)	How many times a connection may be reused
Max Request Times	`hMaxRequestTimes`	`600–900`	Maximum requests per connection (range)
Max Reusable Secs	`hMaxReusableSecs`	`1800–3000`	How long a connection stays reusable (seconds, range)
Keep Alive Period	`hKeepAlivePeriod`	`""` (empty)	The keep-alive period for holding the connection open

> **Important. Max Connections** and **Max Concurrency** cannot be set at the same time — xray-core will reject such a config. By default, when XMUX is enabled the panel pre-fills `Max Concurrency = 16–32` ; if you set **Max Connections** (a value greater than `0`), the panel drops the default `Max Concurrency` value to avoid the conflict.

6.8. Hysteria transport (`hysteriaSettings`)

The **Hysteria** transport is not selected in the "Transport" list: it is activated automatically when an inbound is created with the Hysteria protocol, and is hidden for other protocols (when leaving the Hysteria protocol, the network is forcibly reset to `tcp`). The parameters:

Field	Key	Default	Description
Version	`version`	`2` (fixed, the field is locked)	The Hysteria version. Only Hysteria 2 is supported
UDP idle timeout (s)	`udpIdleTimeout`	`60`	The UDP session idle timeout (seconds). The accepted range is 2–600; xray-core rejects values outside this interval at startup
Masquerade	`masquerade`	off (absent)	Enables disguising the listener as an HTTP/3 server when probed

When **Masquerade** is enabled, a type selector (`type`) and fields depending on it appear:

- `""` **— default (404 page)**: a standard 404 page is served (no additional fields required).
- `proxy` **(reverse proxy)**: reverse proxying to an external site.
  - `url` (**Upstream URL**, placeholder `https://www.example.com`) — the target address;
  - `rewriteHost` (**Rewrite Host**, default `false`) — substitute the `Host` header;
  - `insecure` (**Skip TLS verify**, default `false`) — do not verify the upstream's TLS certificate.
- `file` **(serve directory)**: serving files from a directory.
  - `dir` (**Directory**, placeholder `/var/www/html`).
- `string` **(fixed body)**: a fixed HTTP response.
  - `statusCode` (**Status code**, default `0` , range 0–599);
  - `content` (**Body**) — the response body;
  - `headers` (**Headers**) — a `name → value` map.

Masquerade allows a Hysteria-based inbound to look like an ordinary HTTP/3 server to active probes, which increases stealth. By default masquerade is off.

Example `hysteriaSettings` with reverse proxying (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Here, when probed, the listener serves the response from `https://www.example.com` , disguising itself as an ordinary HTTP/3 site.

6.9. Related parameters

In addition to the network choice, two common blocks that do not depend on a specific transport are available on the same tab (in detail — in the corresponding sections):

- **External Proxy** (`externalProxy`) — a list of external addresses/ports that are substituted into subscription links instead of the panel's own address.
- **Sockopt** (`sockopt`) — low-level socket options (TCP Fast Open, mark, domain strategy, transparent proxying, etc.).

Real client IP (recovering the real IP behind a CDN/relay)

When an inbound sits behind an intermediary (a CDN such as Cloudflare, an L4 tunnel/relay, or another panel), Xray sees the intermediary's address rather than the real visitor's. That address is what shows up in the online-clients list and what the per-client IP limit counts against, which makes both useless behind a proxy. To recover

the real IP, the **Sockopt** section of the inbound form provides a **Real client IP** preset selector that bundles the `acceptProxyProtocol` and `trustedXForwardedFor` settings:

Preset	What it does	When to use
Off / direct	Clears both fields.	Inbound reachable directly by clients
Cloudflare CDN	Sets `sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]`.	WebSocket / HTTPUpgrade / XHTTP / gRPC behind Cloudflare's CDN (orange cloud)
L4 relay / Spectrum (PROXY)	Enables `acceptProxyProtocol = true`.	An L4 tunnel/relay in front of the inbound, or Cloudflare **Spectrum**

The presets are mutually exclusive: choosing one clears the other field, so a stale `trustedXForwardedFor` cannot override the PROXY-protocol-recovered IP. The raw **Proxy Protocol** switch and **Trusted X-Forwarded-For** list stay visible below the preset selector — the preset just fills them in for you, and you can edit them manually if needed. If the selected preset is not supported by the current transport (for example, the PROXY protocol on mKCP), the form shows a warning. These fields apply to the server side only and are **never sent to clients in subscriptions**.

> **Use one, not both.** `acceptProxyProtocol` reads the real IP from the L4 PROXY-protocol header, while `trustedXForwardedFor` reads it from an HTTP request header; mix them by hand only if your upstream chain requires it.

- **FinalMask** (`finalmask`) — a common transport-layer obfuscation mechanism (including legacy mKCP obfuscation) that has replaced the separate "seed"/"header type" fields inside the network sub-forms.

---

7. Connection Security: TLS, XTLS, and REALITY

Every inbound that supports transport-stream delivery (VMess, VLESS, Trojan, Shadowsocks, Hysteria) has a **Security** tab in the editor. This tab controls how the transport channel is encrypted and disguised. Three modes are available, selected with radio buttons:

Mode	UI label	When available
`none`	**None**	Always (except Hysteria, where TLS is mandatory)
`tls`	**TLS**	For VMess/VLESS/Trojan/Shadowsocks on `tcp`, `ws`, `http`, `grpc`, `httpupgrade`, `xhttp` networks; for Hysteria — always
`reality`	**Reality**	Only for VLESS/Trojan on `tcp`, `http`, `grpc`, `xhttp` networks

The **None** button is hidden when the protocol is Hysteria (TLS is required there). The **Reality** button appears only for a valid combination of protocol and network (see the table above).

When the mode is switched, the panel fully rebuilds the `streamSettings` block: it removes any `tlsSettings` and `realitySettings` left by the previous mode and fills in the defaults for the selected one. In particular, when **Reality** is selected, the panel immediately and automatically: fills in a random `target` + `serverNames` (SNI) pair from a built-in list of popular domains, generates random `shortIds`, and fetches a fresh X25519 key pair (privateKey/publicKey) from the server.

7.1. The Difference: TLS vs XTLS vs REALITY

- **TLS** — classic transport encryption using TLS 1.2/1.3. A valid certificate (your own domain + chain) must reside on the server. Traffic looks like ordinary HTTPS, but to an active censor the TLS handshake pointing to your domain is recognizable; if blocked by SNI or if there is no trusted certificate, the connection is blocked or returns an error.
- **XTLS (Vision)** — this is not a separate option in the Security list, but a *flow* mechanism on top of TLS **or** Reality. It is enabled on the client side of an inbound via the **Flow** field = `xtls-rprx-vision` (or `xtls-rprx-vision-udp443`). Vision is available for VLESS on the `tcp` network with `security = tls` or `security = reality`, and also for VLESS over the `xhttp` transport with VLESS encryption enabled (vlessenc / ML-KEM) — in that case the **Flow** field can also be set to `xtls-rprx-vision`, and the value is correctly placed in the `vless://` link (`flow=xtls-rprx-vision`). Vision reduces "double encryption" (TLS-in-TLS) by delivering the payload directly after the handshake, which speeds up transfer and improves disguise. For this reason the combination **VLESS + Reality + Flow** `xtls-rprx-vision` is considered the recommended modern configuration.

> **Automatic Vision flow restoration.** If encryption (ML-KEM, the decryption/encryption fields) is enabled on a VLESS/XHTTP inbound after clients have already been added, the inbound becomes flow-eligible. In this situation the panel automatically restores `flow = xtls-rprx-vision` for those clients who should have it but whose **Flow** field was empty. Previously, in this scenario Vision would silently disappear from configs, share links, and subscriptions (especially noticeable on hub inbounds). No manual action is required: the fix is applied automatically when the inbound is saved and once during a panel update. The

> behavior is conservative — the panel does not invent a flow value and does not overwrite a value that the client has explicitly set.

- **REALITY** — a disguise mechanism that needs no certificate of its own. The server "borrows" the TLS handshake of a real third-party site (`target` / `serverNames`), so to an observer the connection is indistinguishable from a visit to that site, and no certificate is needed at all. Authentication is built on an X25519 key pair and a set of `shortIds` . REALITY is resistant to active probing and SNI blocking because the SNI points to a genuine external domain. The trade-off is stricter setup requirements (a correct `target` with a port, key synchronization with the client).

Quick selection rule:

- you have your own domain and a valid certificate and want a simple HTTPS appearance → **TLS** (with Vision where possible);
- you have no domain/certificate or need maximum invisibility to DPI → **REALITY** (with Vision for VLESS/TCP).

7.2. None Mode (`none`)

The transport is carried without a TLS wrapper: `tlsSettings` and `realitySettings` are removed from `streamSettings` . The mode has no additional fields. It is appropriate when:

- the inbound listens only on `127.0.0.1` and serves as a fallback target (per the panel rule, a child inbound for fallback must listen on `127.0.0.1` with `security=none`);
- encryption/disguise is provided by an outer layer (for example, an Nginx reverse proxy in front of the panel);
- a protocol with its own encryption (Shadowsocks) is used on an internal network.

None mode is not recommended for inbounds that are exposed to the outside.

Example: `streamSettings` block for TLS on the `tcp` network (VLESS/Trojan/VMess). This is the result after selecting **TLS** mode and filling in the SNI and certificate paths:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. TLS Mode

Fields of the `tlsSettings` block. Default values are taken from the panel schema.

Main Parameters

Field (label)	Default value	Description
SNI (`serverName`)	`""` (empty)	Server Name Indication — the domain name presented in the TLS handshake. Must match the domain of the certificate. Placeholder hint: "Server Name Indication".
Cipher Suites (`cipherSuites`)	`""` → **Auto**	List of allowed cipher suites. Empty by default — the choice is left to Xray/Go (the **Auto** option). Change only when you need to explicitly restrict ciphers.
Min/Max Version (`minMaxVersion`)	min = `1.2` , max = `1.3`	Minimum and maximum TLS versions. Available values: `1.0` , `1.1` , `1.2` , `1.3` . It is recommended to leave `1.2` – `1.3` ; lowering the minimum to 1.0/1.1 is undesirable (outdated, insecure versions).
uTLS (`settings.fingerprint`)	`chrome` (in the form — the **None** option = `""` is available)	Simulated TLS fingerprint for the client hello (uTLS fingerprint), so the handshake looks like that of a popular browser. See the list below. In TLS mode the first item in the list is **None** (`""`), which disables fingerprint simulation.
ALPN (`alpn`)	`["h2", "http/1.1"]`	List of application-layer protocols negotiated in TLS (multi-select). Allowed values: `h3` , `h2` , `http/1.1` . By default `h2` and `http/1.1` are offered.

Possible **uTLS fingerprint** values (identical for TLS and REALITY): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . In TLS mode an additional empty **None** option is also available (fingerprint simulation is not applied).

Available **Cipher Suites** values (TLS 1.3 and ECDHE suites): `TLS_AES_128_GCM_SHA256`, `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`, `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA`, `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA`, `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA`, `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA`, `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256`, `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384`, `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`, `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`, `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256`, `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256`.

TLS Toggles

Toggle	Default	Description
Reject Unknown SNI (`rejectUnknownSni`)	off (`false`)	When enabled, the server drops a connection if the SNI presented by the client does not match the name in the certificate. Increases stealth (the server does not respond to "foreign" requests), but requires an exact SNI match on the client.
Disable System Root (`disableSystemRoot`)	off (`false`)	Disables use of the system trusted root certificate store.
Session Resumption (`enableSessionResumption`)	off (`false`)	Enables TLS session resumption (session tickets).

Additional TLS Parameters (vcn, curves, key log, ECH Sockopt)

Additional fields are available below the main TLS settings.

Field (label)	Default	Description
Verify Peer Cert By Name (`settings.verifyPeerCertByName`)	`""`	Names (comma-separated) against which the client verifies the server certificate instead of the SNI. This is the modern replacement for the `allowInsecure` field removed from Xray after 2026-06-01. Panel-only value: it is not written to the xray server config, but is included in invite links and subscriptions (`vcn=…`) so that the client applies it on its end. Placeholder: `example.com`.
Curve Preferences (`curvePreferences`)	`""`	Restriction and ordering of TLS key-exchange curves, in order of preference (e.g. `X25519MLKEM768`, `X25519`). Empty — Xray-core defaults are used.
Master Key Log (`masterKeyLog`)	`""`	Path for writing TLS master keys in `SSLKEYLOGFILE` format (for decrypting traffic in Wireshark during debugging). Placeholder: `/path/to/sslkeylog.txt`. Leave empty in production — the file allows decrypting all traffic.
ECH Sockopt (`echSockopt`)	off	Toggle with socket parameters for the connection through which Xray requests the ECH config list. When enabled, the following are available: **Dialer Proxy** (`dialerProxy` — route the request through the specified outbound by tag), **Domain Strategy** (`domainStrategy`), **TCP Fast Open** (`tcpFastOpen`), **Multipath TCP** (`tcpMptcp`). Leave off unless needed.

The fields `verifyPeerCertByName`, `curvePreferences`, `masterKeyLog`, and `echSockopt` reside at the top level of `tlsSettings` and survive the panel's field trimming when the configuration is saved.

Certificates

The **SSL Certificate** section (UI heading "SSL Certificate") is a list: clicking **+** adds a new certificate entry, clicking **– Delete** removes one (the delete button is available only when there is more than one entry). By default, when TLS is enabled, one empty entry is created.

For each entry, an input mode toggle (`useFile`):

- **Certificate Path** (value `useFile = true`, default) — file paths on the server are specified:
  - **Public Key** (`certificateFile`) — path to the certificate file (`.crt` / `.pem`);
  - **Private Key** (`keyFile`) — path to the private key file (`.key`).
- **Certificate Content** (value `useFile = false`) — the content is pasted directly into the fields (multi-line text areas):
  - **Public Key** (`certificate`) — PEM content of the certificate;
  - **Private Key** (`key`) — PEM content of the key.

Below the "Certificate Path" mode fields, two buttons are available:

- **Set Panel Certificate** — fills in the paths to the panel's own SSL certificate. For an inbound on the central panel, the panel's certificate is used (`POST /panel/setting/all` → `webCertFile` / `webKeyFile`); for an inbound assigned to a node, the node's own certificate is used (`GET /panel/api/nodes/webCert/{nodeId}`), because the central panel's paths do not exist on the node. If no certificate is configured, a warning is shown: "*No certificate is configured for the panel. Set one in Settings first.*" (The panel's own certificate is configured in Settings → Security.)
- **Clear** — erases both paths.

Additional fields for each certificate entry:

Field	Default	Description
OCSP Stapling (`ocspStapling`)	`0` (off)	OCSP stapling refresh interval in seconds (minimum `0`). Disabled by default (`0`) for new inbounds: this prevents xray log errors for certificates without an OCSP responder (for example, Let's Encrypt, which dropped OCSP). Enable only for certificates that support stapling.
One-time Loading (`oneTimeLoading`)	off (`false`)	When enabled, the certificate is read from disk only once at startup and is not re-read when the file changes.
Usage (`usage`)	`encipherment`	Certificate purpose. Allowed values: `encipherment` (encryption — a normal server certificate), `verify` (verification), `issue` (issuance — the server signs/issues certificates itself).
Build Chain (`buildChain`)	off (`false`)	Shown **only** when `usage = issue`. Build the certificate chain.

> There is no separate button in the inbound editor for generating a self-signed certificate: the panel does not generate a self-signed certificate on the fly for an inbound. A certificate is either specified by path/content or pulled from the panel settings with the "Set Panel Certificate" button. Issuing/obtaining the

> panel's own SSL certificate (including file upload and domain binding) is done in **Settings** → **Security**; there are no ACME/Let's Encrypt endpoints for individual inbounds here.

ECH and Certificate Pinning (Advanced TLS Fields)

Field	Default	Description
ECH key (`echServerKeys`)	`""`	Server keys for Encrypted Client Hello.
ECH config (`settings.echConfigList`)	`""`	ECH config list (client side, included in the link).
Peer Certificate SHA-256 (`settings.pinnedPeerCertSha256`)	`[]`	SHA-256 hashes of the peer certificate (hex strings, comma-separated). Verbatim hint: "*SHA-256 hashes of the peer certificate as a hex string (e.g. e8e2d3…), comma-separated. Panel-only — not written to the xray server config, but included in invite links so clients can pin the certificate.*"

Buttons: Next to the **Peer Certificate SHA-256** field there are two auto-fill buttons:

- **Fill from this inbound's certificate** (shield icon) — inserts the SHA-256 hash of this inbound's own certificate (fetched locally via the `getCertHash` endpoint).
- **Fetch the hash by pinging the SNI (xray tls ping)** (download icon) — retrieves the hash of the live server certificate by making a TLS connection to the specified SNI (the server calls `getRemoteCertHash`). The **SNI** (`serverName`) field must be filled in — otherwise the hint "*Set the SNI (serverName) first to ping the remote certificate.*" is shown.

The fetched hashes are appended to the field (comma-separated) and included in invite links so the client can pin the certificate.

- **Get New ECH Certificate** — asks the server for a new ECH pair for the current SNI (`POST /panel/api/server/getNewEchCert` , the server runs `xray tls ech --serverName <SNI>`); fills in the **ECH key** and **ECH config** fields.
- **Clear** — empties both ECH fields.

7.4. REALITY Mode

Fields of the `realitySettings` block. REALITY does not use an SSL certificate: instead it uses a borrowed TLS handshake from an external domain and an X25519 key pair.

Disguise Parameters

Field (label)	Default value	Description
Show (`show`)	off (`false`)	Debug output for REALITY in Xray logs. Usually left off.
Xver (`xver`)	`0`	PROXY protocol version forwarded to the backend (`0` — disabled). Minimum `0` .
uTLS (`settings.fingerprint`)	`chrome`	Simulated TLS fingerprint (same list as in TLS mode, but without the empty None option).

Field (label)	Default value	Description
Target (`target`)	`""` (as of 3.4.2 it stays empty when REALITY is enabled)	**Required field.** The real domain whose TLS handshake REALITY borrows. Verbatim hint: "*Required. Must include a port (e.g. example.com:443). Without a port Xray-core will not start.*" Panel validation checks that a port is present and valid; otherwise errors are shown: "REALITY Target is required" / "REALITY Target must include a port…" / "REALITY Target has an invalid port". Next to the field are the **"Scan"** button (check the current target "live") and the **"Find targets"** button (open the REALITY target scanner); see below.
SNI (`serverNames`)	`[]` (filled together with the target)	List of allowed SNIs (multi-input with tags). Must correspond to the domain in **Target**. On a successful target scan, the SNI is filled from its certificate.
Max Time Difference (ms) (`maxTimediff`)	`0`	Maximum allowed clock skew between client and server in milliseconds (`0` — no limit). Minimum `0` .
Min Client Version (`minClientVer`)	`""`	Minimum Xray client version (placeholder `25.9.11`). Empty — no restriction.
Max Client Version (`maxClientVer`)	`""`	Maximum Xray client version. Empty — no restriction.
Short IDs (`shortIds`)	`[]` (generated when enabled)	List of short identifiers (hex) that distinguish clients. Multi-input with tags; the refresh button generates a random set.
SpiderX (`settings.spiderX`)	`/`	Spider path (the client-side REALITY component) used when simulating access to the external site. Included in the invite link.

As of version 3.4.2, **Target** (`target`) and **SNI** (`serverNames`) are **no longer** filled automatically when REALITY is enabled — both fields stay empty, and the target is picked by the live scanner (see below). Choose a heavyweight, stable third-party HTTPS site that supports TLS 1.3 and HTTP/2 and is not behind your own server.

Finding a REALITY target (live scanner)

As of version 3.4.2 the former "random target" button (with its static built-in list) has been replaced by two actions next to the **Target** field:

- **"Scan"** — checks the current target "live": it establishes a TLS connection and, if the target is suitable, fills the SNI from its certificate. Toasts: "Target is suitable — target and SNI filled." / "Target is reachable but not suitable for REALITY." / "Failed to scan the REALITY target.".
- **"Find targets"** — opens the **"REALITY target scanner"** window: you can specify a domain, an **IP or CIDR range** (the panel then finds targets by the certificates of live hosts), or leave the field empty (the built-in candidates are checked). The results are ranked; the columns are "Status"/"Suitable", "Key exchange", "Certificate", `TLS` , `ALPN` , "Latency". The **"Use"** button substitutes the selected row into the inbound fields.

A target is considered **suitable** if the server negotiates **TLS 1.3**, **HTTP/2 (ALPN h2)**, **X25519/X25519MLKEM768** key exchange and presents a **trusted** (non-wildcard) certificate. Scanning is performed on the panel's server side (`POST /panel/api/server/scanRealityTarget` and `…/scanRealityTargets`).

Example: `streamSettings` block for REALITY on the `tcp` network (VLESS). No certificate is needed — instead, a borrowed domain and an X25519 key pair are used:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": ["", "6ba85179e30d4fc2"],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Here the panel's **Target** (`target`) field corresponds to `dest` in the final Xray config. If a REALITY inbound was created with the destination in the `dest` key (by older panel versions, through the API, or external tools), the panel normalizes `dest` → `target` when parsing, provided `target` is empty — so such an inbound loads correctly, the **Target** field is not left empty, and re-saving does not break the working REALITY.

REALITY Keys (X25519)

Field	Default	Description
Public Key (`settings.publicKey`)	`""`	X25519 public key (placed by the client into its configuration/link).
Private Key (`privateKey`)	`""`	X25519 private key (stored on the server only).

Buttons below the keys:

- **Get New Certificate** — requests a new key pair from the server (`GET /panel/api/server/getNewX25519Cert` ; the server runs `xray x25519`), fills in **Private Key** and **Public Key**. The same pair is generated automatically the first time REALITY mode is enabled.

Example: obtain an X25519 key pair via API (outside the form, e.g. in a script). The request returns the private and public keys:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Response:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

`cookie.txt` — the session cookie file obtained after logging in via `POST /login` .

- **Clear** — empties both keys.

Post-Quantum ML-DSA-65 Signature (mldsa65)

An optional additional layer of post-quantum authentication for REALITY:

Field	Default	Description
mldsa65 Seed (`mldsa65Seed`)	`""`	Server seed for the ML-DSA-65 key.
mldsa65 Verify (`settings.mldsa65Verify`)	`""`	Verification value (client side, included in the link).

Buttons:

- **Get New Seed** — requests a new pair (`GET /panel/api/server/getNewmldsa65` ; the server runs `xray mldsa65`), fills in **mldsa65 Seed** and **mldsa65 Verify**.
- **Clear** — empties both fields.

Fallback Speed Limit and REALITY Key Log

REALITY settings include a fallback traffic speed limit — it prevents active probes from using the server as a free channel to the borrowed domain. The setting is configured separately for two directions — **Limit Fallback Upload** and **Limit Fallback Download** (`limitFallbackUpload` / `limitFallbackDownload`), each with the same set of fields:

Field (label)	Default	Description
After Bytes (`afterBytes`)	`0`	How many bytes to pass at full speed before throttling begins. `0` — throttle from the first byte.
Bytes Per Sec (`bytesPerSec`)	`0`	Fallback traffic speed ceiling in bytes per second after the threshold. `0` — no limit (disables this direction).
Burst Bytes Per Sec (`burstBytesPerSec`)	`0`	Allowance for brief bursts above the sustained rate (token-bucket size). If lower than **Bytes Per Sec**, it is raised to that value.

A **Master Key Log** (`masterKeyLog`) field is also present here — the path for writing TLS master keys in `SSLKEYLOGFILE` format for debugging in Wireshark; leave empty in production.

7.5. Practical Configuration Recommendations

1. **VLESS + Reality (recommended):** create a VLESS inbound on the `tcp` network, select **Reality** on the Security tab — the panel will automatically fill in random `target` /SNI, `shortIds` , and generate X25519 keys. If you need your own key pair, click "Get New Certificate". For VLESS clients, enable **Flow** = `xtls-rprx-vision` (XTLS Vision) — this gives maximum performance and stealth.

Example: resulting VLESS + Reality + Vision client link. This is what the invite link looks like when the panel generates it for such an inbound (key/ID values are illustrative):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Here `pbk` is the X25519 public key, `sni` is the borrowed domain from **Target**, `sid` is one of the **Short IDs**, and `flow=xtls-rprx-vision` is XTLS Vision enabled. 2. **TLS with your own domain:** select **TLS**, fill in the **SNI** with the domain name, add a certificate (by file paths or by content), or click "Set Panel Certificate" if the domain and certificate are already configured in Settings → Security. Leave **Min/Max Version** = `1.2` – `1.3` and **uTLS** = `chrome` to disguise traffic as a normal browser. 3. Do not leave **None** mode for inbounds exposed to the outside — use it only for local fallback targets (`127.0.0.1`) or when TLS is provided by an external proxy. 4. A tip from the UI: for most advanced fields the hint reads "*It is recommended to leave the settings at their defaults*" — change them only if you understand the consequences.

---

8. Clients

A client is a VPN user account: a set of credentials (UUID or password) bound to one or more inbounds, with its own traffic quota, expiry date, and concurrent connection limit. In this fork a client is a standalone entity (the `clients` table): the same client can be bound to multiple inbounds at once, sharing a common UUID/password and a shared traffic counter. The **Clients** section shows all panel accounts regardless of inbound, with search, filters, sorting, and bulk operations.

8.1. Client fields

Every field of the **Add client** / **Edit client** editor is described below.

The client form is split into two tabs: **General** (email, inbound binding, limits, expiry, group, comment, reverse tag) and **Credentials** (UUID/password/auth, Flow, VMess Security). In field labels the quota is shown as **Traffic limit (GB)** and the time periods as **Duration (days)** and **Auto-renewal (days)**; the **Traffic limit (GB)** and **IP limit** fields have tooltips explaining that `0` means "no limit". When editing an existing client the random-email generation button is hidden, and the IP log button is placed directly next to the **IP limit** field and shows the number of recorded addresses.

Field	JSON key	Default	Description
Email	`email`	— (required)	Unique client identifier
UUID	`id`	generated	Identifier for VMess/VLESS
Password	`password`	generated	Password for Trojan/Shadowsocks
Authorization	`auth`	generated	Password for Hysteria
Flow	`flow`	empty	Flow control (XTLS), VLESS only
VMess Security	`security`	`auto`	VMess encryption method
IP limit	`limitIp`	`0` (no limit)	Maximum concurrent IPs
Total sent/received (GB)	`totalGB`	`0` (no limit)	Traffic quota
Expiry	`expiryTime`	`0` (never)	Expiration timestamp
Auto-renewal	`reset`	`0` (off)	Traffic reset period, days
Telegram user ID	`tgId`	`0` (none)	Numeric Telegram ID
Subscription ID	`subId`	generated	Subscription identifier
Group	`group`	empty	Logical grouping label
Comment	`comment`	empty	Arbitrary note
Enabled	`enable`	`true`	Whether the account is active

Email (identifier)

The **Email** field is the primary and mandatory client identifier. Despite the name, it does not have to be an email address: any text label (username, number) will work. The value must be **unique** within the panel; attempting to create a second client with an already-used email is rejected (`email already in use`), unless the `subId` also matches (which is interpreted as binding the same client).

Email **cannot be left empty** (`client email is required`) and it **cannot contain spaces, `/` , `\` , or control characters** ("Email cannot contain spaces, '/', '\', or control characters"). Email is used for traffic accounting, IP logging, the online list, and operation names — changing it retroactively is not recommended.

UUID / Password / Authorization (credentials)

The specific credential field depends on the protocol of the inbound the client is being bound to. Values are filled in automatically if the field is left empty:

- **UUID** (field `id`) — for **VMess** and **VLESS** protocols. If not set, a random UUID v4 is generated.
- **Password** (field `password`) — for **Trojan** and **Shadowsocks**. For Trojan a UUID without hyphens is generated by default. For Shadowsocks a Base64 key of the required length is generated depending on the inbound encryption method: 16 bytes for `2022-blake3-aes-128-gcm` , 32 bytes for `2022-blake3-aes-256-gcm` and `2022-blake3-chacha20-poly1305` ; for other methods — a UUID without hyphens. If a manually entered key does not match a 2022-blake3 method, it will be replaced by a generated one.
- **Authorization** (field `auth`) — password for **Hysteria**. A UUID without hyphens by default.

Since one client can be bound to inbounds of different protocols, a client record may simultaneously have a UUID, a password, and auth — each protocol uses its own field.

Example: how client credentials appear in `settings` of different inbounds. The same client in a VLESS inbound is identified by `id` , in Trojan by `password` , in Shadowsocks by `password` (Base64 key):

```
// fragment of settings.clients in a VLESS inbound
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-
vision" }

// the same client in a Trojan inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// the same client in a Shadowsocks inbound (method 2022-blake3-aes-256-gcm)
{ "password": "GPyOaA3f7CO0az53eaQ8eqMfRDjmBlOh7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (field `flow`) — XTLS flow control. Applicable **only to VLESS** and only when the inbound is configured for XTLS Vision: VLESS over a **TCP** transport with security `tls` or `reality` . The allowed value is `xtls-rprx-vision` (as well as the historical `xtls-rprx-vision-udp443`); an empty value means no flow.

If the inbound does not support XTLS flow, the configured flow **is silently cleared** when the client is saved: for the same client bound to multiple inbounds, flow is applied only where it is allowed. Change this only if you deliberately use VLESS-Vision.

VMess Security

VMess Security (field `security`) — payload encryption method for VMess. The default value is `auto` (Xray selects the cipher automatically). Allowed values are the standard VMess ones: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. The field is not used for other protocols.

IP limit (concurrent connections)

IP limit (field `limitIp`) — the maximum number of **different IP addresses** from which the client can be connected simultaneously. The default value is `0`, meaning **no limit**. With a positive value the panel tracks the client's active IPs and, when the limit is exceeded, disables the account via a background job. (Starting with **3.3.1** IP counting uses the Xray core online-stats API and **does not require** the access log; on older core versions the panel falls back to reading the access log, which must then be enabled.) Use this to prevent sharing one subscription across many devices: for example, `2` allows two devices.

The IP limit is enforced via **Fail2ban**, so the **IP limit** field is active only when Fail2ban is installed and operational (the panel checks its status via `GET /panel/api/server/fail2banStatus`). If Fail2ban is not installed, the client editor field (and the bulk-add form) is disabled and hovering shows a tooltip suggesting installing Fail2ban from the `x-ui` bash menu ("Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option."); on Windows the tooltip states that Fail2ban is unavailable there ("Fail2ban is not available on Windows, so the IP limit cannot be enforced."), and if the feature is disabled on the server — "The IP limit feature is disabled on this server.". When the panel is updated, clients on servers without Fail2ban have their saved IP limit zeroed out by a one-time migration, since it was not enforced there anyway.

Example values. `limitIp: 0` — no limit; `limitIp: 1` — strictly one device at a time; `limitIp: 3` — up to three different IPs. When a fourth active IP appears, the background job will disable the client (`enable = false`) until you perform **Reset IP limit**.

Related operations: **IP log** shows the list of recorded IPs for the client; each entry contains the IP itself, the time of the last connection, and the node label (`@ node_name`) through which the connection was recorded — in a multi-panel configuration you can see which node the client used. **Reset IP limit** clears the accumulated IP log so the client can connect again without waiting for entries to expire naturally.

Total sent/received (GB) — traffic quota

Total sent/received (GB) (field `totalGB`) — the combined traffic quota (upload + download). The default value `0` means **unlimited**. Once the quota is reached (`up + down >= total`), the client is considered **depleted** and is disabled. In the UI values are usually entered in gigabytes; the database stores them in bytes.

In the client list the **Traffic** column shows a colored usage bar: the amount of traffic consumed, the limit label (or ∞ for unlimited), and a hover tooltip breaking down upload/download and the remaining amount. The same compact indicator appears in client cards on mobile.

Expiry

Expiry (field `expiryTime`) sets the moment the account expires. The field has three modes:

- **Never** — `0`. The client never expires by time.
- **Specific date** — a positive Unix timestamp (in milliseconds). When reached (`expiryTime <= now`) the client is considered expired and is disabled. In the UI this is usually set by choosing a date or entering a duration in days (**Duration**, unit **Days**).
- **Start on first use** — a **negative** value encoding a duration. While the client has not transferred a single byte, the expiry stays negative ("deferred start"). On the first traffic accounting tick the panel converts it to an absolute date: `now + |duration|`. This allows selling, for example, "30 days from first connection" without knowing in advance when the client will activate. The conversion is performed once per email so that all bound inbounds receive the same expiry.

Example expiry encoding. Fixed date 1 March 2026, 00:00 UTC → `expiryTime: 1772323200000` (positive timestamp in milliseconds). "30 days from first connection" → `expiryTime: −2592000000` (negative value, `30 × 24 × 60 × 60 × 1000`); on the first byte of traffic the panel replaces it with `now + 2592000000`. Never expires → `expiryTime: 0`.

Auto-renewal (client traffic reset period)

The **Auto-renewal** field (field `reset`) is the automatic renewal/reset period in days. Tooltip: "Auto-renewal after expiry. (0 = disabled) (unit: day)".

- `0` — auto-renewal **disabled** (default). When the expiry is reached the client simply becomes depleted.
- `> 0` — the background job, upon expiry, **resets the traffic counters to zero** (`up = down = 0`), **advances the expiry** by `reset` days (by as many periods as needed until the new expiry is in the future), and if necessary **re-enables** the client. This implements a recurring subscription (e.g., monthly). Auto-renewal **is not applied to inbounds on node servers** (`node_id IS NOT NULL`).

Important consequence: clients with `reset > 0` are **excluded** from the "depleted" category in bulk-delete operations — their traffic/expiry are expected to be reset by auto-renewal rather than making the account a deletion candidate.

Telegram user ID

Telegram user ID (field `tgId`) — the numeric Telegram identifier for linking the client to the panel's built-in Telegram bot (notifications, self-service statistics). Tooltip: "Numeric Telegram user ID (0 = none)". Default value `0` — no link. This field supports filtering (**Has** / **None**).

Subscription ID (subId)

Subscription ID (field `subId`) — the identifier under which the client is included in a **subscription**. All clients with the same `subId` are served by a single subscription link. If the field is left empty at creation, the panel **automatically generates a random** `subId` (UUID). The value must be **unique** among clients with a different email (`subId already in use`) and is subject to the same character restrictions as email ("Subscription ID cannot contain spaces, '/', '\', or control characters").

Without a `subId` the subscription link for the client is unavailable ("This client has no subId, the share link is not available.").

Links tab (external links and subscriptions)

In addition to the **General** and **Credentials** tabs, the client editor has a third tab, **Links** (tooltip: "Add third-party share links and remote subscription URLs to include in this client's subscription."). The **Add External Link** button adds third-party share links (`vless://` , `vmess://` , `trojan://` , `ss://` , `hysteria2://` , `wireguard://`), and the **Add External Subscription** button adds remote subscription URLs (e.g., `https://provider.example/sub/…`).

All of these are merged into the subscription output for this client (raw, JSON, and Clash formats): links are added as-is, while remote subscriptions are periodically fetched by the panel (with caching and a short timeout) and their configurations are merged with the panel's own. This lets a single client subscription link deliver both your own servers and external configurations.

Group

Group (field `group`) — a logical label for grouping related clients. Tooltip: "Logical label for grouping related clients (e.g., team, customer, region). Filterable from the toolbar.", placeholder — "e.g., customer-a". The field is optional (empty by default). You can filter the list by group and perform bulk operations; to remove the label from a client use the **Ungroup** action.

You can also remove the group directly in the single-client editor: clear the **Group** field and save — the label is correctly removed and the client no longer appears under the previous group.

Comment

Comment (field `comment`) — an arbitrary text note for the administrator (empty by default). The content is included in search and supports filtering (**Has** / **None**).

Enabled

Enabled (field `enable`) — the account activity flag. Enabled by default (`true`); at creation, even if the flag is not passed, the panel forces it to `true` . A disabled client (`enable = false`) cannot connect and is classified as **inactive** (deactive) in the summary. The panel automatically disables clients that have exhausted their quota, expired, or exceeded the IP limit.

Read-only fields

The client card also displays service fields: **Created** (`created_at`) and **Updated** (`updated_at`) — creation and last-modification timestamps, filled automatically and not editable. The **Reverse tag** field (`reverse`) — an optional Reverse tag for a simple VLESS reverse proxy ("Optional Reverse tag").

8.2. Inbound binding

Each client must be bound to at least one inbound — at creation a minimum of one is required (`at least one inbound is required`). In the editor this is the **Bound inbounds** field with the tooltip **Select one or more inbounds**.

- **Bind** — add the client to the selected inbounds (same UUID/password and shared traffic). Existing bindings are preserved.
- **Unbind** — remove the client from the selected inbounds. The client record itself is preserved (use **Delete** for full removal). Pairs where the client was not bound are silently skipped.

When saving a client bound to multiple inbounds, fields that are incompatible with a specific protocol/transport (e.g., Flow outside VLESS-Vision) are automatically set to allowed values for each inbound.

Above the inbound selection list (in the client form, in the bulk-add form, and in the bulk attach/detach windows) there are **Select all** and **Clear** buttons. In these lists each inbound is labeled with its remark if one is set, otherwise with the inbound tag.

8.3. Per-client operations

For an individual client (via the **Client info** card or the **Actions** context menu) the following operations are available:

Viewing information, QR code, and link

- **Client info** — a card with all fields, used/remaining traffic (**Remaining**), expiry date, and bound inbounds.

Fetching a client via the API (`GET /panel/api/clients/get/:email`) returns, alongside the `client` and `inboundIds` fields, the additional `usedTraffic` — the actual consumed traffic (upload + download, including node data), which makes it easy to compare consumption against the `totalGB` quota.

- **QR code** and **Link** — the client configuration link for importing into a client application. Generated from all bound inbounds with a supported protocol (`GET /links/:email`). If there are no suitable links: "No share links available — bind the client to an inbound with a supported protocol first.".
- **Subscription link** — the subscription URL by `subId` (`GET /subLinks/:subId`). Available only if the client has a `subId` and the subscription service is enabled in **Panel settings** → **Subscription** (otherwise "Subscription service is disabled."). A **JSON subscription URL** is also provided.

Reset traffic

Reset traffic (`POST /resetTraffic/:email`) zeroes the upload/download counters (`up` , `down`) for the specific client. The quota (`totalGB`) and expiry are **not affected** — only the consumed volume is zeroed. Toast: "Traffic reset". If the client is not bound to any inbound: "Bind this client to an inbound first.".

The **Reset traffic** button is also available inside the client edit form — in the bottom panel, next to **Cancel** / **Save** (a confirmation dialog is shown before resetting). If the client was disabled due to traffic exhaustion, resetting traffic (individually or in bulk) automatically **re-enables** it (`enable = true`) and immediately propagates this change to node servers — no need to manually re-enable the client on the master and nodes.

Reset IP limit

Clears the accumulated IP log for the client (`POST /clearIps/:email`), lifting the temporary block imposed by exceeding the concurrent connection limit. Toast: "Log has been cleared".

Delete

Delete (`POST /del/:email`) — permanently removes the client. Confirmation dialog: title "Delete client {email}?", body "The client will be removed from all bound inbounds and its traffic record will be destroyed. This action cannot be undone.". Deletion removes the client from **all** inbounds and destroys its traffic record. Toast: "Client deleted".

8.4. Bulk operations

In the client list you can select multiple records (**Select all**, **Clear all**); the counter shows "{count} selected". Available actions for the selection:

- **Delete ({count})** (`POST /bulkDel`) — bulk delete. Confirmation: "Delete {count} clients?", "Each selected client is removed from all bound inbounds and its traffic record is destroyed. This action cannot be undone.". Toast: "Deleted clients: {count}", and on partial failure — "Deleted: {ok}, failed: {failed}".
- **Edit ({count})** / **Adjust** (`POST /bulkAdjust`) — bulk change of expiry and/or quota. Dialog "Edit {count} clients" with the tooltip "Positive values add, negative values subtract. Clients with unlimited expiry or traffic are skipped for the corresponding field.". Fields: **Add days**, **Add traffic (GB)**, and **Set flow**. Logic:
  - **Expiry:** clients with unlimited expiry (`expiryTime == 0`) are skipped ("unlimited expiry"); for clients with a date the expiry is shifted by the specified number of days; for clients in "after first use" mode (negative expiry) the waiting duration is adjusted. A reduction that exceeds the remaining time is skipped ("reduction exceeds remaining time/delay window").
  - **Traffic:** clients with unlimited traffic (`totalGB == 0`) are skipped ("unlimited traffic"); otherwise the quota is changed by the specified amount, not going below zero.
  - **Flow:** the **Set flow** dropdown lets you set or clear the XTLS flow for all selected clients at once. **No change** is selected by default. **Disable (clear flow)** clears the flow, while `xtls-rprx-vision` and `xtls-rprx-vision-udp443` set the corresponding vision flow. Setting a vision flow is applied only to inbounds that support flow; incompatible inbounds are left unchanged and marked as skipped, while clearing flow is always allowed.
  - **Auto-enable (as of 3.4.2):** a client disabled **solely because of exhaustion** (expired term or exceeded quota) is automatically re-enabled on extension — both locally and on its node — if the adjustment brings it back within the limits. Clients disabled manually or still exhausted remain disabled (the `enable` field is now written explicitly, so the state is preserved even for clients without an inbound).
  - If no days, traffic, or flow are specified: "Specify days, traffic, or flow before applying.". Toast: "Updated: {count}" / "Updated: {ok}, skipped: {skipped}".

Example: extend selected clients by 30 days and add 50 GB. In the **Edit** dialog set **Add days** = `30` , **Add traffic (GB)** = `50` . To instead subtract a week and reduce the quota by 10 GB, enter negative values: **Add days** = `-7` ,

Add traffic (GB) = `-10` (clients with unlimited expiry or unlimited traffic for the corresponding field will be skipped).

- **Bind ({count})** / **Unbind ({count})** (`POST /bulkAttach` / `bulkDetach`) — bulk bind/unbind of selected clients to selected inbounds. Targets are multi-user inbounds only. Unbind result: "Detached {detached}, skipped {skipped}.".
- **Sub links ({count})** — a summary table of subscription and JSON-subscription URLs for the selected clients with a **Copy all** button. If none of them have a subId: "None of the selected clients have a subscription ID.".
- **Add to group** and **Ungroup** — assign and remove the group label.
- **Enable ({count})** / **Disable ({count})** (`POST /bulkEnable` / `bulkDisable`) — bulk enable and disable of selected clients. **Enable** activates each selected client on all bound inbounds; clients with an exhausted traffic quota or expired expiry will be automatically disabled again. **Disable** immediately revokes access for clients but their records and accumulated traffic are preserved. Before execution the panel asks for confirmation, and after the operation shows a notification with the number of processed clients and, if applicable, the number that failed.

Reset traffic and delete by status

- **Reset traffic of all clients** (`POST /resetAllTraffics`) — zeroes the `up` / `down` counters of **all** panel clients. Confirmation: "Reset traffic of all clients?" and "The upload/download counters of all clients are reset to zero. Quotas and expiry are not affected. This action cannot be undone.". Toast: "Traffic of all clients reset".
- **Delete depleted** (`POST /delDepleted`) — deletes all clients whose **quota is exhausted** (`total > 0 and up + down >= total`) **or whose expiry has passed** (`expiry_time > 0 and expiry_time <= now`), provided `reset = 0` (clients with auto-renewal are not touched). Confirmation: "Delete depleted clients?", "All clients with an exhausted traffic quota or expired date are deleted. This action cannot be undone.". Toast: "Depleted clients deleted: {count}".

Export, import, and deleting unbound clients

When nothing is selected, the **More** menu on the **Clients** page offers three operations.

Export clients (`GET /clients/export`) opens a viewer with a JSON list of all clients in the `{client, inboundIds}` format with copy and download buttons (file `clients-export.json`). **Import clients** (`POST /clients/import`) opens an editor where you paste such a JSON and click **Import**: clients with `inboundIds` are created and bound to inbounds, clients without bindings are restored as standalone "bare" records, and already-existing emails are **never overwritten** — they end up in the skipped list. Toasts: "{count} clients imported", "{ok} imported, {failed} skipped".

Delete clients without inbound (`POST /clients/delOrphans`) — a destructive operation: deletes all clients not bound to any inbound, along with their traffic record, IP log, and external links. Confirmation: "Delete clients without an inbound?", "Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.". Toast: "{count} unattached clients deleted". The action is irreversible.

8.5. Search, filters, and sorting

Above the list there is a search bar ("Search email, comment, sub ID, UUID, password, auth…") — it searches by email, comment, subId, UUID, password, and auth. Result counter: "Showing {shown} of {total}".

The client list updates automatically: the panel fetches the current page every few seconds, so newly connected clients and changed sort order appear without a manual refresh (the loading indicator does not flash during background polling).

The **Client filters** panel lets you filter by status (category), protocol, bound inbound, expiry date range, used traffic range, presence of auto-renewal (**Has/None**), presence of Telegram ID and comment, and by group. On panels with nodes a **Nodes** multi-select appears: you can narrow the list to clients of selected nodes; a separate **Local panel** option filters clients of inbounds not bound to a node (the filter is visible only when nodes exist). Sorting: **Oldest/Newest first**, **Recently updated**, **Recently online**, **Email A→Z / Z→A**, **Most traffic**, **Most remaining**, **Expiring soonest**.

8.6. Badges and statuses

Status priority: depleted/expired → inactive → expiring soon → active.

- **Online** / **Offline** — a client with an active connection (present in the current online list) and **enabled**. The online list is updated by separate requests (`/onlines` , `/onlinesByGuid`).
- **Depleted** — quota consumed (`up + down >= totalGB`) **or** expiry passed (`expiryTime <= now`). Such a client is automatically disabled and is subject to **Delete depleted**.
- **Expiring soon** (expiring) — an enabled client whose time until expiry is below the threshold interval **or** whose remaining quota is below the threshold amount (thresholds are configured in panel settings). Not counted if the client is already depleted/disabled.
- **Inactive** (deactive) — a client with `enable = false` (disabled manually or by a background job).
- **Active** — enabled, not depleted, not expired, and still well above the thresholds.

---

9. Client groups

> This is a feature of this fork of 3X-UI. The original 3x-ui project (MHSanaei) has no concept of a "client group" — here a separate groups table, a **Groups** page in the panel menu, and the corresponding API methods have been added. If you migrate the configuration to the original 3x-ui, the group label simply will not be taken into account anywhere.

9.1. What a client group is and why you need it

A **group** is a named logical label that can be attached to one or more clients. It does not create a new way of connecting and is neither an inbound nor a node — it is purely an organizational tag that makes it convenient to filter clients and process them in bulk.

The key idea of the client model in this fork: **a client is a top-level entity identified by email** (the `email` field in the `clients` table has a unique index). The same client (one email with the same credentials) can simultaneously be listed in several inbounds and even on several nodes, including with different protocols. The group label is stored **once per client**, so it automatically propagates to all of that client's inbound bindings at once.

The group label is a logical grouping label:

Layer	Where it is stored	Field
Client record (DB)	`clients` table	`group_name` (default empty string `''`)
Groups directory (DB)	`client_groups` table	`name` (unique index, non-empty)
Inbound settings (Xray)	JSON `settings.clients[].group`	duplicated into each client object of each inbound the client belongs to

Why this is useful in practice:

- **One client across several inbounds/nodes.** If a client is "sold" as access to several inbounds at once (for example, different protocols or different nodes), the group lets you manage it as a single whole: reset traffic, delete, or rename the label — with a single operation across all of its inbounds.
- **Bulk operations and filtering.** On the **Clients** page the list can be filtered by group; on the **Groups** page bulk actions over all members of a group are available.
- **Organizing a large fleet of clients.** Labels such as `vip`, `trial`, `team-A` help sort thousands of clients into logical buckets without proliferating separate inbounds.

9.2. How a group relates to clients, inbounds, nodes, and protocols

This is the most important subsection to understand, because synchronizing the label is non-trivial.

A group describes a client, not an inbound. The label lives in the client record (`clients.group_name`). When a client is attached to several inbounds, on any change of group the panel goes through **all** inbounds the client belongs to and sets/clears the `group` field inside their Xray settings (`settings.clients[]`). Technically this

is done as follows: by the client's email, all inbounds the client belongs to are found, then in the JSON settings of each such inbound the client object with that email is edited. Therefore:

- The group **does not depend on the protocol.** One email can be a VLESS client in one inbound and a Hysteria client in another — its group label is still the same and applies to both (the credentials for each protocol are their own and are stored separately).
- The group **spans nodes.** Inbounds belonging to nodes differ from the inbounds of the main panel by the `nodeId` field (for the main panel's inbounds it is `null` / `0`). The group label propagates to client objects in inbounds regardless of whether it is a main or a node inbound — as long as a client with that email is present there.

The group label is resistant to synchronization from nodes and to settings rebuilds. This behavior is intentional:

- When a node sends a traffic snapshot, its data does **not** overwrite the local `group_name` and `comment` of the client in the panel DB. The group and comment are considered "panel-local" fields — the node does not manage them.
- When inbound settings are rebuilt, an empty `group` value in the incoming data does **not** reset the already-stored label. The group is managed specifically through the **Groups** page (and not through editing an inbound's Xray settings), so an "empty group" during an ordinary settings rebuild is interpreted as "do not touch" rather than "clear".

The practical takeaway: the only operations that **intentionally clear** the label are deleting a group and explicitly removing a client from a group (see below). An ordinary inbound edit or a background sync with a node will not lose the group.

9.3. The groups directory and "empty" groups

The list of groups on the page is built by merging two sources:

1. **Derived groups** — all non-empty `group_name` values actually occurring among clients, with a count of clients.
2. **Stored groups** — records from the `client_groups` table.

This union has an important effect: a group can exist **without a single client**. Such a group is created by the explicit "Add Group" button (a record in `client_groups`) and is shown in the list with a count of `0`. These records are what count as **empty groups**. The list is always sorted by name case-insensitively.

Summary counters on the page:

Field	What it shows
Total groups	The total number of groups (stored and derived together).
Clients with a group	How many clients have a non-empty group label.
Empty groups	How many groups exist without clients (count of `0`).
Clients in group	The number of clients in a specific group (a table column).

9.4. Group fields and columns

A group record in the `client_groups` table contains:

Field	Type	Default	Description
`Id`	int	autoincrement	Primary key of the group record.
`Name`	string	— (required)	The group name. Unique index, cannot be empty. In the UI it is the **Name** column.
`CreatedAt`	int64 (ms)	creation time	The moment the group record was created.
`UpdatedAt`	int64 (ms)	modification time	The moment of the last modification.

The table on the page displays at least the **Name** and **Clients in group** columns, as well as action buttons (see below).

9.5. Creating a group

The **Add Group** button.

Steps:

1. Click **Add Group**.
2. Enter the group name.
3. Confirm.

Backend behavior (`POST /panel/api/clients/groups/create` , body `{"name": "..."}`):

- The name is trimmed of leading and trailing whitespace. An empty name is rejected with the error "group name is required".
- If a group with that name already exists — the error "group already exists".
- On success a record is created in `client_groups` (initially without clients — this is an empty group).

Success message: **"Group "{name}" created."**

Example: create an empty group via the API. Prepare a set of labels in advance, before populating them with clients:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"name": "vip"}'
```

Response on success:

```
{ "success": true, "msg": "Group \"vip\" created.", "obj": null }
```

Calling it again with the same name returns `"success": false` and the message `group already exists`.

> Creating an empty group in advance is convenient when you want to prepare a set of labels and then populate them with clients via "Add clients…".

9.6. Renaming a group

The **Rename** button, the dialog title is **"Rename {name}"**.

Behavior (`POST /panel/api/clients/groups/rename`, body `{"oldName": "...", "newName": "..."}`):

- Both names are trimmed of whitespace. An empty old name gives the error "old group name is required", an empty new name "new group name is required".
- If the new name matches the old one — nothing is done (`0` clients affected).
- Otherwise the rename is performed atomically:
  - the record in `client_groups` is renamed;
  - for all clients with `group_name = oldName` the field is updated to `newName`;
  - in **all inbounds** the affected clients belong to (including node inbounds), the `group` value in the Xray settings is changed from the old one to the new one.
- After the rename the panel marks Xray as requiring a restart and sends a notification about the client change.

Messages:

- Success: **"Group renamed for {count} client(s)."**
- Name conflict in the UI: **"A group named "{name}" already exists."**

9.7. Adding clients to a group

The **Add clients…** button, title — **"Add clients to group "{name}""**.

The verbatim hint in the dialog:

> "Select clients to add to this group. Existing inbound bindings are kept; only the group label changes. Clients already in this group are not shown."

If there is no one to add, **"No other clients to add."** is shown.

Behavior (`POST /panel/api/clients/groups/bulkAdd`, body `{"emails": [...], "group": "..."}`):

- The group name is required (otherwise the error "group name is required"); an empty list of emails does nothing.
- If such a group does not yet exist either in `client_groups` or among the derived ones — it will be created automatically.
- For the selected emails the clients get `group_name = group`; the **bindings of clients to inbounds are not changed** — only the label is affected. Then the `group` field is set in all inbounds of these clients.

- The number of affected client records is returned; Xray is marked for restart.

Success message: "**Added {count} client(s) to {name}.**"

Example: label several clients with a group in a single request. Clients are specified by email, and inbound bindings are not changed:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

If the `vip` group does not exist yet, it is created automatically. After the request these clients get `group_name = "vip"` in their record, and in the Xray settings of each of their inbounds the client object gains a `"group": "vip"` field:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Removing clients from a group (without deleting the clients themselves)

The **Remove clients…** button, title — "**Remove clients from group "{name}"**".

The verbatim hint:

> "Select members to remove from this group. The clients themselves are kept (use "Delete group clients" for a full deletion)."

Behavior (`POST /panel/api/clients/groups/bulkRemove`, body `{"emails": [...]}`): technically this is the same as "Add to group" with an empty group. For the selected clients `group_name` is cleared, and in their inbounds the `group` field is removed from the Xray settings. The clients themselves and their inbound bindings remain.

Success message: "**Removed {count} client(s) from {name}.**"

9.9. Resetting group traffic

The **Reset traffic** button.

Confirmation dialog:

- Title: "**Reset traffic for group {name}?**"
- Text: "**Only the group's traffic counter is reset. Individual client counters are not affected.**"

Behavior: only the **group's own displayed counter** is zeroed out — the panel remembers the group's current `up/down` sum as a baseline marker and from then on shows `max(0, sum − baseline marker)` in the groups list. The `up` / `down` counters, quotas and the `enable` field of individual clients are **not changed**, and no Xray restart is required. This is a behavior change from earlier versions, where the reset zeroed out the traffic of all the group's clients.

Request: `POST /panel/api/clients/groups/resetTraffic` with the body `{"name": "<group name>"}`.

Success message: "**Traffic for group {name} has been reset.**"

9.10. Deleting a group and deleting group clients

The page has **two fundamentally different delete operations** — they are easy to confuse, so the distinction is critical.

9.10.1. Delete group (keep clients)

The "**Delete group (keep clients)**" button.

Dialog:

- Title: "**Delete group {name}?**"
- Text: "**This deletes the group and clears its label from {count} client(s). The clients themselves are not deleted.**"

Behavior (`POST /panel/api/clients/groups/delete`, body `{"name": "..."}`): the group record is deleted from `client_groups`, `group_name` is cleared for all of its clients, and the `group` field is removed from their inbounds. **The clients, their connections, and their traffic are kept.** Xray is marked for restart.

Success message: "**Cleared the group from {count} client(s).**"

9.10.2. Delete group clients (full deletion)

The "**Delete group clients**" button.

Dialog:

- Title: "**Delete all clients in {name}?**"
- Text: "**This permanently deletes {count} client(s) along with their traffic records. The group label is also cleared. This cannot be undone.**"

This is a destructive operation: it deletes the clients themselves (via a bulk deletion by email, endpoint `POST /panel/api/clients/bulkDel`), including their traffic records, and thereby removes them from all inbounds.

Messages:

- Success: "**Deleted {count} client(s).**"
- Partial result: "**{ok} deleted, {failed} skipped**"

> If the group is empty, actions over its members are unavailable — "**This group has no clients yet.**" is shown.

9.11. Relationship with the "Clients" page

The group label is visible and used outside the **Groups** page as well:

- The compact client record has a `group` field, so the client list shows group membership.
- The client list (`/panel/api/clients/list/paged`) accepts a `group` filter parameter: you can pass a single name or several names separated by commas. Matching is done on an "OR" basis within the field, case-insensitively. A special case: an empty element in the filter's group list means "clients without a group" (whose `group` is empty).
- The clients page response returns a `groups` array — the full list of names of existing groups, so the UI can build the filter dropdown.

Example: filtering clients by group. This request returns only clients labeled `vip` or `trial` (several names are comma-separated, with "OR" semantics):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

To get clients **without** a group, pass an empty element in the list — for example, the filter value `group=` (empty string) or `group=vip,` (the `vip` label plus clients with no group).

9.12. API endpoints summary

All group routes are mounted under `/panel/api/clients` :

Method and path	Purpose	Request body
`GET /panel/api/clients/groups`	List of groups with client counts	—
`GET /panel/api/clients/groups/:name/emails`	Emails of all members of a group (sorted by email)	—
`POST /panel/api/clients/groups/create`	Create an empty group	`{"name"}`
`POST /panel/api/clients/groups/rename`	Rename a group	`{"oldName","newName"}`
`POST /panel/api/clients/groups/delete`	Delete a group, keeping clients (clear the label)	`{"name"}`
`POST /panel/api/clients/groups/bulkAdd`	Add clients to a group (by email)	`{"emails":[...],"group"}`
`POST /panel/api/clients/groups/bulkRemove`	Remove clients from a group (clear the label)	`{"emails":[...]}`
`POST /panel/api/clients/bulkDel`	Full deletion of clients (used by "Delete group clients")	`{"emails":[...],"keepTraffic"}`

Example: a typical group-lifecycle scenario via the API.

```
# 1. Create the trial label
curl -s .../panel/api/clients/groups/create   -d '{"name":"trial"}'

# 2. Attach it to two clients
curl -s .../panel/api/clients/groups/bulkAdd  -d '{"emails":
["u1@example.com","u2@example.com"],"group":"trial"}'

# 3. Remove one member from the group (label-only)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Delete the group but keep the clients (label is just cleared)
curl -s .../panel/api/clients/groups/delete   -d '{"name":"trial"}'
```

Step 4 removes the group record and clears `group_name` from its clients, but the clients themselves, their connections, and their traffic remain. To permanently delete the clients themselves, use `bulkDel` instead.

Operations that change clients' label (`rename` , `delete` , `bulkAdd` , `bulkRemove`) mark Xray as requiring a restart and send a notification about the client change.

9.13. Traffic by group

New in version 3.3.0: in the **Groups** section (the "Clients" page, group management tab) the groups table now shows not only the number of clients in each group, but also the total traffic consumed by the group. The column is labeled **"Traffic used"**.

What the column shows

For each group row, the sum of traffic across all clients in that group is shown — that is, the added-up `up + down` (sent + received traffic) of all its members. This gives a quick answer to the question "how much did the whole group download/upload in total" without having to open clients one by one and add it up manually.

Alongside in the groups table are shown:

Column	What it means
Name	The group name
Clients	How many clients are labeled with this group (the column was previously named "Clients in group")
Upload	The total `up` (sent traffic) across all clients of the group
Download	The total `down` (received traffic) across all clients of the group
Traffic used	The total `up + down` across all clients of the group

Sent and received traffic are shown as separate **Upload** and **Download** columns, while the **Traffic used** column shows their sum. The client-count column is now simply named **Clients**.

The summary above the table additionally shows aggregates across all groups — **"Total groups"** and **"Clients with a group"**, and the total traffic is split into two cards: **"Total upload / download"** (with up/down arrows — the sent and received traffic of all groups separately) and **"Total traffic"** (with a pie-chart icon — their combined total).

How it is calculated

The calculation is performed with a single SQL query against the clients table with a `LEFT JOIN` to the traffic accounting table:

- by the group label field (`group_name`) clients are grouped, their count is computed — this is the "Clients in group";
- traffic is taken as the sum of `up + down` from the joined `client_traffics` table. That is, both the sent (`up`) and received (`down`) bytes are summed for each client;
- since the email is unique both in the clients table and in the traffic table, the join does not double-count one client's traffic.

Value specifics:

- **Clients without a traffic record** are counted in the member count but add 0 to the sum, so a freshly created group shows traffic of `0` .
- **Empty groups** (created but without clients) are also present in the list with a zero count and zero traffic: besides groups "derived" from client labels, the explicitly stored groups are mixed into the result, after which the list is sorted by name case-insensitively.
- Clients without a group label (`group_name` empty) are not included in the calculation.

Related actions

From the groups table, actions over the whole group are still available, including **"Reset traffic"** — it zeroes out **only the group's own counter** (the baseline marker), without touching the traffic of individual clients (see 9.9). After such a reset, the "Traffic used" column for that group shows `0` .

---

10. Subscriptions (Subscription)

A subscription is a mechanism that gives a client a single permanent URL through which the VPN client downloads and periodically refreshes a complete set of configurations. Instead of manually sending the user a separate link for each inbound, a single address of the form `https://domain:port/sub/<subId>` is provided. At that address the panel assembles on the fly all configurations tied to the given client and returns them in the format the client expects. When server settings change (new address, Reality key rotation, new inbound added) the client receives an up-to-date configuration on the next automatic refresh, with no action required from the user.

Subscriptions are served by a separate HTTP/HTTPS server inside the panel that starts independently of the web panel and listens on its own port. This is a security measure: the subscription port can be exposed to the outside world without exposing the panel port itself.

10.1. What subId is and how the link is formed

Every client in an inbound has a `subId` field (shown in the interface as "Subscription ID"). This value is the subscription key: the panel searches all inbounds for clients whose `subId` matches the requested one and combines their configurations into a single response.

- If several clients (within one inbound or across different inbounds) have the same `subId`, their configurations will be included in one subscription. This is the standard way to give a single user multiple servers/protocols via a single link.

Example: one user — two servers via one link. Suppose there are two inbounds (VLESS on server A and Trojan on server B). To give the user both configurations through a single link, set the same `subId` on both of his clients:

```
Inbound 1 (VLESS):  email = ivan@vpn,  subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn,  subId = ivan2025
```

Then at `https://sub.example.com:2096/sub/ivan2025` the panel will return both configurations at once. If you later add a third inbound with the same `subId`, it will appear for the user on the next automatic subscription update, without sending them a new link.

- If a client's `subId` field is empty, the link cannot be shared publicly. The interface indicates this with a tooltip: "This client has no subId, the share link is unavailable."

External links and client subscriptions (the "Links" tab)

The client form has a **"Links"** tab where you can attach additional configuration sources for an individual client that are merged specifically into that client's subscription (in RAW, JSON, and Clash formats):

- **Add External Link** — a third-party share link (`vless://` , `trojan://` , `ss://` , etc.). It is added to the output as-is; for JSON/Clash it is additionally parsed into a configuration object.
- **Add External Subscription** — the URL of an external subscription. The panel fetches it itself (with caching and a short timeout) and merges the resulting configurations into the client's combined list.

This is convenient for delivering additional servers on top of your inbounds to the client through the same single link. If the remote subscription response is too large it is no longer silently truncated: the panel returns an error and continues using the last successfully cached value.

- The value of `subId` cannot be set arbitrarily: on save, the panel checks that it contains no spaces, `/`, `\`, or control characters. The corresponding validation hint reads: "Subscription ID cannot contain spaces, '/', '' or control characters."

The final link is constructed as `<base>/<subPath>/<subId>` (see the section on subscription server settings and the "Reverse Proxy URI" field). If no client is found for the given `subId` (the client was deleted, the `subId` does not exist), the server returns HTTP 404 with an empty body. On an internal error — HTTP 500. VPN clients rely solely on the response code, so the error body is intentionally empty.

Inbound link order in the subscription

Each inbound has a **"Subscription sort order"** field (`subSortIndex`) — a number starting from 1, which defines the position of that inbound's links in the subscription output. Lower values come first; when values are equal the original creation order (by id) is preserved. The order applies to all output formats — plain text, subscription page, JSON, and Clash. This field does not affect the order of inbounds in the panel itself.

The field is edited in the inbound form alongside the share address settings and is synced to nodes by the usual rules. If at least one inbound has an order value other than 1, a compact **"Order"** column appears in the Inbounds list.

10.2. Subscription server settings

All subscription parameters are located in the panel settings under the **"Subscription"** tab. Each parameter is described below; the internal settings key and default value are shown in parentheses.

The section is divided into tabs: **"Panel settings"**, **"Info"**, **"Profile"**, **"Certificates"**, **"Happ"**, and **"Clash / Mihomo"**. Subscription title, support URL, profile page URL, announcement, and theme directory fields are on the "Profile" tab; Happ and Clash/Mihomo routing rules are on their respective tabs; the subscription update interval is on the "Info" tab.

Main parameters

Field (UI)	Key	Default	Description
Enable subscription	`subEnable`	`true` (enabled)	Starts a separate subscription server. Tooltip: "Subscription feature with separate configuration". If disabled, the subscription server does not start and none of the links work.
Listen IP	`subListen`	empty	IP address on which the subscription server accepts connections. Tooltip: "Leave empty by default to listen on all IP addresses".
Subscription port	`subPort`	`2096`	TCP port of the subscription server. Tooltip: "The port number for the subscription service should not be in use on the server" — the port must be free and must not conflict with the panel or Xray.

Field (UI)	Key	Default	Description
URI path	`subPath`	`/sub/`	Path at which regular subscriptions are served. Tooltip: "Must start with '/' and end with '/'".
Listening domain	`subDomain`	empty	Domain for which subscription access is permitted (Host validation). Tooltip: "Leave empty by default to listen on all domains and IP addresses". If set, requests with a different Host are rejected.

Security note: the default paths `/sub/` (and `/json/` for JSON) are widely known and easy to guess. The panel shows a warning: "Default subscription path '/sub/' is widely known — change it." with a similar warning for JSON. It is recommended to set a custom, non-obvious path.

TLS / certificate

Field (UI)	Key	Default	Description
Subscription certificate public key file path	`subCertFile`	empty	Full path to the certificate file (`.crt` / `fullchain`). Tooltip: "Enter the full path starting with '/'".
Subscription certificate private key file path	`subKeyFile`	empty	Full path to the private key file. Tooltip: "Enter the full path starting with '/'".

If both paths are set and the certificate loads successfully, the subscription server runs over **HTTPS**. If the fields are empty or the certificate cannot be read, the server falls back to **HTTP** (the error is written to the log). A valid TLS certificate also affects base URL construction: when the port is 443 with TLS or 80 without TLS, the port number is omitted from the link.

Update interval

Field (UI)	Key	Default	Description
Subscription update intervals	`subUpdates`	`12`	How often (in hours) the client application should re-fetch the subscription. Tooltip: "Update interval in client application (in hours)".

The value is sent to the client in the `Profile-Update-Interval` HTTP header; modern clients use it as the configuration auto-update period.

Response format and information

Field (UI)	Key	Default	Description
Encode	`subEncrypt`	`true`	Tooltip: "Encrypt the returned configs in the subscription". Technically this is not encryption but **Base64 encoding** of the entire regular subscription body (the format expected by most clients). When disabled, links are returned as plain text, one per line.
Show usage info	`subShowInfo`	`true`	Tooltip: "Show remaining traffic and expiry date after the config name". When enabled, traffic ( ) and expiry (e.g. `5D,3H`) markers are appended to the remark of each configuration; for an expired/ unavailable client `N/A` is shown.

Field (UI)	Key	Default	Description
Include email in remark	`subEmailInRemark`	`true`	Tooltip: "Include client email in the subscription profile name." Adds the client's email to the profile remark.

Remark Template

The display name (remark) of each configuration in the subscription is generated using the **remark template** — the **"Remark template"** field (`remarkTemplate`) on the **"Info"** tab of the subscription settings. The previous remark model builder (separate selection of inbound/email/external proxy parts and a separator character) has been removed from the interface; you now write a free-form name format and insert variables into it. The default value is `{{INBOUND}}-{{EMAIL}}| {{TRAFFIC_LEFT}}| {{DAYS_LEFT}}D` (meaning the profile name includes the client's email by default). If the field is left empty, the previous (non-configurable via the interface) remark model is used as a fallback.

Variables are grouped into **Client**, **Traffic**, and **Time & status** sections and are displayed next to the field as clickable `{{VAR}}` chips with a tooltip on hover; clicking inserts the token into the template, and a live preview is available. Each variable is substituted individually for the specific client at subscription generation time. A simplified single-brace notation is also accepted (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` , etc.) — the panel automatically normalizes it to the internal `{{...}}` format.

Available variables:

- **Client identification:** `{{EMAIL}}` (synonym — `{{USERNAME}}`), `{{INBOUND}}` (the inbound's own remark), `{{HOST}}` (host remark), `{{ID}}` (UUID), `{{SHORT_ID}}` (first 8 characters of UUID), `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}` , `{{PROTOCOL}}` , `{{TRANSPORT}}` .
- **Traffic:** `{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` (and their `*_BYTES` variants in exact bytes), `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}` .
- **Expiry and status:** `{{DAYS_LEFT}}` , `{{TIME_LEFT}}` , `{{EXPIRE_DATE}}` (`YYYY-MM-DD`), `{{JALALI_EXPIRE_DATE}}` (date in the Jalali calendar), `{{EXPIRE_UNIX}}` , `{{CREATED_UNIX}}` , `{{RESET_DAYS}}` , `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}` .
- **Connection:** `{{PROTOCOL}}` — protocol (VLESS, VMess, Trojan, etc.), `{{TRANSPORT}}` — transport network (tcp, ws, grpc, etc.), `{{SECURITY}}` — transport security (TLS, REALITY, NONE; displayed in uppercase). Like the traffic and expiry variables, these three variables only apply within the subscription body and are automatically stripped from the remark in panel-displayed links (QR / "Info") and on the subscription info page.

The template can be split into segments using a vertical bar `|` . A segment in which a variable produces an "unlimited" value (`∞`) — for example `{{TRAFFIC_LEFT}}` or `{{DAYS_LEFT}}` for a client without limits — is automatically hidden. In addition, the traffic and expiry block is shown once, on the client's first link, so it does not repeat in every configuration.

Example. The template `{{EMAIL}}| {{TRAFFIC_LEFT}}| {{DAYS_LEFT}}D` will produce `ivan@vpn 42.00GB 7D` for a client with 42 GB remaining and 7 days left, and simply `ivan@vpn` for an unlimited client (segments with `∞` are omitted).

As of version 3.4.2 the `{{EMAIL}}` variable (and the synonym `{{USERNAME}}`) is output only on the client's **first** link in the subscription body — just like the remaining traffic/expiry block; on the client's other links it is omitted.

On links displayed in the panel (QR code and "Info" windows on the Clients page) and on the subscription info page, the client's email is present in the profile name in the form "inbound-host-email" when a host is set, or "inbound-email" without a host. Traffic, expiry data, and Connection group variables are not substituted into these displayed names — they only work in the subscription body received by the VPN client.

If a client's traffic statistics row has become "orphaned" after deleting and re-creating an inbound, the `{{TRAFFIC_USED}}` variable (and other usage metrics) no longer shows `0.00B`: the panel additionally looks up statistics by the client's email and substitutes the correct used traffic.

| Remark template | `remarkTemplate` | `{{INBOUND}}`| `{{TRAFFIC_LEFT}}`| `{{DAYS_LEFT}}D` | Free-form template for the display name (remark) of each configuration, with `{{VAR}}` variable substitution. Substituted individually for each client when the subscription is generated. The previous "remark model" builder (inbound/email/external proxy selection and separator) has been removed from the interface and is used only as a fallback when the field is left empty. See "Remark Template" above for details. |

Profile metadata (response headers)

These strings are sent to the client in HTTP response headers and displayed in the VPN client as profile metadata. All of them are empty by default.

Field (UI)	Key	Header	Description
Subscription title	`subTitle`	`Profile-Title` (Base64-encoded)	"Subscription name visible to the client in the VPN app". For Clash it is also used as the name of the imported profile via `Content-Disposition`.
Support URL	`subSupportUrl`	`Support-Url`	"Support link displayed in the VPN client".
Profile URL	`subProfileUrl`	`Profile-Web-Page-Url`	"Link to your website displayed in the VPN client". If not set, the actual subscription request URL is used.
Announcement	`subAnnounce`	`Announce` (Base64-encoded)	"Announcement text displayed in the VPN client".

In addition, every response includes the `Subscription-Userinfo` header with the client's aggregated traffic data: `upload`, `download`, `total`, and `expire` (expiry timestamp in seconds). The client uses this to display the remaining traffic and expiry date.

Routing (Happ client only)

Field (UI)	Key	Default	Description
Enable routing	`subEnableRouting`	`false`	"Global setting for enabling routing in the VPN client. (Happ only)". Sent in the `Routing-Enable` header.
Routing rules	`subRoutingRules`	empty	"Global routing rules for the VPN client. (Happ only)". Sent in the `Routing` header.

| Hide server settings | `subHideSettings` | `false` | "Hide server settings in the subscription (Happ only)". When enabled, the Happ client hides the ability to view and change server parameters. This option only affects the Happ client. |

Incy routing (Incy client only)

For the **Incy** VPN client, the subscription settings have a dedicated **"Incy"** tab with two fields: an **"Enable routing"** toggle (`subIncyEnableRouting` , disabled by default) and a **"Routing rules"** text field (`subIncyRoutingRules`) in the format `incy://routing/onadd/<base64>` . When routing is enabled and the field is filled in, this string is appended as a separate line to the subscription body (raw format) — delivering the routing profile to the Incy client without conflicting with the Happ client's `Routing` header. These settings only apply to the Incy client.

Reverse proxy URI

Field (UI)	Key	Default	Description
Reverse proxy URI	`subURI`	empty	"Change the base URI of the subscription URL for use behind proxy servers".

If the field is empty, the panel constructs the base link address itself from the subscription domain and port (taking TLS into account). If the subscription is served through an external reverse proxy/CDN on a different domain or path, you set the final base URI in this field and all links will be built from it. Separate fields of the same kind exist for JSON (`subJsonURI`) and Clash (`subClashURI`).

If only the common `subURI` is set and the individual JSON and Clash fields are left empty, the links for those formats on the subscription page inherit the scheme and host from `subURI` (rather than the sub-server port and `http`) — so they match the reverse proxy address.

Example: subscription behind a reverse proxy. The subscription itself listens on `2096` , but is externally accessible via nginx/CDN at `https://cfg.example.com/u/` . To have links in the response built from the external address instead of the internal `domain:2096` , set the final base URI in the "Reverse proxy URI" field:

```
Reverse proxy URI: https://cfg.example.com/u
```

The resulting link will then look like `https://cfg.example.com/u/ivan2025` . For JSON and Clash formats, fill in the separate `subJsonURI` and `subClashURI` fields the same way if needed.

10.3. Output formats

A subscription can be delivered in three independent formats, each with its own endpoint that can be enabled or disabled separately.

Server address and nodes in the output

The server address in subscription links is substituted using the same share-address strategy as regular links and QR codes in the panel: "listen" — the routable bind address, "custom" — a user-defined address (`shareAddr`), "node" (default) — the node's address. For inbounds without an explicitly set strategy, the

subscription output is unchanged. This allows a node's inbound bound to a specific public IP to deliver a reachable address to clients. The strategy applies to raw, JSON, and Clash formats.

The node name (Node) is not appended to the profile remark (name) in the subscription: the client application shows only the inbound remark set by the administrator, without an internal suffix such as `@node–name`. To distinguish identically named entries in a multi-node subscription, assign them different remarks manually or use managed Hosts with their own Remarks.

If due to a sync mismatch between nodes the same client ends up in the internal JSON inbound twice, the subscription output automatically deduplicates such entries by email across all three formats, so duplicate profiles do not appear in the output.

Managed Hosts

The **Hosts** section (side menu item; overview page showing Total/Enabled/Disabled counts and a list) defines address overrides for subscription links. For each inbound you can add one or more **hosts** — endpoints that are substituted into the subscription links delivered to the client **in place of the inbound's address, port, and TLS parameters**. This is convenient for routing traffic through a CDN or relay without modifying the inbound itself.

Each host has:

- **Remark** and Description, binding to a specific **Inbound**, an **Enable** toggle, and assignment to **Nodes**.
- **Address** (empty — inherits the inbound address) and **Port** (`0` — inherits the inbound port); **Tags** (applied only in the RAW subscription).
- A **Security** tab — `same` / `tls` / `none` / `reality` with SNI, fingerprint, ALPN, pinned-cert, `allowInsecure`, and ECH.
- An **Advanced** tab — Host header, Path, VLESS flow, Mux, Sockopt, Final Mask, and exclusion of the host from individual subscription formats (raw / json / clash).
- A **Clash (mihomo)** tab — IP version, Mihomo X25519, shuffle host.

Hosts are ordered within their inbound and support bulk enable, disable, and delete. Managed Hosts replace the previous External Proxy array.

VLESS route. As of version 3.4.2 this is a single number `0–65535` (rather than a port list; hint — "a single VLESS route value (0-65535) baked into the UUID, e.g. 443; empty — none", placeholder `443`). The specified value is actually "baked" into the UUID of every generated subscription (raw / JSON / Clash): Xray reads bytes 6-7 of the UUID and masks them before authentication, so the client still matches. An empty or invalid value leaves the UUID unchanged.

Regular links (SUB) — Base64 / plain text

The base format, endpoint `subPath` (default `/sub/`). Always enabled (when subscriptions are enabled overall). Returns a list of Xray links (`vless://`, `vmess://`, `trojan://`, `ss://`, etc.) — one per line. When the "Encode" option (`subEncrypt`) is enabled, the entire list is Base64-encoded; when disabled it is returned as plain text. This format is understood by virtually all clients (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ, etc.).

Example: response body with "Encode" disabled. With `subEncrypt = false` the `/sub/` endpoint returns plain text — one link per line:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

With `subEncrypt = true` (default) the same list is Base64-encoded in its entirety and returned as a single string — the form expected by most clients.

JSON subscription (sing-box and compatible)

Endpoint `subJsonPath` (default `/json/`), enabled by a separate checkbox.

Field (UI)	Key	Default	Description
JSON subscription	`subJsonEnable`	`false`	"Enable/disable the JSON subscription endpoint independently."

Returns a complete JSON configuration (in the format understood by sing-box and derivative clients — Podkop, OpenWRT sing-box, Karing, NekoBox). Additional parameters are available for this format (the `subFormats` tab):

- **Mux** (`subJsonMux`, empty by default) — JSON multiplexing (Mux) settings that are injected into each stream outbound in the JSON subscription. "Transmission of multiple independent data streams over a single connection."
- **Final Mask** (`subJsonFinalMask`, empty by default) — "xray finalmask (TCP/UDP) masks and QUIC settings added to every JSON subscription stream. Requires a recent version of xray on the client." Configured via sub-fields: "Packets" (`packets`), "Length" (`length`), "Interval" (`interval`), "Max split" (`maxSplit`), "Noises" (`noises`: "Type"/`type`, "Packet"/`packet`, "Delay (ms)"/`delayMs`, "Apply to"/`applyTo`, "+ Noise" button), as well as "Concurrency" (`concurrency`), "xudp concurrency" (`xudpConcurrency`), and "xudp UDP 443" (`xudpUdp443`).
- **Routing rules** (`subJsonRules`, empty by default) — global rules added to the JSON configuration.

Clash / Mihomo subscription (YAML)

Endpoint `subClashPath` (default `/clash/`), enabled by a separate checkbox.

Field (UI)	Key	Default	Description
Clash / Mihomo subscription	`subClashEnable`	`false`	Enables generation of a YAML configuration for Clash and Mihomo clients.
Enable routing	`subClashEnableRouting`	`false`	"Add global Clash/Mihomo routing rules to generated YAML subscriptions."
Global routing rules	`subClashRules`	empty	"Clash/Mihomo rules prepended to each YAML subscription before MATCH,PROXY."

The response is returned with content type `application/yaml; charset=utf-8`. If a "Subscription title" (`subTitle`) is set, it is also sent in the `Content-Disposition` header (`attachment; filename*=UTF-8''<title>`) so that the Clash client names the imported profile accordingly.

The format of generated links and YAML is kept up to date for modern clients: Shadowsocks-2022 (SS2022) no longer Base64-encodes userinfo; Shadowsocks links with HTTP obfuscation are output in SIP002 format with the `obfs-local` plugin; Clash/Mihomo subscriptions include a complete set of XHTTP fields. No separate settings are required — links are simply recognized more correctly by clients.

> Note: this build supports exactly three formats — regular links (Base64/text), JSON (sing-box compatible), and Clash/Mihomo (YAML). There is no separate Outline format in the subscription server.

10.4. Subscription info page and QR codes

If the subscription link is opened in a browser (or the `?html=1` or `?view=html` query parameter is explicitly appended, or the `Accept: text/html` header is sent), the server returns a visual **subscription info page** ("Subscription Info") instead of a raw response. VPN clients still receive the machine-readable response because they do not request HTML.

The page (a single-page application built with Vite) shows:

- **Subscription information** (Descriptions block):
  - "Subscription ID" — the `subId` value;
  - "Status" — "Active", "Inactive", or "Unlimited". The "inactive" status is set if the client is disabled, has exhausted the traffic limit, or has expired;
  - "Downloaded" and "Uploaded" — traffic volumes;
  - "Total limit" — the traffic limit, or `∞` if unlimited;
  - "Expiry date" — the expiry date, or "No expiry";
  - remaining traffic and the time of last online activity.
  - Dates are displayed in the Gregorian or Jalali calendar depending on the panel's "Calendar Type" setting (`datepicker`, default `gregorian`).
- **Subscription links**: for each enabled format — a separate row with a colored tag (green **SUB**, purple **JSON**, gold **CLASH**), a copy button, and a **QR code** button (popup, 240 px). The JSON and CLASH rows appear only if the corresponding format is enabled in settings.
- **Individual links** ("Copy link"): a full list of individual configurations included in the subscription, each with its own protocol tag, copy button, and QR code (QR codes are not generated for post-quantum links).
- **"Copy all configurations" button** (above the individual links list): copies all configuration links to the clipboard at once (each on its own line) without requiring them to be copied one by one; a "All configurations copied" notification is shown on completion.
- **Quick-import buttons for apps** (platform dropdowns): for Android — v2box, v2rayNG (deep link `v2rayng://install-config?url=…`), Sing-box, V2RayTun, NPV Tunnel, Happ (`happ://add/…`), Incy (`incy://add/…`); for iOS — Shadowrocket (via `flag=shadowrocket` parameter), v2box (`v2box://install-sub?url=…&name=…`), Streisand (`streisand://import/…`), V2RayTun, NPV Tunnel, Happ, Incy. These buttons either open the target app's deep link with the subscription address pre-filled, or copy the link to the clipboard.

The info page is returned with no-cache headers (`Cache-Control: no-cache`) so that the client always sees up-to-date traffic and expiry information.

10.5. Custom subscription page templates

Starting from 3.3.0, the default subscription landing page can be replaced with a custom HTML template. By default the subscription URL returns the built-in page, but if a directory containing your own template is specified, the panel will render it and inject the current client data (traffic, expiry, links, etc.) into it.

Important: the panel does **not** ship any ready-made templates — you must create your own theme from scratch. Authoring instructions and the list of available variables are in `docs/custom-subscription-templates.md`.

Where to enable it

The theme directory is set in the panel settings:

Settings → **Subscription** → **"Subscription info" section**, field **"Subscription theme directory"** (`subThemeDir`).

Field description in the interface: "Absolute path to the folder containing the custom template (index.html/sub.html) for the subscription page (e.g., /etc/3x-ui/sub_templates/my-theme/). Leave empty to use the default page."

The "Subscription theme directory" field description contains a **"Template guide ↗"** link to documentation on creating custom subscription page themes.

Adjacent fields in the same section whose values are available in the template:

- **"Subscription title"** (`subTitle`) — the name visible to the client;
- **"Support URL"** (`subSupportUrl`) — a link to support.

Settings parameter

Parameter	Default	Purpose
`subThemeDir`	`""` (empty)	Absolute path to the directory containing your HTML template. Empty = built-in default page.

How to set up your own template

1. Create a theme folder on the server (anywhere you like), for example `/etc/3x-ui/sub_templates/my-theme/`.
2. Place an HTML file named `index.html` or `sub.html` inside it.

Example: theme path. The final layout on the server and the field value in settings:

```
/etc/3x-ui/sub_templates/my-theme/
└── index.html        (or sub.html — it takes priority)
```

```
Settings → Subscription → "Subscription theme directory":
/etc/3x-ui/sub_templates/my-theme/
```

The path must be **absolute** (start with `/`). If the folder contains neither `index.html` nor `sub.html`, the panel will serve the built-in page. 3. In the panel, open **Settings** → **Subscription** and enter the **absolute** path to that folder in the "Subscription theme directory" field. 4. Save the settings.

File selection and rendering behavior:

- If the directory contains `sub.html`, it is used; otherwise `index.html` is used. That is, `sub.html` takes priority over `index.html`.
- The template is rendered by the standard Go `html/template` engine.
- The parsed template is **cached** and re-read from disk only when the file's modification time changes. Therefore, template edits are picked up without restarting the panel, but without the overhead of reading/parsing on every request.
- The response is buffered in full before being sent to the client: if the template fails during execution, a partially generated (broken) page will not reach the user.

Default behavior and fallback

- Field is empty → the built-in SPA page is returned (data is injected into `window.__SUB_PAGE_DATA__`).
- Path does not exist or is not a directory → the default page is used.
- Directory contains neither `index.html` nor `sub.html` → a warning "subThemeDir set but no usable template found" is written to the log, and the default page is returned.
- Template file exists but fails to parse → an error "custom template parse failed" is written to the log, and the default page is returned.
- Error during template execution → "custom template execution failed" is written to the log, and the default page is returned.

In other words, any problem with the custom template does not "break" the subscription — the panel always falls back to the built-in page. All subscription pages (custom and default alike) are returned with no-cache headers (`Cache-Control: no-cache, no-store, must-revalidate`) so that clients always receive fresh traffic and expiry data.

Available template variables

A set of client subscription data is passed in the template context. Access via `{{ .name }}`:

Variable	Type	Description
`{{ .sId }}`	string	Subscription ID (UUID).
`{{ .enabled }}`	bool	Whether the client/subscription is enabled.
`{{ .download }}`	string	Formatted download volume (e.g. "2.5 GB").
`{{ .upload }}`	string	Formatted upload volume.
`{{ .total }}`	string	Formatted total traffic limit.
`{{ .used }}`	string	Formatted used traffic (download + upload).
`{{ .remained }}`	string	Formatted remaining traffic.

Variable	Type	Description
`{{ .expire }}`	int64	Expiry time — Unix timestamp in **seconds** (`0` = no expiry). Multiply by 1000 for a JS `Date`.
`{{ .lastOnline }}`	int64	Last online time — Unix timestamp in **milliseconds** (`0` = never).
`{{ .downloadByte }}`	int64	Download in exact bytes.
`{{ .uploadByte }}`	int64	Upload in exact bytes.
`{{ .totalByte }}`	int64	Total limit in exact bytes.
`{{ .subUrl }}`	string	Subscription page URL.
`{{ .subJsonUrl }}`	string	JSON subscription configuration URL.
`{{ .subClashUrl }}`	string	Clash/Mihomo configuration URL.
`{{ .subTitle }}`	string	Subscription title from settings (may be empty).
`{{ .subSupportUrl }}`	string	Support URL from settings (may be empty).
`{{ .links }}`	[]string	List of configuration strings (VMess, VLESS, etc.). Iterate with `{{ range .links }} … {{ end }}`.
`{{ .emails }}`	[]string	List of emails associated with the subscription.
`{{ .datepicker }}`	string	Current panel calendar format: `gregorian` or `jalali` (taken from the "Calendar Type" setting; defaults to `gregorian` if empty).

Minimal example template body using some of the variables:

```
<h1>{{ .subTitle }}</h1>
<p>Used: {{ .used }} of {{ .total }} (remaining {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

Example: expiry date from `expire`. The `{{ .expire }}` field is a Unix timestamp in **seconds**, so for JavaScript it must be multiplied by 1000; a value of `0` means "no expiry":

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? 'No expiry'
    : 'Until ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Note that `{{ .lastOnline }}` is already in **milliseconds** — it does not need to be multiplied by 1000.

---

11. Xray: routing, outbounds, DNS, and extensions

The **"Xray Settings"** section is an editor for the Xray-core configuration template, from which the panel generates the final `config.json` to launch the core. The section tooltip reads: *"The Xray configuration file is created based on the template."* Unlike inbounds (which are stored separately in the database and injected into the template at config-build time), everything else — logs, routing, outbounds, DNS, policy, statistics — is configured here.

> Important: the template value is stored in the database under the key `xrayTemplateConfig`. On save, the panel runs it through a series of automatic transformations (see 11.11). Any syntactically invalid JSON will be rejected with the error *"xray template config invalid"*.

Location in the menu: "Outbounds" and "Routing"

"Outbounds" and **"Routing"** are separate sidebar menu items (directly below "Hosts", above "Panel Settings"), each with its own URL: `/outbound` and `/routing`. Direct links to these pages and page reloads work as expected. The **"Xray Configurations"** submenu retains only: Basic, Balancer, DNS, and Advanced Template. In the description below, sections 11.3 and 11.4 correspond to the "Routing" and "Outbounds" pages.

11.1. Editor structure: tabs/modes

The editor offers several template display modes (filters by JSON section):

Mode	What it shows
Basic	Base sections (Log, basic routing, general settings)
Advanced Template	Full Xray JSON template
All	All sections simultaneously

Logical groups of settings within the editor:

- **General Settings** (tooltip: *"These parameters describe general settings"*).
- **Log** (see 11.10).
- **Basic Connections**: blocks and direct routes.
- **Inbounds** (tooltip: *"Modify the configuration template to connect specific clients"*).
- **Outbounds** (see 11.4).
- **Balancer** (see 11.5).
- **Routing** (tooltip: *"The priority of each rule is important!"*, see 11.3).
- **DNS / Fake DNS** (see 11.6).

11.2. General Settings

Freedom Protocol Strategy

Field	Label	Description	Default
`FreedomStrategy`	**Freedom Protocol Strategy Setting**	Network output strategy for a direct (freedom) outbound. Tooltip: *"Set the network output strategy in the Freedom protocol"*. Controls the `domainStrategy` field inside `settings` of an outbound with the `freedom` protocol.	In the reference template, `domainStrategy` for the `direct` freedom outbound is **`AsIs`** (the address is not resolved, passed as-is).

`domainStrategy` for freedom (Xray-core values): `AsIs` (do not resolve domain on the server side), as well as the `UseIP` / `UseIPv4` / `UseIPv6` family and their "forced" variants `ForceIP*`, which force the exit server to resolve the domain and connect to the resulting IP. Switch to `UseIPv4` if the exit server has no IPv6 or you need to force IPv4-only connections.

Freedom Happy Eyeballs (IPv4/IPv6)

Field	Label	Description
`FreedomHappyEyeballs`	**Freedom Happy Eyeballs (IPv4/IPv6)**	Tooltip: *"Dual-stack dialing for direct (freedom) outbound — useful on exit servers with both IPv4 and IPv6."* Enables the Happy Eyeballs algorithm (simultaneous attempts over both address families) for the freedom outbound.
try delay	(tooltip)	*"Milliseconds before trying the other address family. 150–250ms is a good starting point."* Delay before switching to the alternative address family. Recommended range: 150–250ms.

Overall Routing Strategy

Field	Label	Description	Default
`RoutingStrategy`	**Domain Routing Strategy Setting**	Overall DNS resolution strategy for routing. Tooltip: *"Set the overall DNS resolution routing strategy"*. Controls the `routing.domainStrategy` field.	In the reference template, `routing.domainStrategy` = **`AsIs`**.

`routing.domainStrategy` determines how IP routing rules are matched against domain queries: `AsIs` (domain rules only, no resolution),
`IPIfNonMatch` (if the domain doesn't match any rule — resolve and check IP rules), `IPOnDemand` (resolve immediately when an IP rule is encountered). For IP rules (e.g., `geoip:*`) to fire on domain queries, `IPIfNonMatch` is typically required (see 11.2).

Outbound Test URL

Field	Label	Description	Default
`outboundTestUrl`	**Outbound Test URL**	URL for connectivity checks when testing an outbound. Tooltip: *"URL for testing outbound connection"*. Stored separately from the template under the key `xrayOutboundTestUrl`.	`https://www.google.com/generate_204`

The value is sanitized. When an actual outbound test is run, it is additionally validated as a public URL — this protects against SSRF: a client cannot supply an arbitrary (including internal) URL; the test URL is always taken from the server-side setting. An empty value is replaced with the default `generate_204` on save/test.

Block BitTorrent

Field	Label	Description
`Torrent`	**Block BitTorrent**	Adds a rule to `routing.rules` that routes traffic with `protocol: ["bittorrent"]` to the `blocked` outbound. This rule is present by default in the reference template.

Connection Limits

Tooltip: *"Connection-level policies for users at level 0. Leave the field empty to use Xray's default value."* These parameters are written into `policy.levels.0`.

Field	Label	Description	Default
`connIdle`	**Idle Timeout** (seconds)	*"Closes a connection after it has been idle for the specified number of seconds. Reducing this value frees memory and file descriptors faster on heavily loaded servers (Xray default: 300)."*	empty → Xray default **300**
`bufferSize`	**Buffer Size** (KB)	*"Size of the internal buffer per connection in KB. Set to 0 to minimize memory usage on servers with low RAM (Xray default depends on the platform)."* Placeholder: **"auto"**.	empty → platform-dependent; `0` — minimize

Example (`policy.levels.0`). Fields from this group are written into the level-0 policy. On a heavily loaded server with low RAM, you can speed up resource release as follows:

```
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

Here the connection is closed after just 120 seconds of idle time (instead of the default 300), and `bufferSize: 0` minimizes buffer memory usage. A field left empty in the form simply won't appear in the JSON — and Xray will apply its own default value.

11.3. Routing Rules (routing)

The list of `routing.rules`. **Order is critical** (*"The priority of each rule is important!"*): rules are evaluated top to bottom, and the first match wins. Tooltip: *"Drag to change order"*. Order-control buttons: **First**, **Last**, **Move Up**, **Move Down**.

Each rule has `type: "field"`. Buttons: **Create Rule**, **Edit Rule**. Tooltip for list fields: *"Comma-separated items"*.

On the "Routing" page, the **"Import Rules"** and **"Export Rules"** buttons are grouped in a **"more"** dropdown menu — just like on the "Outbounds" page. The **"Export Rules"** button does not download a file immediately; instead it opens a modal with a JSON preview and **"Copy"** and **"Download"** buttons: the content can be reviewed before saving. Outbound export on the "Outbounds" page works the same way.

Route Tester

The Routing tab has a **Route Tester** sub-tab — it asks the running Xray which outbound would handle a specific connection, without sending real traffic. Specify a domain or IP, port, network (TCP/UDP), and optionally an inbound and sniffed protocol (`http` / `tls` / `quic` / `bittorrent`), then click **Test Route**. The decision comes directly from the live routing engine.

The response shows the matched outbound, and when a balancer is used — also the balancer tag. If no rule matched, the tester reports that traffic goes to the default outbound (first in the `outbounds` list). This is handy for verifying rule order before relying on it.

Enabling and disabling individual rules

An individual routing rule can be temporarily **disabled** with a toggle, without deleting it. The rule table has an **"Enable"** column with a toggle (Switch), and the rule form has an **"Enable"** field — also a toggle. A disabled rule is not included in the final Xray configuration, but remains in the template and can be re-enabled at any time.

The statistics service rule (`inboundTag: ["api"] → outboundTag: "api"`) cannot be disabled — its toggle is locked to prevent breaking the panel's traffic accounting (see 11.11).

Rule form fields

Form field	Label	JSON field	Description
Source	**Source**	`source`	Source IP addresses/subnets. Comma-separated list.
Source Port	**Source Port**	`sourcePort`	Source port(s).
Destination	**Destination**	`domain` + `ip` + `port`	Target domains, IPs, and ports. Domains support the prefixes `domain:` , `full:` , `regexp:` , `keyword:` , and `geosite:*` ; IPs support `geoip:*` and CIDR.
Network	—	`network`	`tcp` , `udp` , or `tcp,udp` . As of 3.4.2 it is written to the config in lowercase (`tcp` / `udp`) and displayed in uppercase (`TCP` / `UDP`) in the routes table; empty — any network.
Protocol	—	`protocol`	`http` , `tls` , `bittorrent` (determined via sniffing).

Form field	Label	JSON field	Description
User	**User**	`user`	Filter by e-mail/user identifier.
Attributes / Value	**Attributes / Value**	`attrs`	HTTP header attributes for matching.
VLESS route	**VLESS route**	—	Routing by the route field for VLESS.
Inbound Tags	**Inbound Tags**	`inboundTag`	One or more inbound tags to which the rule applies (including built-in `api` and the DNS tag from DNS settings). In inbound lists, displayed as "tag (remark)" if the inbound has a separate remark, otherwise just the tag; saved rules still store only tags.
Outbound Tag	**Outbound Tag / Outbound Connection**	`outboundTag`	Where to route matched traffic.
Balancer Tag	**Balancer Tag / Balancer**	`balancerTag`	Tooltip: *"Routes traffic through one of the configured load balancers"*.

> Mutual exclusion of `outboundTag` and `balancerTag`: *"It is impossible to use balancerTag and outboundTag at the same time. If used simultaneously, only outboundTag will work."* In a single rule, set either the outbound tag or the balancer tag.

Built-in rules of the reference template

In the standard `config.json`, the `routing` section contains three rules (in this order):

1. `inboundTag: ["api"] → outboundTag: "api"` — a service rule for the panel's statistics gRPC API.
2. `ip: ["geoip:private"] → outboundTag: "blocked"` — blocks private address ranges.
3. `protocol: ["bittorrent"] → outboundTag: "blocked"` — blocks BitTorrent.

> The `api → api` rule is always automatically moved to position 0 on save (see 11.11), so that a statistics request is not consumed by an upstream catch-all rule.

Example rule. Route all traffic to Russian sites and private networks directly (bypassing the proxy), and everything else to a balancer. Order matters: the "route directly" rule must appear above the catch-all. In `routing.rules`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

For IP rules (`geoip:ru`) to fire on domain queries, you typically need `routing.domainStrategy: "IPIfNonMatch"` at the top level of routing (see 11.2).

Pre-configured routing groups (Basic Connections)

In "Basic Connections" mode, the panel helps you build typical rules from ready-made lists:

Group	Fields	Tooltip
Block by protocols/ sites	—	*"Configure to prevent clients from accessing certain protocols"*
Block by country	**Blocked IPs**, **Blocked Domains**	*"These settings will block traffic depending on the destination country."*
Direct connections	**Direct IPs**, **Direct Domains**	*"Direct connection means that certain traffic will not be redirected through another server."*
IPv4 Rules	—	*"These settings will allow clients to route to target domains via IPv4 only"*
WARP Rules	—	*"These options will route traffic to a specific destination via WARP."*
NordVPN Routing	—	*"These options will route traffic to a specific destination via NordVPN."*

MTProto inbound: routing Telegram traffic through Xray

The MTProto inbound has a **"Route through Xray"** toggle (disabled by default) and an optional **Outbound** selector. When enabled, the panel adds a loopback SOCKS bridge to the Xray config with the inbound's own tag, and mtg routes Telegram traffic through it. After that, the outgoing Telegram traffic is controlled by the router: it can be matched with normal rules on the Routing tab by the inbound tag, or forced into a selected outbound or balancer via the **Outbound** field. Leave **Outbound** empty to let routing rules decide.

11.4. Outbounds (outgoing connections)

The list of `outbounds`. Buttons: **Create Outbound**, **Edit Outbound**. Tooltip: *"Modify the configuration template to define outgoing connections for this server"*.

The reference template has two mandatory outbounds:

- `protocol: "freedom"`, `tag: "direct"` — direct exit to the internet (with `domainStrategy: "AsIs"` and `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole"`, `tag: "blocked"` — a black hole for blocked traffic.

Common outbound form fields

Field	Label	Description
Tag	**Tag** (tooltip: *"Unique tag"*)	Unique outbound identifier. Placeholder: *"unique-tag"*. Validation: *"Tag is required"*, *"Tag is already used by another outbound"*.
Protocol	—	Outbound type (see below).
Address / Port	**Address** / Port	Connection target. Address and port are required.

Field	Label	Description
Send Through	**Send Through**	Local IP address of the outgoing interface (`sendThrough`). Placeholder: *"local IP"*.
Dialer proxy (chain)	—	Tooltip: *"Connect this outbound through another outbound (by tag) to build a proxy chain. Leave empty for a direct connection."* Placeholder: *"Select outbound to chain"*. Implemented via `streamSettings.sockopt.dialerProxy` .

The **Dialer Proxy** dropdown shows not only local outbounds but also outbound tags from subscriptions — so a chain can also be built through a subscription-fetched exit. The blackhole outbound and the outbound currently being edited are still excluded from the list. Leave the field empty for a direct connection.

Supported outbound protocols

Protocols supported by the form:

- **`freedom`** — direct exit. Fields `settings.domainStrategy` , `finalRules` (see below), Happy Eyeballs. Cannot be tested (*"Outbound has no testable endpoint"*).
- **`blackhole`** — drops traffic. Field **Response Type**. Not testable.
- **`socks`** , **`http`** — `settings.servers[]` list with `address` / `port` ; field **Auth Password**. For the **`http`** protocol, below the **Username**/**Password** fields there is a **Headers** editor — key/value pairs for CONNECT headers sent to the upstream HTTP proxy. These headers are preserved when the outbound is re-opened and saved (previously they were lost). Note: only settings-level headers (`settings.headers`) apply; per-server headers are ignored by xray-core.
- **`vmess`** — `settings.vnext[]` (`address` / `port`).
- **`vless`** — `settings.address` / `settings.port` .
- **`trojan`** , **`shadowsocks`** — `settings.servers[]` .
- **`wireguard`** — `settings.peers[]` with `endpoint` , plus keys (see 11.8).
- **`hysteria`** — `settings.address` / `settings.port` (UDP transport).

For the **loopback** outbound type, a **Sniffing** block is available with the same parameters as an inbound: enable, **destOverride**, **Metadata Only**, **Route Only**, and the **excluded domains** list.

In the **UDP** mask (FinalMask) for **Hysteria2**, additional modes are available. The **Salamander** mask has a **Mode** selector with **Salamander** and **Gecko** values: Gecko mode adds random packet padding with **Min**/**Max** size fields (`packetSize` , range 1–2048, default 512–1200) — this protects against fingerprinting by packet length. The **Realm** mask (UDP hole-punching) has gained an optional **TLS Config** block with fields **Server Name** (SNI), **ALPN** (`h3` / `h2` / `http/1.1`), **Fingerprint** (uTLS), and an **Allow Insecure** toggle.

Example: chain through an upstream SOCKS. The `upstream` outbound connects to an external SOCKS5 proxy, and `chained` sends its traffic through it (`dialerProxy`), forming a chain. In `outbounds` :

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

A routing rule with `outboundTag: "chained"` will then route traffic to the internet through `upstream`.

Importing an outbound from a share link

An outbound can be imported from a share link (`vless://`, `vmess://`, etc.). On import, **xmux** (XHTTP) multiplexer settings passed in the `extra=` block of the link are also preserved: after import their values are populated into the **XMUX** sub-form of the created outbound.

Mux fields (multiplexing)

Max Concurrency, **Max Connections**, **Max Reuse**, **Max Requests**, **Max Reuse Seconds**, **Keep Alive Period**. These parameters configure the mux/XUDP behavior of the outbound.

Sockopts (socket settings)

The **Sockopts** group: **Keep Alive Interval**, **Mark (fwmark)**, **Interface**, **IPv6 Only**, **Accept Proxy Protocol**, **Proxy Protocol**, **TCP User Timeout (ms)**, **TCP Keep-Alive Idle (s)**. The dialer proxy chain is also configured here.

Freedom finalRules (overriding private IP blocking)

For a freedom outbound, the **Final Rules** group is available:

Field	Label	Description
`overrideXrayPrivateIp`	**Override Xray Default Private IP Block**	Removes Xray's built-in restriction on outgoing connections to private IPs.
`action`	**Action**	`allow` (as in the reference template: `finalRules: [{action: "allow"}]`), `redirect` (**Redirect**), or others.
`blockDelay`	**Block Delay (ms)**	Delay before dropping the connection.
`redirect` / `fragment`	**Redirect** / **Fragment**	Redirect and traffic fragmentation actions.

fragment mask: per-segment Lengths and Delays

In the **fragment** mask (the fragment type in FinalMask, for TCP), the single Length and Delay fields are replaced by **Lengths** and **Delays** lists: each segment can have its own length range (e.g., `100–200`) and delay in milliseconds (e.g., `10–20` or `0`). List entries can be added and removed; previously saved single values are automatically migrated into a single-element array.

Other form fields

- **UDP over TCP** and **UoT Version** — for shadowsocks-like protocols.
- **No gRPC Header**, **Uplink Chunk Size** — gRPC transport parameters.
- TLS/uTLS fields: **Verify Peer Name**, **Pinned SHA256**, **Short ID**, **Vision testpre**, placeholder "server name".

Testing outbounds

Buttons: **Test**, **Test All**. States: **Testing connection…**, **Test successful**, **Test failed**, **Failed to test outbound connection**. Result: **Test Result**, latency in milliseconds.

Two modes (tooltip: *"TCP: fast dial-only probe. HTTP: full request via xray."*):

- **TCP** (`mode=tcp`) — a simple dial to `host:port`, executed in parallel across all endpoints, ~5s timeout. Checks only TCP reachability, does not validate the proxy protocol. For `freedom` / `blackhole` /the `blocked` tag, returns *"Outbound has no testable endpoint"*.
- **HTTP** (`mode=http` or empty) — spins up a temporary Xray instance, runs a real HTTP request (probe URL = the server-side `outboundTestUrl`), measures actual latency. Authoritative but expensive: serialized by a global lock (*"Another outbound test is already running, please wait"*). Single-attempt timeout is 10s, result wait window is 15s (increased so that healthy outbounds on slow or tunneled links are not marked as "Failed"). On failure, the actual cause (DNS error, connection refused, deadline exceeded, TLS error, etc.) is written to the panel/Xray log, which generic timeout messages point to.

> UDP protocols (`wireguard`, `hysteria`) and UDP transports (`kcp`, `quic`, `hysteria`) are **always** tested in HTTP mode, even if TCP was requested — a bare UDP dial cannot distinguish a "live" endpoint from a "dead" one. For wireguard, `noKernelTun: true` is forced in the test configuration.

Batch testing and phase breakdown

Test and **Test All** in HTTP mode spin up a single shared temporary Xray instance for the batch of outbounds, create a loopback SOCKS inbound with a routing rule for each, and send a real HTTP request through it in parallel; **Test All** checks outbounds in batches. **Test All** also tests outbounds obtained from subscriptions (the read-only "from subscriptions" table) — their rows are also highlighted with the test result. The `freedom` ("direct") and `dns` outbounds are not tested in either mode (they are not proxies): the test button is disabled for them, **Test All** skips them, and the server-side guard blocks their HTTP test even on direct API calls. Beyond pass/fail, the popup result shows the HTTP response status and a phase breakdown: **Proxy connect** (connecting to the proxy), **TLS via outbound** (TLS through the outbound), and **First byte** (time to first byte) — this helps pinpoint which step caused a delay or failure.

Outbound traffic statistics

The panel maintains traffic counters by tag (`up` / `down` / `total`). The reset button resets counters for a specific tag or for all tags (`tag = "–alltags–"`). The **Account Information** and **Outbound Status** fields show a summary.

11.5. Balancers

The list of `routing.balancers` . Buttons: **Create Balancer**, **Edit Balancer**.

The Balancers tab has live-state columns: **Live Target** shows the current active target of the balancer in the running Xray, and **Override** lets you manually override the target selection (**Auto (strategy)** returns selection to the strategy). The state is refreshed with a dedicated button. If the balancer is not yet active in the running Xray, the panel will suggest saving changes or starting Xray first.

Field	Label	Description
Tag	**Tag** (tooltip: *"Unique tag"*)	Unique identifier. Placeholder: *"unique balancer tag"*. Validation: *"Tag is required"*, *"Tag is already used by another balancer"*.
Selectors	**Selectors**	List of outbound tags (by substring) from which the balancer chooses an exit. At least one must be selected: *"Select at least one outbound"*.
Fallback	**Fallback**	Fallback outbound tag if no selector matched.
Strategy	**Strategy**	Selection algorithm (see below).

Strategy and observation parameters

The strategy (`strategy.type`) determines how the balancer selects an outbound from the selectors. Xray-core values: `random` (random), `roundRobin` (round-robin), `leastPing` (minimum latency based on observatory results), `leastLoad` (minimum load). For `leastLoad` / `leastPing` , parameters from `strategy.settings` are used:

Field	Label	Description
`expected`	**Expected**	Placeholder: *"optimal number of nodes"* — target number of live nodes.
`maxRtt`	**Max RTT**	Upper bound for acceptable RTT when selecting candidates.
`tolerance`	**Tolerance**	Tolerance when comparing latency/load values.
`baselines`	**Baselines**	Latency thresholds for grouping nodes.
`costs`	**Costs**	Weight coefficients (cost) for individual tags.

Strategy examples. The `strategy` block lives inside the balancer (in JSON — alongside `tag` and `selector`):

```
"strategy": { "type": "random" }      // random selection from selectors
"strategy": { "type": "roundRobin" }  // round–robin, in turn
"strategy": { "type": "leastPing" }   // minimum latency (requires observatory)
```

For `leastLoad`, parameters are specified in `settings`:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regexp": false, "match": "proxy-premium",   "value": 0.1 },
      { "regexp": true,  "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}
```

How it works (example). Suppose the observatory has measured exit latencies: `A = 250ms`, `B = 280ms`, `C = 700ms`, `D = 1500ms`. With the settings above, selection proceeds as follows:

1. **`maxRTT: "1s"`** — exits with latency above 1s are discarded: `D` (1500ms) is eliminated. Remaining: `A`, `B`, `C`.
2. **`baselines` + `expected`** — exits are grouped by latency thresholds, and the **lowest** threshold containing at least `expected` exits is chosen. The `500ms` threshold already contains `A` and `B` — that's 2 (= `expected`), so the group { `A`, `B` } is selected. `C` (700ms) is not included in the selection while fast exits are available (it is a "hot standby").
3. **`tolerance: 0.05`** — within the selected group, exits whose latencies differ by no more than 5% are considered equivalent, and load is shared equally among them. `A` (250) and `B` (280) differ by ~12% (> 5%), so all else being equal, the faster `A` is preferred; if the difference were within 5%, traffic would flow through both `A` and `B`.
4. **`costs`** — before comparison, they adjust the "cost" of individual exits: a lower `value` makes an exit more attractive, a higher one less so. In the example, `proxy-premium` gets `0.1` (becomes "cheaper" and is preferred), while all `proxy-cheap-*` (by regex, `regexp: true`) get `5` (become "more expensive" and are used last). This allows soft-prioritizing exits without hard-excluding them.

Result: traffic will flow mainly through `A` (or equally through both `A` and `B` if latencies are close), `C` remains on standby, `D` is excluded until its RTT drops below `maxRTT`.

Observer: `observatory` and `burstObservatory` (measurements for `leastPing` / `leastLoad`)

The `leastPing` and `leastLoad` strategies do not measure anything themselves — they need latency and availability data for each outbound. This is collected by the **observer** (observatory): it periodically "pings" each monitored outbound and records response time and availability. The same data is shown on the **"Observatory"** tab (statuses **Active / Unavailable**, **"Last Activity"**, **"Last Attempt"**).

As of version 3.4.2 the "Balancers" page has been split into the **"Balancer settings"** tab (the table and adding) and the **"Observatory"** tab, and the observer is edited with a **structured form** (the former raw JSON editor has been removed). The panel creates and removes observers as needed: a regular `observatory` for the **Least Ping** strategy, an extended `burstObservatory` for **Least Load**, as well as for **Random/Round-robin with a**

`fallbackTag` set (xray-core requires this; when the last `fallbackTag` is cleared, the observer is removed). Creating or removing an observer requires a full Xray restart.

Two variants are available:

- **`observatory`** — simple: `subjectSelector` + `probeURL` + `probeInterval`.
- **`burstObservatory`** — advanced, with fine-grained ping configuration via `pingConfig`; convenient for multiple exits.

Example `burstObservatory` block:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Field descriptions:

Field	What it sets
`subjectSelector`	List of **outbound tag prefixes** to monitor. Xray selects all outbounds whose tags start with the specified strings. In the example, exits `WS-SE…`, `WS-FR…`, `WS-PL…` are monitored. These tags must match what is selected in the balancer's **Selectors**.
`pingConfig.destination`	URL requested **through each outbound** to measure latency. Use a "lightweight" page that returns `204` with no body — e.g., `https://www.google.com/generate_204`. The time to response is the measured latency.
`pingConfig.interval`	How often to ping each outbound. Duration string: `"1m"` — once per minute, also `"30s"`, `"5m"`, etc. More frequent means fresher data but more background traffic.
`pingConfig.connectivity`	(optional) URL to check the **basic connectivity** of the server itself. If it is unreachable — the problem is in the server's network, and the observer does **not** mark an outbound as unavailable (protection against false positives from a local failure). Usually also an endpoint returning `204`.
`pingConfig.timeout`	How long to wait for a response to a single ping before counting the attempt as failed (e.g., `"5s"`).
`pingConfig.sampling`	How many recent measurements to store and average per outbound. `2` — account for the two most recent pings (smooths out random spikes).
`pingConfig.httpMethod`	**New in 3.4.2.** The probe HTTP method: `HEAD` (default) or `GET`.

On the **"Observatory"** tab the same parameters are set with a form (not JSON). For a regular `observatory` these are **Watched Outbounds** (`subjectSelector`, read-only), **Probe URL** (`probeURL`, default `https://www.google.com/generate_204`), **Probe Interval** (`probeInterval`, `1m`) and **Enable Concurrency** (`enableConcurrency`, on by default). For `burstObservatory` — the fields listed above plus the new **"HTTP**

method". If the balancer has both observers, a segmented toggle switches between the Observatory and Burst forms.

How to wire it all together:

1. On the **"Observatory"** tab, configure the observer parameters (it is created automatically for the chosen strategy; `subjectSelector` follows the balancer's selectors).
2. Create a balancer: **Strategy** = `leastPing`, and in **Selectors** specify the same outbound tags (`WS-SE`, `WS-FR`, `WS-PL`).
3. Route traffic to it with a routing rule (the **Balancer Tag** field, see 11.3).
4. Restart Xray. The **"Observatory"** tab will show exit statuses, and the balancer will start selecting the fastest live exit.

> A single rule cannot have both `balancerTag` and `outboundTag` set — only `outboundTag` will take effect.

What happens when deleting an outbound or balancer (as of 3.4.2)

When an outbound or balancer is deleted, in the same action the panel **cleans up the references to it in routing** and shows a **preview of the consequences** in the confirmation dialog (the header "Deleting it will also update your routing:", then for each rule — "removed (no destination left)" or "kept (now uses …)", and "Balancer … removed (no targets left)"). This prevents a "dangling" reference (`balancerTag` / `outboundTag` / `dialerProxy`) that previously could keep xray-core from starting on the next restart. Deleting a balancer: rules keep their `outboundTag`, otherwise they are removed. Deleting an outbound: its tag is removed from balancers' selectors and `fallbackTag`, the `dialerProxy` of other outbounds is cleared, emptied balancers are removed together with the now-purposeless rules, and observers are rebuilt.

11.6. DNS

The `dns` section. Enable with: **Enable DNS** (tooltip: *"Enable the built-in DNS server"*).

General DNS parameters

Field	Label	JSON	Description / tooltip
`tag`	**DNS Tag Name**	`dns.tag`	*"This tag will be available as an inbound tag in routing rules."* Allows routing DNS queries themselves via `inboundTag`.
`clientIp`	**Client IP**	`dns.clientIp`	*"Used to notify the server of the specified IP location during DNS queries"* (EDNS Client Subnet).
`strategy`	**Query Strategy**	`dns.queryStrategy`	*"Overall strategy for resolving domain names"*. Values: `UseIP`, `UseIPv4`, `UseIPv6`.

Field	Label	JSON	Description / tooltip
`disableCache`	**Disable Cache**	`dns.disableCache`	*"Disables DNS caching"*.
`disableFallback`	**Disable Fallback DNS**	`dns.disableFallback`	*"Disables fallback DNS queries"*.
`disableFallbackIfMatch`	**Disable Fallback DNS on Match**	`dns.disableFallbackIfMatch`	*"Disables fallback DNS queries when the DNS server's domain list matches"*.
`enableParallelQuery`	**Enable Parallel Queries**	—	*"Enable parallel DNS queries to multiple servers for faster resolution"*.
`useSystemHosts`	**Use System Hosts**	`dns.useSystemHosts`	*"Use the hosts file from the installed system"*.

Example `dns` block. Queries for Google domains are resolved via Cloudflare's DoH server, everything else via `1.1.1.1`; for Google queries, only non-private IPs are accepted. At the top level of the config:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

A string server (`"1.1.1.1"`) without fields is the default server for all other domains. The `dns-inbound` tag can then be used as an `inboundTag` in routing rules to route DNS queries themselves through the desired outbound.

Stale record cache

Field	Label	Description
`serveStale`	**Serve Stale**	*"Return stale results from the cache while refreshing in the background"*.
`serveExpiredTTL`	**Stale TTL**	*"Lifetime (seconds) of stale cache entries; 0 = unlimited"*.

DNS servers (`dns.servers` list)

Buttons: **Create DNS**, **Edit DNS**, **Delete All** (confirmation: *"All DNS servers will be removed from the list. This action cannot be undone."*). Templates: **Use Template**, the **DNS Templates** window, including the **Family** preset.

When clicking **Edit DNS** on a DNS server record (as well as on a Fake DNS record), the edit window populates the saved server values rather than default values.

DNS server fields:

Field	Label	Description
address	—	DNS address (IP, DoH URL, `localhost`, `fakedns`, etc.).
`domains`	**Domains**	List of domains for which this server is used.
`expectIPs`	**Expected IPs**	Accept the response only if the IP is in the list.
`unexpectIPs`	**Unexpected IPs**	Discard responses with the specified IPs.
`skipFallback`	**Skip Fallback**	Do not use this server as a fallback.
`finalQuery`	**Final Query**	Marks the server as final in the chain.
`timeoutMs`	**Timeout (ms)**	Query timeout for this server.

Hosts (static records)

The **Hosts** group (`dns.hosts`). Button **Add Host**; empty state **No Hosts defined**. Fields: domain (placeholder: *"Domain (e.g. domain:example.com)"*) and values (placeholder: *"IP or domain — enter and press Enter"*).

DNS Logs

See 11.10: the **DNS Logs** flag (`dnsLog`) in the logging section.

11.7. Fake DNS

The `fakedns` section. Buttons: **Create Fake DNS**, **Edit Fake DNS**.

Field	Label	Description
`ipPool`	**IP Pool Subnet**	CIDR range from which fake IPs are assigned (e.g., `198.18.0.0/15`).
`poolSize`	**Pool Size**	Number of addresses to keep in the ring pool.

Fake DNS is used in conjunction with sniffing on an inbound: the core gives the client a fake IP, records the domain⇄IP mapping, and restores the domain during routing. For Fake DNS to work, a DNS server with address `fakedns` must be added to the DNS servers list.

Example: Fake DNS + DNS server combination. First define the fake address pool, then add the `fakedns` DNS server so domain queries receive IPs from that pool:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Additionally, sniffing must be enabled on the inbound with `destOverride: ["fakedns"]`, otherwise the core has no way to restore the real domain.

11.8. WireGuard / WARP / NordVPN

WireGuard fields (`wireguard`)

Field	Label	Description
`secretKey`	**Secret Key**	Private key of the local interface.
`publicKey`	**Public Key**	Peer's public key.
`psk`	**Pre-Shared Key**	PreShared Key (optional).
`allowedIPs`	**Allowed IPs**	Ranges routed into the tunnel.
`endpoint`	**Endpoint**	Peer's `host:port`.
`domainStrategy`	**Domain Strategy**	Resolution strategy for the WireGuard outbound.

Cloudflare WARP (`warp`)

The integration uses the API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`). Controller actions (`/xray/warp/:action`): `config`, `reg`, `license`, `data`, `del`.

Step by step:

1. **Create WARP account** → `reg`: the panel generates/accepts private (`privateKey`) and public (`publicKey`) keys, registers a device with Cloudflare, and saves `access_token`, `device_id`, `license_key`, `private_key` (and `client_id`) in the `warp` setting.
2. **WARP / WARP+ license key** → `license`: sets a 26-character WARP+ key (placeholder: *"26-character WARP+ key"*). On error: *"Failed to set WARP license."* If config has not been obtained yet: *"Get WARP config first."*
3. **Account information**: **Device Name**, **Device Model**, **Device Enabled**, **Account Type**, **Role**, **WARP+ data**, **Quota**, **Usage**.
4. **Add outbound** — creates a WireGuard outbound with the obtained keys and Cloudflare endpoint.
5. **Delete account** → `del`: clears saved WARP data.

NordVPN (`nord` / `nordvpn`)

The integration uses NordLynx (= WireGuard). Controller actions (`/xray/nord/:action`): `countries` , `servers` , `reg` , `setKey` , `data` , `del` .

Step by step:

1. **Access token** → `reg` : the panel requests NordLynx credentials from `api.nordvpn.com` and extracts `nordlynx_private_key` . Saves `private_key` and `token` in the `nord` setting. Alternative — `setKey` : enter the **Private Key** directly (cannot be empty).
2. **Country** → `countries` loads the list of countries; **City** (or **All cities**).
3. **Server** → `servers` loads servers for the selected country (`countryId` is validated as a number — injection protection). Filter: only servers with **Load** > 7% are shown. If no servers found: *"No servers found for the selected country"*. If a server has no NordLynx public key: *"The selected server does not report a NordLynx public key."*
4. Creating/updating the outbound: toasts *"NordVPN outbound added"* / *"NordVPN outbound updated"*.

IPv4 priority and userspace TUN

WireGuard outbounds generated by the WARP and NordVPN wizards use `domainStrategy: "ForceIPv4v6"` (IPv4 priority with fallback to IPv6 on v6-only hosts) instead of `ForceIP` — this eliminates handshake hangs on hosts with a partially configured IPv6 stack when a Cloudflare AAAA endpoint is selected. Additionally, userspace TUN (`noKernelTun: true`) is enabled for them instead of kernel TUN: the latter requires privileges and fwmark routing and silently fails on many VPS instances, while the panel's built-in connectivity check always tests via userspace TUN — now real traffic and the check follow the same path. This change applies only to newly added or reset outbounds; already saved templates keep their settings.

11.9. Reverse proxy and TUN

Reverse (reverse proxy)

The `reverse` section of the Xray configuration. The outbound form has a toggle for the **Reverse Proxy** type. Buttons: **Create Reverse Proxy**, **Edit Reverse Proxy**.

Field	Label	Description
Type	**Type**	**Bridge** or **Portal** — the two roles of the Xray reverse proxy.
Domain	**Domain**	Service label domain for the bridge↔portal pair.
Tag / Connection	**Tag** / **Connection**	Tags for linking bridge and portal.
Reverse Tag	**Reverse Proxy Tag**	Tooltip: *"Outbound connection tag for a simple VLESS reverse proxy. Leave empty to disable."* Placeholder: *"outbound tag (empty = disabled)"*. Implements a simplified VLESS reverse.

The outbound form also contains reverse-flow fields: **Reverse Sniffing**, **Workers**, **Reserved**, **Min Upload Interval (ms)**, **Max Upload Size (bytes)**.

TUN (`tun`)

Field	Label	Description	Default
name	—	*"TUN interface name."*	`xray0`
mtu	—	*"Maximum Transmission Unit. Maximum size of data packets."*	**1500**
`userLevel`	**User Level**	*"All connections established through this inbound will use this user level."*	**0**

11.10. Logs and statistics (Stats, metrics)

Log (`log`)

Tooltip: *"Logs can slow down the server. Enable only the log types you need when necessary!"* The `log` section of the reference template: `access: "none"`, `error: ""`, `loglevel: "warning"`, `dnsLog: false`, `maskAddress: ""`.

Field	Label	JSON	Description	Default
`logLevel`	**Log Level**	`loglevel`	*"Log level for error logs..."* Values: `debug`, `info`, `warning`, `error`, `none`.	`warning`
`accessLog`	**Access Logs**	`access`	*"Path to the access log file. The special value 'none' disables access logs."*	`none`
`errorLog`	**Error Logs**	`error`	*"Path to the error log file. The special value 'none' disables error logs."*	`""` (default)
`dnsLog`	**DNS Logs**	`dnsLog`	*"Enable DNS query logs"*	**false**
`maskAddress`	**Address Masking**	`maskAddress`	*"When enabled, the real IP address is replaced with a masking address in the logs."*	`""` (off)

Statistics (`stats` / `policy`)

The **Statistics** group. Enables counters in `policy.system` and `policy.levels`. In the reference template: `statsInboundUplink: true`, `statsInboundDownlink: true`, `statsOutboundUplink: false`, `statsOutboundDownlink: false`; for level `0` — `statsUserUplink: true`, `statsUserDownlink: true`.

Field	Label	Description	Default
`statsInboundUplink`	**Inbound Uplink Statistics**	*"Enables statistics collection for the outgoing traffic of all inbound proxies."*	**true**
`statsInboundDownlink`	**Inbound Downlink Statistics**	*"Enables statistics collection for the incoming traffic of all inbound proxies."*	**true**
`statsOutboundUplink`	**Outbound Uplink Statistics**	*"Enables statistics collection for the outgoing traffic of all outbound proxies."*	**false**
`statsOutboundDownlink`	**Outbound Downlink Statistics**	*"Enables statistics collection for the incoming traffic of all outbound proxies."*	**false**

> Per-client and per-inbound statistics (uplink/downlink) form the basis of traffic display in the dashboard and for clients; disabling them is not recommended. Outbound statistics are disabled by default and are only needed if you track traffic by outbound tag.

Metrics

The reference template includes a `metrics` section (`listen: "127.0.0.1:11111"` , `tag: "metrics_out"`) and a corresponding `metrics_out` API. The panel uses this listener to collect metrics and observatory snapshots: it parses `metrics.listen` from the template, polls `/debug/vars` , and aggregates latency history by tag. If you change the `metrics.listen` address/port, the panel will connect to the new address; removing the `metrics` section disables observatory graph collection.

> Outbound testing in HTTP mode spins up a **separate temporary** Xray instance with its own `metrics` listener on a random port — this is not the same listener as in the main config.

11.11. Saving, restart, and automatic transformations

Buttons

Button	Action
Save	`POST /xray/update` : validates and saves the template + `outboundTestUrl` .
Restart Xray	Reloads the service with the saved configuration. Confirmation: *"Restart xray?"* / *"Reloads the xray service with the saved configuration."*

Toasts: success — *"Xray restarted successfully"*, *"Xray stopped successfully"*; errors — *"An error occurred while restarting Xray."*, *"An error occurred while stopping Xray."* The **Xray Restart Output** window shows diagnostic output from the core.

Hot-apply changes (without full restart)

Changes to inbounds, outbounds, and routing rules are applied "live": when **Save** is clicked, the panel computes the diff between the old and new config and applies only the changed parts through the Xray gRPC API (HandlerService/RoutingService), without restarting the process. A full restart is performed automatically only when sections without a hot-reload API change (`log` , `dns` , `policy` , `observatory` , etc.). Therefore, the Xray page does not need a separate "Restart" button — **Save** itself applies the changes. A core restart is still performed automatically when needed (see also auto-reload on subscription updates and WARP rotation).

Restoring the default template

The endpoint `GET /xray/getDefaultJsonConfig` returns the reference template (`config.json` , embedded in the binary). It can be used to reset the configuration to factory defaults.

Automatic transformations on save

When saving Xray settings, the panel performs (in this order):

1. **Unwrapping** — removes wrappers of the form `{ "xraySetting": <config>, "inboundTags": …, "outboundTestUrl": … }` if they accidentally ended up in the value (otherwise layers would accumulate with each save). Up to 8 layers are unwrapped.
2. **Configuration validation** — the JSON is parsed into an Xray config structure; on error — rejected with *"xray template config invalid"*.
3. **Guaranteeing the statistics rule** — the rule `inboundTag: ["api"] → outboundTag: "api"` is forcibly moved to position 0 in `routing.rules` (or added if absent). This ensures that the panel's gRPC statistics request will not be intercepted by an upstream catch-all rule (otherwise clients may appear offline with zero traffic while the proxy is running).

> Due to point 3, do not try to remove or move the `api → api` rule — the panel will put it back in place on the next save. This is service infrastructure for statistics, not a user route.

11.12. Subscription outbounds (with auto-update)

Starting from version 3.3.0, the panel can import `outbound`s directly from a subscription URL — the same format that VPN providers serve to client applications. Subscriptions are re-fetched in the background on a schedule, so the set of `outbound`s on the server stays up to date without manual template editing.

The section is titled **"Outbound Subscriptions"**, description: "Import outbounds from remote subscription URLs (vmess/vless/trojan/ss/...). Tags remain unchanged for use in balancers and routing rules. Updates are performed automatically." The section is located on the Xray page, above the outbound settings panel.

How it works

Subscriptions are stored separately from the Xray configuration template. The template is **never overwritten**: `outbound`s obtained from subscriptions are merged into the final configuration on the fly each time the Xray config is generated.

Adding a subscription

The "Add Subscription" form has the following fields:

Field	Key	Default	Purpose
Subscription URL	`url`	— (required)	Subscription address. Placeholder: "https://... (base64 link list)". Only HTTP(S) is accepted; the address is validated for safety.
Remark	`remark`	empty	Arbitrary label (placeholder "e.g. HK nodes").
Tag Prefix	`tagPrefix`	`subN-`	Prefix that imported `outbound` tags start with. If left empty, the panel automatically picks the lowest available number in the form `sub1-`, `sub2-`, etc.

Field	Key	Default	Purpose
Update Interval	`updateInterval`	600 seconds (10 minutes)	How often the subscription is re-fetched. Set in hours/minutes in the UI.
Enabled	`enabled`	yes (`true`)	Only enabled subscriptions are included in the config and updated automatically.
Allow Private Addresses	`allowPrivate`	no (`false`)	Allows URLs on localhost, LAN, and private IPs. Disabled by default as SSRF protection — enable only for a trusted local source.
Before Manual Outbounds	`prepend`	no (`false`)	If enabled, `outbound` s from this subscription are placed **before** manual `outbound` s from the template, and one of them can become the default `outbound` . Otherwise they are appended **after**.

The **"Preview"** button (`POST /outbound-subs/parse`) lets you download and parse the URL before saving to see which `outbound` s and tags will result; nothing is written to the database at this point. If nothing is recognized at the URL, "No outbounds found at this URL." is displayed.

The order of multiple subscriptions in the overall `outbound` s list is set by priority (`priority`) and changed with up/down arrows (`POST /outbound-subs/:id/move`).

Accepted subscription formats

The response body from the URL is processed as follows:

- The content is first tried as **base64** (standard and URL-safe variants, with automatic padding and removal of spaces/newlines). If it is base64 — it is decoded; otherwise taken as-is.
- The body is then split into lines. Each non-empty line that does not start with `#` is parsed as a link. Unrecognized lines (comments, unsupported protocols) are silently skipped.
- Supported link schemes: `vmess://` , `vless://` , `trojan://` , `ss://` (Shadowsocks), `hysteria2://` / `hy2://` , `wireguard://` / `wg://` .

In other words, a standard subscription of the form "base64-encoded list of links", as used by most providers, is accepted.

Stable tags

Each link is assigned a stable "identity" (URI core without the fragment/remark; for vmess — internal JSON without the `ps` field). The "identity → tag" mapping is persisted, so on the next update the same server gets the same tag, even if the remark or secondary parameters changed. This is specifically designed so that balancers and routing rules continue to work after updates:

- An exact tag in a balancer/rule will continue to point to the same server.
- A prefix/wildcard selector (e.g., `hk-*`) will automatically pick up new servers that the subscription returns later — this is the recommended way to "subscribe to a pool".
- If a server disappears from the subscription, its tag simply drops out of the final `outbound` s array; if the balancer has a `fallbackTag` , Xray uses it.

- If the provider changed a server's UUID/host/credentials, the identity changes — this is treated as a new `outbound` with a new tag.

Within a single fetch, tags are deduplicated with a `-N` suffix. Subscription tags preserve non-ASCII characters (e.g., Cyrillic) and remain readable: Unicode letters and digits are kept in the slug, while punctuation is replaced with a hyphen — tags from Cyrillic names no longer collapse to just digits.

How auto-update works

- The subscription update background task runs on a schedule **every 5 minutes**.
- On each run it iterates through all enabled subscriptions and updates only those whose own interval has expired: a subscription is updated if it has never been updated yet, or if at least its `updateInterval` has passed since the last update. This way the task checks subscriptions frequently, but each individual subscription is re-fetched no more often than its `updateInterval` (default 10 minutes). The UI reflects this with a corresponding tooltip.
- Update: the URL is re-validated for safety as a public URL (private addresses are blocked unless the subscription has `allowPrivate` set), the request goes through the panel's proxy client with the header `User-Agent: 3x-ui-outbound-sub/1.0`. The redirect chain is limited to 10 hops, and each hop is also checked for privacy (SSRF protection). HTTP 200 is expected; otherwise an error is recorded.
- After successful parsing, the result is saved, the last-update time is set, and the error is cleared. On error, its text is visible in the UI as "Last Error", and the previously fetched `outbound`s remain in effect.
- If at least one subscription actually updated, the task marks Xray for restart and sends a UI invalidation so the interface pulls the new `outbound`s. The actual Xray reload happens on the nearest 30-second manager cycle.

Manual update of a single subscription — the **"Refresh Now"** button (`POST /outbound-subs/:id/refresh`); it also marks Xray for restart. Adding, modifying, or deleting a subscription also sets the Xray restart flag (on deletion, its `outbound`s drop out of the config on the next reload). The UI hints: "After adding or updating, restart Xray (or wait for the next auto-reload) for the outbounds to become active."

How it gets into the Xray config

On each Xray configuration generation, active subscription `outbound`s are divided into two groups — `prepend` (the "Before Manual Outbounds" flag) and the rest — and merged with the template: `[subscription prepend] + [template outbounds] + [remaining subscriptions]`. Within each group, subscriptions are ordered by priority. Manual `outbound`s from the template are not affected; if the template's `outbound`s array fails to parse for some reason, subscription `outbound`s are not merged in (to avoid losing manual ones).

Imported `outbound`s are additionally shown in the outbound panel itself in a separate **"From Outbound Subscriptions (read-only)"** block — they cannot be edited there; management is only through the "Outbound Subscriptions" section.

11.13. IP rotation in WARP

In 3X-UI you can set up a WARP outbound — an outgoing WireGuard connection to Cloudflare WARP (tag `warp` in the Xray config). The panel registers a device account on Cloudflare servers, obtains WireGuard keys and addresses, and injects them into the outbound with tag `warp`. Through this outbound, traffic exits the internet

under a Cloudflare WARP IP address. New in version 3.3.0 — the ability to change this outgoing IP manually or on a schedule, without manually recreating the WARP account.

Management is located in the **Xray** section in the WARP card (after clicking "Create WARP Account" and obtaining the config; until then the actions are unavailable — the panel will prompt "Get WARP config first").

What happens when the IP is changed

The **"Change IP"** button initiates an IP change. The logic:

1. A new WireGuard key pair is generated.
2. A device is re-registered with WARP on Cloudflare servers using the new key (new `device_id`, `access_token`, addresses, and peer data).
3. The new data is written to the WARP outbound in the Xray config: `secretKey`, `address` (v4 `/32` and v6 `/128`), `reserved` (from `client_id`), and also `publicKey` and `endpoint` of the peer are updated.
4. If a WARP+ license key was previously set (at least 26 characters long), it is automatically re-applied to the new account. On failure, this is only a warning in the logs — the IP change is not rolled back.
5. After a successful change, Xray is marked as requiring a restart so that the new outbound takes effect.

On success, the interface shows "WARP IP address changed successfully!".

Automatic rotation on a schedule

The WARP card has an **"Automatic IP address update"** toggle and an **"Interval (days)"** field. Tooltip: "0 — disable. Automatically changes the IP address."

Parameter	Value
DB setting	`warpUpdateInterval` (integer, ⩾ 0)
Default value	`0` (auto-rotation disabled)
Unit	days
`0`	disables automatic rotation
`> 0`	change IP every N days

Saving the interval stores `warpUpdateInterval`, and if the value is greater than 0, resets the "last update time" to the current moment — otherwise the scheduler would change the IP on the very next tick.

The schedule is executed by a background task that runs once per hour — i.e., the panel checks once per hour whether it is time to rotate. Check algorithm:

- if the interval is ⩽ 0 — does nothing;
- if the "last update time" is 0 (e.g., the interval was set by directly editing the DB) — this is the first run: the task only records the baseline timestamp and does NOT change the IP immediately;
- if at least `interval × 24 × 3600` seconds have passed since the last update — the same IP change is performed, the timestamp is updated, and an Xray restart is scheduled.

Important detail: a manual IP change via the "Change IP" button also resets the last update timestamp. Therefore, after a manual rotation the automatic interval counter starts over and a scheduled change will not fire immediately afterward.

"Via panel proxy"

> **Changed in 3.3.1.** The separate "Panel Network Proxy" setting (`panelProxy`) has been removed. The panel's own outgoing traffic (including requests to the WARP API) is now routed through the selected **outbound for panel traffic** — an Xray outbound or balancer (see section 13). The description below applies to versions before 3.3.1.

All requests to the Cloudflare WARP API (registration, config retrieval, license setting, IP change) go not directly but through the panel's HTTP client with a 15-second timeout. This client respects the **"Panel Network Proxy"** (`panelProxy`) setting from the panel settings.

From the setting description: the proxy routes the panel's own outgoing requests (geo-database updates, Xray/panel version checks, Telegram, and now WARP requests as well) — to bypass server-side filtering. Addresses of the form `socks5://` or `http(s)://` are accepted, for example a local SOCKS inbound of Xray itself. If the field is empty or the proxy is incorrectly configured — a direct connection is used (behavior does not break).

Use case for WARP: if the server cannot directly reach `api.cloudflareclient.com`, registration and rotation would previously fail. Now, by specifying a working proxy in `panelProxy` (including Xray's own inbound), you can guarantee WARP API availability and the functioning of both the manual button and scheduled rotation.

When is this useful

- Regular rotation of the outgoing IP for an outbound that goes through WARP — reduces the risk of blocks and tracking via a single address.
- Manually "refresh" the IP if the current Cloudflare address has been blacklisted or is performing poorly.
- Servers that have no direct access to the Cloudflare WARP API: routing requests through `panelProxy` makes registration and rotation functional.

---

12. Nodes (multi-panel, master/slave)

The **Nodes** section turns a standard 3X-UI installation into a **central (master) panel** that remotely monitors and manages other (child) 3X-UI panels. Each node is a separate 3X-UI installation on its own server; the master communicates with it via its own HTTP API, polls its status, and synchronizes the inbounds and clients assigned to it. This is the **multi-panel** capability: instead of logging into each panel separately, you see all servers in a single list and manage them centrally.

An important principle: **a node is not an agent, but a fully functional 3X-UI panel.** The master does not "install" anything on it — it simply connects to its API using a token. Removing a node from the list only stops monitoring; the remote panel itself is not affected (tooltip: "This will stop monitoring the node. The remote panel will not be affected").

12.1. Summary at the top of the list

Aggregate counters are displayed above the nodes table:

Field	Description
Total nodes	Total number of nodes in the list.
Online	How many nodes have `online` status.
Offline	How many nodes have `offline` status.
Average latency	Average latency (ping) to nodes, in milliseconds.

12.2. Adding and editing a node

The **Add node** and **Edit node** buttons open a form with node fields.

The required fields (tooltip: "Name, address, port, and API token are required") are **Name**, **Address**, **Port**, and **API Token**.

When clicking "Save" (both when adding and editing), the panel **first checks node reachability** with a 6-second timeout. If the node does not respond, the record is not saved and an error is shown. This means you cannot add a node that is knowingly unreachable.

Form fields

Field	Default	Allowed values	Description
Name	— (required)	non-empty string, **unique**	Internal node name. A uniqueness constraint is applied to the name column — two nodes with the same name cannot be created. Placeholder hint: `e.g. de-frankfurt-1`. Spaces at the edges are trimmed on save.
Note	empty	any string	Optional comment/description for the node. Does not affect functionality.

Field	Default	Allowed values	Description
Scheme	`https`	`http` / `https`	Connection protocol to the remote panel. If left empty or set to an invalid value, normalization will set `https`. If the node responds over plain HTTP but the scheme is set to `https`, the panel will return a helpful hint: "the server speaks HTTP, not HTTPS; set the node scheme to http".
Address	— (required)	host or IP	Address of the remote panel. Placeholder: `panel.example.com or 1.2.3.4`. The address is normalized; by default, private/local addresses are blocked to protect against SSRF — see "Allow private address".
Port	— (required)	integer **1–65535**	Web panel port of the remote node. Values outside the range are rejected ("node port must be 1-65535").
Base path	`/`	path string	Base path (web base path) of the remote panel, if configured. Normalized: guaranteed to start and end with `/` (empty value → `/`). The panel appends `panel/api/server/status` to it when polling.
API Token	— (required)	remote panel token	Bearer token for accessing the node's API. Passed in the `Authorization: Bearer <token>` header. Placeholder: "Token from the Settings page of the remote panel". Tooltip: "The remote panel shows its API token in Settings → API Token". This means the token must be created **on the node itself** (Settings → API Token), then pasted here.
Enabled	`true`	yes/no	Enables node monitoring and synchronization. Disabled nodes **are not polled** by background tasks (heartbeat and traffic-sync skip them) and do not participate in bulk panel updates.
Allow private address	`false`	yes/no	Removes the SSRF protection and allows connecting to a node via a private/local address. Tooltip: "Enable only for nodes on a private network or VPN". Enable only when the node is truly on a private network or accessible via VPN.

Obtaining and regenerating the token on the node side

The token is obtained on the remote panel in the **Settings** → **API Token** section. It can also be reissued there: the **Regenerate token** button with a warning: "Regenerating will invalidate the current token. Any master panel using it will lose access until updated. Continue?". After regeneration, the old token in the master panel will stop working — it must be updated in the node form.

Connection outbound

The **Connection outbound** field (`outboundTag`) defines how the master's API traffic to this node leaves the server. If you select an Xray outbound tag, the panel's requests to the node will go not directly, but through the selected outbound; the panel will automatically add a loopback bridge inbound to the running configuration and apply the change live, without a restart. Tooltip: "Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection".

The selector works like a panel outbound selector: tags are grouped into **Outbounds** (regular outbounds) and **Balancers** (load balancers); blackhole outbounds are hidden from the list. An empty value (placeholder "Direct connection") = direct connection to the node.

Import inbound (selecting inbounds to synchronize)

The node form has an **Import inbound** setting (`inboundSyncMode`) with two modes: **All inbounds** (`all` , default) and **Selected** (`selected`). By default, the master synchronizes all inbounds that have this node selected; existing nodes continue working in "All inbounds" mode.

In **Selected** mode, a multi-select of inbound tags appears below the field. Click **Load inbounds** — the master will use the entered (not yet saved) connection parameters to request the node's list of inbounds (endpoint `POST /panel/api/nodes/inbounds`) and display their tags; check the ones you need. The panel will synchronize and deploy only the checked tags to the node, while other inbounds existing directly on the node will remain untouched — the master does not delete or manage them.

Example: request the node's inbound list for selective import. The body contains the not-yet-saved connection parameters; the response contains the tags of inbounds available on the node:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. TLS verification (for https nodes)

A group of fields defines how the master verifies the HTTPS certificate of a node. These settings **are only relevant for the `https` scheme**; they are ignored for `http` nodes.

TLS verification — a drop-down list, tooltip: "How the panel verifies the node's HTTPS certificate. Pinning or Skipping — for self-signed certificates (https nodes only)".

Mode	Value	Default	Description
Verify (standard CA)	`verify`	yes (default)	Standard certificate chain verification by a trusted CA. Suitable for nodes with a public/Let's Encrypt certificate. Also used for all `http` nodes.
Pin certificate (SHA-256)	`pin`	—	The CA chain is not verified, but the SHA-256 fingerprint of the node's leaf certificate is compared against the stored fingerprint (constant-time comparison). Preserves MITM protection for **self-signed** certificates. Requires filling in the fingerprint field.
Skip verification	`skip`	—	Certificate verification is completely disabled. Warning: "Skipping verification removes protection against man-in-the-middle attacks — the API token may be intercepted. Pinning the certificate is preferable."

In addition to the three modes above, a fourth was added in 3.4.0 — **Mutual TLS (client certificate)** (`mtls`), available, like the others, only for the `https` scheme.

Mode	Value	Default	Description
Mutual TLS (client certificate)	`mtls`	—	In addition to verifying the node's certificate, the master additionally authenticates itself to the node with a **client certificate** issued by its own CA. For a node in this mode, the **API token becomes optional** — the node recognizes the master by the certificate. When this mode is selected, a tooltip is shown: "This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it".

To enable mutual TLS for a node: on the node side, set the **Mutual TLS** mode, copy the managing panel's CA from the **Node mTLS** section (see below), set it as the **Trusted parent CA** on the node, and restart the node.

If any value other than `skip`, `pin`, or `mtls` is selected, normalization will forcibly set `verify`.

Certificate pinning

When **Pin certificate** is selected, the following appear:

- **SHA-256 of the pinned certificate** — an input field. Accepts a fingerprint in **base64** format (`pinnedPeerCertSha256` from Xray) or in **hex** with or without colons (`openssl -fingerprint` style). Tooltip: "SHA-256 of the node's certificate in base64 or hex. Click 'Fetch' to read it from the node now". Placeholder: "SHA-256 in base64 or hex". When `pin` is selected, an empty or incorrect fingerprint causes a validation error on save.

Example: the same fingerprint in two formats. The field accepts either — both refer to the same certificate:

```
# base64 (pinnedPeerCertSha256 format from Xray)
6O7TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex with colons (openssl x509 -fingerprint -sha256 style)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

If the fingerprint is not yet known, click **Fetch** — the master will read it from the node over HTTPS and fill in the field.

- The **Fetch** button — connects to the node over HTTPS without certificate verification and reads the SHA-256 fingerprint of the current leaf certificate (endpoint `POST /certFingerprint`), inserting it into the field. On success — "Current node certificate retrieved"; on failure — "Failed to retrieve certificate". Available only for https nodes.

Node mTLS (mutual TLS authentication between panels)

On the **Nodes** page there is a separate **Node mTLS** section — a mutual TLS authentication setting that adds a second factor (client certificate) on top of the API token for "panel → node" calls. Mutual TLS is opt-in; if the section fields are empty, nodes work as before — **with API token only** (tooltip: "Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth"). The section has two operations:

- **Copy this panel's CA** (`POST /panel/api/nodes/mtls/ca`) — copies this panel's root certificate (CA) to the clipboard. This CA must be passed to the managed nodes so that they trust the panel's client certificate; on

the nodes themselves, the TLS verification mode is then set to **Mutual TLS** (tooltip: "Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS"). After copying — "CA certificate copied to clipboard".

- **Trusted parent CA** (`POST /panel/api/nodes/mtls/trustCA`) — a field used when this panel itself acts as a node for an upstream (managing) panel. Paste the managing panel's CA here to require its client certificate, and click **Save trust CA**. The change requires a **panel restart** (tooltip: "When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply").

12.4. What is shown for each node

Table columns and node card fields (observed state, populated on each heartbeat poll):

Field	Description
Status	`online` / `offline` / `unknown` — see below.
CPU	CPU load of the remote server in percent.
Memory	RAM usage in percent (calculated as `current/total*100`).
Uptime	Continuous server uptime (in seconds).
Latency	Response time of the node on the last poll (ms).
Last ping	Time of the last successful heartbeat (unix seconds; `0` = "never"; a recent value is shown as "just now").
Xray version	Version of Xray-core running on the node.
Panel version	Version of 3X-UI on the node — compared with the latest for the update indicator.
(inbounds)	How many inbounds are physically hosted on this node.
(clients)	Number of clients on the node's inbounds.
(online)	How many of the node's clients are currently online.
(exhausted)	How many of the node's clients have **expired or exceeded their traffic limit**. Manually disabled clients are not included in this counter.
(speed)	Current (live) transfer speed on inbounds hosted on the node.

The inbounds/clients/online counters are linked to the node by its stable GUID (`panelGuid`), not by the local id — so that a client on a sub-node is counted under the sub-node, not under an intermediate node through which it is synchronized.

For inbounds hosted on the node, the page shows online clients, counters, and **current transfer speed**. Binding by stable GUID correctly separates even "cloned" nodes with the same `panelGuid` .

Node statuses

Status	When set
`online`	Node responded with `success=true` to a `panel/api/server/status` poll.

Status	When set
`offline`	Node did not respond, returned an HTTP error, `success=false`, or an unrecognizable response.
`unknown`	Initial value, while the node has not yet been polled.

When polling fails, the error text is saved and shown as a readable message, which helps diagnose the cause of the "offline" state.

12.5. Node actions

- **Test connection** (`POST /test`) — in the node form, tests the connection using the entered (not yet saved) parameters with a 6-second timeout. Result: "Connection OK ({ms} ms)" or "Failed to connect". Useful for debugging address/port/token/TLS before saving.
- **Check now** (the "Check now" button, `POST /probe/:id`) — an unscheduled poll of an already saved node; immediately updates status and metrics (CPU/memory/uptime/latency/versions) and records a heartbeat. On failure — "Check failed".

Example: test and poll a node via the master API. "Test connection" tests the not-yet-saved parameters from the form:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGci...", "tlsMode": "verify" }
```

Unscheduled poll of an already saved node with id 7:

```
POST /panel/api/nodes/probe/7
```

- **Update panel** (`POST /updatePanel` with body `{ids:[…]}`) — launches the node's built-in self-updater: the node downloads the latest 3X-UI release and restarts on it. The **Update selected ({count})** button does this for several checked nodes at once. Next to a node, an indicator is shown: **Update available** or **Up to date**, based on comparing the node's panel version with the latest.

Example: update multiple nodes with one request. The body contains the ids of the checked nodes; only enabled and `online` nodes will be updated, the rest will be returned as skipped.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Response of the form "Update launched on 2 nodes, 1 failed": node 12, for example, may have been offline and therefore skipped.

- Confirmation: "Update {count} nodes to the latest version? Each selected node will download the latest release and restart. Only enabled online nodes are updated."

- **Only enabled nodes with `online` status are updated.** A disabled node is marked "node is disabled" in the results, an offline node — "node is offline". Result: "Update launched on {ok} nodes, {failed} failed". If no suitable nodes are selected — "Select at least one enabled online node".

In the update confirmation dialog (both for a single node and for bulk updates), there is a checkbox **Update to the development channel (latest commit)**. If checked, the selected nodes will install the dev-latest rolling build (the latest commit from the main branch) instead of a stable release; if unchecked, the node updates via its normal channel. When the checkbox is enabled, a warning is shown below it: "Development builds track every commit in main and are not stable releases — there is no automatic rollback". The dev flag is passed through `POST /panel/api/nodes/updatePanel` to the node, and it launches the update via the dev channel.

- **Set Cert from Panel** (auxiliary, `GET /webCert/:id`) — when creating an inbound on a node, allows substituting paths to the node's **own** web TLS certificate (not the central panel's), so that the files exist on the node itself. Requires that the node is enabled and reachable.
- **Delete node** (`POST /del/:id`) — confirmation: "Delete node "{name}"? This will stop monitoring the node. The remote panel will not be affected". Deletes the node record and its accumulated traffic statistics; the remote panel continues working normally. **A node can only be deleted after all inbounds have been removed from it.** If at least one inbound is still linked to the node (via `node_id`), the panel will reject the deletion with an error like "cannot delete node: N inbound(s) still attached to it; detach or delete them first" — first detach or delete those inbounds, then delete the node. This prevents "orphaned" inbounds with a dangling reference to a deleted node.

12.6. Metrics history

The history button/chart queries `GET /history/:id/:metric/:bucket`. Available metrics: **`cpu`** and **`mem`** — they accumulate on each successful heartbeat. The aggregation interval size (`bucket`, in seconds) is restricted to an allowlist:

Example: history request. CPU load history for node 7 with 60-second aggregation intervals (up to 60 data points are returned):

```
GET /panel/api/nodes/history/7/cpu/60
```

For memory and "real-time" mode (2 s) — `…/7/mem/60` and `…/7/cpu/2` respectively. Values outside the allowlist are rejected ("invalid metric" / "invalid bucket").

Bucket (s)	Purpose
2	Real-time mode
30	30-second intervals
60	1-minute intervals
120	2-minute intervals
180	3-minute intervals
300	5-minute intervals

Up to 60 data points are returned. An invalid metric or bucket is rejected ("invalid metric" / "invalid bucket").

12.7. How inbounds and clients are synchronized

An inbound "belongs" to a node through the `node_id` field (the node is selected in the inbound editor):

Example: token in the node form. The token is obtained on the child panel (Settings → API Token) and pasted into the master's **API Token** field. On each poll, the master sends it in the header:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

If the child panel has a **base path** (web base path) set, e.g. `/secret/`, the master automatically prepends it before `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Configuration deployment (reconcile).** Whenever an inbound/client linked to a node is modified, the node is marked "dirty". A background task, for each enabled node **with `online` status**, deploys the node's inbounds (by `node_id`) when there are pending changes, then clears the "dirty" flag. A node that is disabled, offline, or "dirty" is considered "pending" — deployment to it is deferred until connectivity is restored.

2. **Traffic collection.** The same task requests a traffic snapshot from the node and merges it into local statistics. Based on the merged traffic, limit/expiry checks are performed and clients are disabled if necessary; the "exhausted" counter for the node reflects exactly this. If the node is unreachable, its online clients are cleared.

   For a client linked to multiple panels, the master in the same task additionally distributes to the nodes the **total across all panels** traffic consumed by that client (in a separate table on the node, keyed by the master's GUID; overwritten on each send, so a reset on the master side also propagates). On the node, the larger of the two values — local or received — is displayed in the client's traffic, and when the total quota is exceeded, the client is disconnected **locally on the node itself** (via the same Xray restart mechanism used during auto-disconnect, which terminates already established connections). This eliminates the situation where a node only saw its share of traffic, underestimated usage, and continued serving a client who had already exhausted the total limit. When traffic is reset, auto-renewed, or the client is deleted, the sent counters are cleared.

   On the **first** synchronization of an inbound hosted on a node (adding a new node or re-importing an inbound), the master initializes the client traffic counters with the real values from the node. Previously in this situation, the overall inbound counter was transferred correctly, but individual client counters were zeroed, and the master underestimated client usage by the entire history accumulated before the node was connected. Now, if the inbound is created in the same synchronization pass, the new `client_traffics` row inherits the counter value from the node (the baseline is set equal to it, so the next delta is zero and traffic is not counted twice). The counter seeding applies only to an inbound created in the same pass: a client appearing again under an already existing inbound still starts from zero (protection against "phantom" traffic), and a recently deleted client does not "resurrect" when its inbound is recreated.

3. **Heartbeat.** A separate background task periodically polls all **enabled** nodes (with a concurrency limit) via `panel/api/server/status`, updates status/metrics/versions, and, when web clients are connected, broadcasts the updated node tree over WebSocket.

12.8. Node chains (sub-nodes / transitive nodes)

The topology can be non-flat: a node can itself be a master for its own nodes. Such downstream panels appear on your side as **Sub-nodes** — these are **read-only projections** received from the direct node.

- Tooltip: "Read-only: a subordinate node accessible via {parent}. Manage it from {parent}'s own panel". That is, a sub-node cannot be edited, deleted, or updated here — all operations on it are performed from its direct parent's panel.
- The identity of a sub-node is determined by its GUID; this ensures that online clients and inbounds are counted under the physical node that hosts them, even in a chain `Node1 → Node2 → Node3` (the master "descends" one level through each direct node).
- If a direct node becomes unreachable, its sub-node cache is cleared, and sub-nodes disappear from the tree until connectivity is restored.

12.9. Nodes: new in 3.3.0

In version 3.3.0, the **Nodes** section received three notable improvements: correct traffic attribution and online client tracking in multi-hop topologies, client-IP synchronization across nodes, and a separate status indicator for the case when a node's panel is alive but the Xray core on it has crashed.

1. Multi-hop: correct traffic attribution along the sub-node chain

Previously, counters (number of inbounds, online clients, exhausted) were calculated at the level of the "direct" node. If you had a chain like `Master → Node1 → Node2 → Node3`, everything physically living on `Node2` / `Node3` was incorrectly attributed to `Node1`, through which it reached the master. In 3.3.0, attribution goes by the real source.

How it works:

- **Sub-nodes become visible as separate rows.** Each panel publishes a list of its direct nodes; only nodes with a known `Guid` are included — stable identity is needed to attribute a node one "hop" up. The master periodically (from the heartbeat job) fetches these lists and caches them, then adds "transitive" sub-nodes to the direct nodes.
- **Transitive nodes are read-only.** In the UI they are marked as **"Sub-node"** with the tooltip: *"Read-only: a subordinate node accessible via {parent}. Manage it from {parent}'s own panel."* There are no management buttons for such a row — the node is managed from its direct parent's panel.
- **Hierarchy via GUID.** A direct node's `ParentGuid` is the master's own GUID; a transitive node's is the GUID of its parent node. This is how the tree is built.
- **The source of truth for counters is `origin_node_guid` on the inbound.** This is the `panelGuid` of the node that physically holds that inbound. It is set during inbound synchronization from the node and **preserved as-is through further hops**, so a deeply nested inbound is attributed to the real node, not an intermediate one. The inbound count, online client count, and exhausted client count are recalculated using this GUID. Key selection logic:

Inbound state	Attributed to
`origin_node_guid` is set	that GUID (real source node)
empty, but `node_id` is set	synthetic GUID of the node (old build, has not yet reported its `panelGuid`)
empty and `node_id` is empty	the master's own GUID (inbound on local Xray)

Online clients are likewise grouped by GUID, so each node row shows only those actually connected to it.

What the user sees: in a flat topology (nodes directly under the master) nothing changes — counters by GUID and by `id` match. But as soon as a node chain appears, "Sub-node" rows appear in the list, and the inbound/online/exhausted numbers for each node now reflect its own load, not the sum of everything that passed through it in transit.

2. Client-IP synchronization from access.log across nodes

The IP limit (`limitIp` on a client) relies on addresses that Xray writes to its access.log. Previously, each node only saw connections to itself, so the "no more than N IPs per client" restriction did not work in a cluster: a client could connect to different nodes and bypass the limit. In 3.3.0, observed IPs are synchronized across the entire cluster.

How it works:

- On each node, a background job parses access.log, extracting the IP, client email, and timestamp from each line, and stores them in a local table (one record per email, IPs stored as a JSON array `{ip, timestamp}`). Local addresses `127.0.0.1` and `::1` are discarded.
- **Every 10 seconds**, synchronization performs a two-way exchange for each enabled online node: it pulls IPs from the node and merges them into the local table, then sends the master's consolidated picture to the node.
- Merging combines old and incoming observations **without double-counting** an IP seen on multiple nodes, and **without resurrecting stale** records: the same age threshold as in the local job is applied — **30 minutes**. The most recent timestamp is kept for each IP. Records from other nodes receive a new local id (node id spaces are independent); concurrent insertion of the same email is protected against duplicates.
- When counting the limit, an IP is considered "live" if it was observed in the current local scan, or has a very recent timestamp from the synchronized database (**within 2 minutes**). This is what makes the limit work at cluster scale, even if the address was observed on another node. When the limit is exceeded, the oldest "live" IPs are sent to the fail2ban log and connections are forcibly terminated (remove/re-add client via Xray API).

What the user sees: the IP count limit now applies to the entire cluster, not to each node individually; in the panel, a client's IPs seen on any node (within the 30-minute window) are visible. There is no separate button/setting for this — synchronization runs automatically in the background, provided the node has access.log enabled and accessible (the limit itself also requires Fail2Ban on the node).

3. Separate status indicator: node panel online, but Xray has crashed

Previously, the node status was essentially "online / offline". If the node's panel responded, the node was considered online — even when the Xray core on it was not running, and clients could not actually connect. In 3.3.0, panel health and Xray core health are separated.

How it works:

- When polling the node, the master takes the `xray.state` and `xray.errorMsg` fields from the remote `/panel/api/server/status` response and stores them in the node. These fields are populated even on a successful panel ping, when the core is unhealthy — specifically to distinguish panel availability from the Xray state.
- Values of `xray.state`: `"running"` (running), `"stop"` (stopped), `"error"` (error).
- These values are translated into node statuses. New ones have been added to the familiar ones:

Status key	Caption	When shown
`online`	"Online"	panel responds, Xray is running (`running`)
`offline`	"Offline"	panel is unreachable / ping failed
`unknown`	"Unknown"	state not yet determined
`xrayError`	"Xray error"	panel online, but Xray core is in `error` state (has `errorMsg`)
`xrayStopped`	"Stopped"	panel online, but Xray is stopped (`stop`)

- For such a state, the UI uses a **separate purple indicator** (a color different from the green "online" and red "offline"). Purple signals directly: the node is reachable, the problem is in the Xray core itself, not in the network or the panel.

What the user sees: instead of a misleading "green" when the core has crashed, the node is highlighted in **purple** with the status **"Xray error"** or **"Stopped"**. This immediately shows that the Xray on the node needs fixing (restart the core, check `errorMsg`), rather than troubleshooting the node's own accessibility. The same `xrayState` / `xrayError` is also propagated to transitive sub-nodes (see point 1), so an incorrect core state is visible throughout the chain.

---

13. Panel Settings

The "Settings" section (page title — **Panel Settings**) controls the behavior of the 3X-UI web panel itself: which address and port it listens on, how it is protected, how it interacts with the Telegram bot and external services, and in which time zone it runs scheduled tasks. Each parameter is stored in the database `settings` table as a key–value pair; if a value is absent from the DB, the default value is applied.

> **Important — applying changes.** Any change on this page must be saved with the **Save** button, and then the panel must be restarted for the changes to take effect. The literal hint: "Every change made here needs to be saved. Please restart the panel to apply changes." When saving, the notification "Settings changed" is shown.

13.1. Saving and restarting the panel

Element	Purpose
Save	Writes all form fields to the DB (`POST /panel/setting/update`). Before writing, the values pass validation — invalid addresses, ports, or paths will be rejected, and the panel will return an error.
Restart Panel	Restarts the panel web server (`POST /panel/setting/restartPanel`). The restart happens with a 3-second delay. Hint: "Are you sure you want to restart the panel? If you cannot access the panel after restarting, please view the panel log info on the server." On success — "The panel was successfully restarted."
Reset to Default	Deletes all settings saved in the DB, after which the panel uses the default values. Administrator credentials are not reset by this operation.

The restart is performed by sending the `SIGHUP` signal to the panel process (or via a registered restart hook). On Windows, automatic restart via signal is not supported. **Changes to listening parameters (IP, port, path, domain, certificates, time zone) are applied only after the panel is restarted.**

13.2. General settings ("Panel" tab / *General*)

Interface language (*Language*)

The language of the panel web interface. The available languages are: `en-US` (English), `ru-RU` (Russian), `zh-CN` , `zh-TW` , `fa-IR` , `ar-EG` , `es-ES` , `id-ID` , `ja-JP` , `pt-BR` , `tr-TR` , `uk-UA` , `vi-VN` . This is a display setting and does not affect how Xray works.

Calendar type (*Calendar Type*)

- **Key:** `datepicker`
- **Default value:** `gregorian` (Gregorian).
- **Purpose:** the calendar type used in date selection (for example, when setting client expiry dates). Hint: "Scheduled tasks will run based on this calendar." The alternative value is the Persian (Jalali) calendar, which is in demand among the panel's Iranian audience.

Pagination size (*Pagination Size*)

- **Key:** `pageSize`
- **Default value:** `25`
- **Allowed values:** an integer from `0` to `1000`.
- **Purpose:** the number of rows per page in tables (connection/inbound lists). Hint: "Define page size for inbounds table. (0 = disable)" — with `0`, pagination is disabled, and all records are shown as a single list.
- **No panel restart required** (display setting).

Restart Xray after auto disable (*Restart Xray After Auto Disable*)

- **Key:** `restartXrayOnClientDisable`
- **Default value:** `true`
- **Purpose:** when a client is automatically disabled (due to expiration or reaching the traffic limit), Xray is restarted to tear down that client's already-established connections. Hint: "When a client is automatically disabled due to expiration or traffic limit, restart Xray.". The feature itself is unchanged — the toggle simply lives on the "Panel" (*General*) tab alongside the other general settings.

Remark model and separation character (*Remark Model & Separation Character*)

- **Key:** `remarkModel`
- **Default value:** `-ieo`
- **Purpose:** defines how the configuration name (remark) is formed in the subscription. The string consists of the **first character** — the separator, followed by a **sequence of order letters**:
  - `i` — inbound remark;
  - `e` — client email;
  - `o` — extra label (*extra*).

  With the default value `-ieo`, the separator is `-`, and the order of the parts is: inbound → email → extra (for example, `MyInbound-user@mail-extra`). Empty parts are skipped. The "Sample Remark" field in the interface shows a preview of the generated name. Including the email in the name additionally depends on the "Include Email in Name" parameter in the subscription settings (subscription section).

Example: how the `remarkModel` value shapes the configuration name. Suppose the inbound is named `VLESS-Reality`, the client email is `alex@vpn`, and the extra label is `RU`. Then:

Field value	Resulting name (remark)
`-ieo` (default)	`VLESS-Reality-alex@vpn-RU`
`_ie`	`VLESS-Reality_alex@vpn`
`-ei`	`alex@vpn-VLESS-Reality`
`o` (space separator, label only)	`RU`

The first character of the string is always the separator; the remaining letters define which parts go into the name and in what order.

13.3. Panel access: IP, port, path, domain, certificate

This group defines the panel's network entry point. **All changes here are applied only after the panel is restarted.**

Field	Key	Default value	Description
Listen IP (*Listen IP*)	`webListen`	`""` (empty)	The IP on which the web panel listens. Empty = listen on all IPs. Hint: "The IP address for the web panel. (leave blank to listen on all IPs)". If specified, it must be a valid IP address (otherwise saving is rejected).
Listen Domain (*Listen Domain*)	`webDomain`	`""` (empty)	The panel's domain name for validating the request by domain. Empty = accept connections from any domains and IPs. Hint: "The domain name for the web panel. (leave blank to listen on all domains and IPs)"
Listen Port (*Listen Port*)	`webPort`	`2053`	The port on which the panel runs. Hint: "The port number for the web panel. (must be an unused port)". Allowed `1–65535`. The port must be free; the panel and the subscription service cannot use the same `IP:port` pair at the same time.
URI Path (*URI Path*)	`webBasePath`	`/`	The panel's base URL path (basePath). Hint: "The URI path for the web panel. (begins with '/' and concludes with '/')". When saving, the panel automatically adds a leading and trailing `/` if they are missing. Disallowed characters in the path are rejected.

Panel certificate (TLS / HTTPS)

Field	Key	Default value	Description
Public Key Path (*Public Key Path*)	`webCertFile`	`""`	The full path to the certificate (chain) file. Hint: "The public key file path for the web panel. (begins with '/')".
Private Key Path (*Private Key Path*)	`webKeyFile`	`""`	The full path to the private key file. Hint: "The private key file path for the web panel. (begins with '/')".

If **at least one** of the certificate/key paths is specified, the panel attempts to load the certificate + key pair when saving; on an error (a non-existent file, a mismatch between key and certificate) saving is rejected. When both correct paths are specified, the panel switches to HTTPS. Both fields empty = the panel works over plain HTTP.

> **Security warnings** (*Security warnings*). The panel shows a "Your panel may be exposed:" block with warnings if it detects an insecure configuration:
>
> - working over plain HTTP — "Panel is served over plain HTTP — set up TLS for production.";
> - the default port 2053 — "Default port 2053 is well-known — change it to a random port.";
> - the default base path `/` — "Default base path "/" is well-known — change it to a random path.";

> - the default subscription path `/sub/` and JSON subscription `/json/` — "Default subscription path "/sub/" is well-known — change it." / "Default JSON subscription path "/json/" is well-known — change it." These are recommendations, not blocks.

13.4. Session, panel proxy, and trusted proxies ("Proxy and Server" tab / *Proxy and Server*)

Session duration (*Session Duration*)

- **Key:** `sessionMaxAge`
- **Default value:** `360` (minutes, i.e. 6 hours).
- **Allowed values:** from `1` to `525600` minutes (1 year).
- **Purpose:** how long the administrator stays logged in without signing in again. The unit is the **minute**. Hint: "The duration for which you can stay logged in. (unit: minute)".

Panel traffic outbound (*Panel Traffic Outbound*)

- **Key:** `panelOutbound`
- **Default value:** `""` (empty = direct connection).
- **Purpose:** selects the Xray **outbound** through which the panel sends its **own requests** — panel/Xray version checks and downloads, requests to Telegram, the normal geo-file update — to bypass server-side filtering of GitHub/Telegram. The field is a **dropdown picker**: it lists outbounds from the Xray configuration template, outbounds derived from outbound subscriptions, and routing **balancers** (as a separate group). `blackhole` outbounds are excluded from the list — routing a download into a black hole is pointless. The literal hint: "Routes the panel's own requests — panel/Xray version checks and downloads, Telegram, and the normal geo-file update — through this Xray outbound to bypass server-side filtering of GitHub/Telegram. A loopback bridge inbound is added to the running config automatically and applied live. The Xray-native Geodata Auto-Update is not affected; it has its own download outbound. Leave empty for a direct connection."

> **How it works.** When an outbound is selected, the panel itself adds a service loopback inbound (a SOCKS bridge with the tag `panel-egress`) to the running config plus a routing rule that steers the panel's own HTTP traffic to the chosen outbound. If a balancer is selected, `balancerTag` is emitted in the rule and the panel's traffic load-balances across its members. The bridge and rule are applied **live**, without a full panel restart. Leave the field empty for a direct connection. Xray's native geo-data Auto-Update is **not affected** by this setting — it has its own outbound inside the Xray router.

- **Format:** `socks5://` (or `socks5h://`) or `http(s)://`, with authorization of the form `socks5://user:pass@host:port` if needed. The supported schemes are strictly: `socks5`, `socks5h`, `http`, `https` — other schemes are considered invalid, and the panel then falls back to a direct connection. A typical example is Xray's own local SOCKS inbound.
- The literal hint: "Routes the panel's own outbound requests (geo updates, Xray/panel version checks, Telegram) through this proxy to bypass server-side filtering of GitHub/Telegram. Accepts socks5:// or http(s)://, e.g. a local Xray SOCKS inbound. Leave empty for a direct connection."

- An invalid proxy does not cause a save error — the panel simply uses a direct connection and writes a warning to the log.

Field value example. If a local Xray SOCKS inbound is already running on the server on port `10808`, route the panel's own requests through it:

```
socks5://127.0.0.1:10808
```

For an external HTTP proxy with authorization:

```
http://user:pass@proxy.example.com:8080
```

After saving and restarting, the panel will fetch geo-database updates, check versions, and reach Telegram through the specified proxy.

Trusted proxy CIDRs (*Trusted proxy CIDRs*)

- **Key:** `trustedProxyCIDRs`
- **Default value:** `127.0.0.1/32,::1/128` (localhost only).
- **Format:** a comma-separated list of IP addresses or CIDR subnets (for example `10.0.0.0/8, 192.168.1.5`). Each element is validated as an IP or CIDR — an invalid value is rejected when saving.
- **Purpose:** lists the sources that are allowed to set the `X-Forwarded-Host`, `X-Forwarded-Proto` headers and the client's real IP header. The literal hint: "Comma-separated IPs/CIDRs allowed to set forwarded host, proto, and client IP headers." It needs to be configured if the panel runs behind a reverse proxy (nginx, Caddy, etc.), so that client IPs and the scheme are determined correctly.

Example: the panel behind a reverse proxy. If nginx runs on the same host and proxies requests to the panel, trust only localhost (the default value):

```
127.0.0.1/32,::1/128
```

If the proxy sits on a separate server in the internal `10.0.0.0/8` network, add its subnet, otherwise the panel will ignore the headers it forwards and will see the proxy's IP instead of the real client:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

A matching nginx block that forwards the real IP and the scheme:

```
proxy_set_header X-Real-IP        $remote_addr;
proxy_set_header X-Forwarded-For  $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Telegram bot ("Telegram Bot" tab / *Telegram Bot*)

Enable Telegram Bot (*Enable Telegram Bot*)

- **Key:** `tgBotEnable`
- **Type/default:** boolean, `false`.
- **Purpose:** enables the Telegram bot. Hint: "Enables the Telegram bot.".

Telegram token (*Telegram Token*)

- **Key:** `tgBotToken`
- **Default:** `""`.
- **Purpose:** the bot token. Hint: "The Telegram bot token obtained from '@BotFather'.".
- **Security specifics:** the token is among the secret values. It is not returned in the panel's response when reading settings (the field is cleared, only a "configured/not configured" flag is provided). If the field is left empty when saving, the previously saved token is **kept** (not overwritten).

Telegram bot language (*Telegram Bot Language*)

- **Key:** `tgLang`
- **Default:** `en-US`.
- **Purpose:** the language of the bot's messages (independent of the web interface language). The list of available languages matches the panel's languages.

Bot admin User ID (*Admin Chat ID*)

- **Key:** `tgBotChatId`
- **Default:** `""`.
- **Format:** one or more numeric Telegram User IDs **separated by commas**.
- **Purpose:** the recipients of notifications and the administrators who are allowed to manage the panel via the bot. Hint: "The Telegram Admin Chat ID(s). (comma-separated)(get it here @userinfobot) or (use '/id' command in the bot)".

Notification time (*Notification Time*)

- **Key:** `tgRunTime`
- **Default:** `@daily` (once per day).
- **Format:** a string in **Crontab** format (both standard cron expressions and abbreviations of the form `@daily`, `@hourly`, `@every 1h` are supported). Hint: "The Telegram bot notification time set for periodic reports. (use the crontab time format)". Controls the bot's periodic reports.

Field value examples.

Value	When the bot sends a report
`@daily`	once a day at midnight (default)

Value	When the bot sends a report
`@hourly`	every hour
`@every 6h`	every 6 hours
`0 9 * * *`	daily at 09:00
`30 8 * * 1`	every Monday at 08:30

The time is interpreted in the zone from the "Time Zone" setting (section 13.6).

SOCKS proxy (*SOCKS Proxy*)

- **Key:** `tgBotProxy`
- **Default:** `""`.
- **Purpose:** a SOCKS5 proxy, used separately for the bot's connection to Telegram. Hint: "Enables SOCKS5 proxy for connecting to Telegram. (adjust settings as per guide)". It applies specifically to the bot's traffic (different from the general "Panel Network Proxy" in section 13.4).

Telegram API Server (*Telegram API Server*)

- **Key:** `tgBotAPIServer`
- **Default:** `""` (use the standard server `api.telegram.org`).
- **Format:** a `http(s)://…` URL; when saving, it passes a URL validity check — an invalid address is rejected. Hint: "The Telegram API server to use. Leave blank to use the default server.". Needed for a self-hosted Telegram Bot API server.

Bot notifications (the "Notifications" group / *Notifications*)

Field	Key	Default	Description
Database Backup (*Database Backup*)	`tgBotBackup`	`false`	Send the DB backup file to Telegram together with the report. Hint: "Send a database backup file with a report.".
Login Notification (*Login Notification*)	`tgBotLoginNotify`	`true`	Notify on a panel login attempt. Hint: "Get notified about the username, IP address, and time whenever someone attempts to log into your web panel.".
Expiration Date Notification (*Expiration Date Notification*)	`expireDiff`	`0`	How many **days** before a client's expiry date to send a notification. `0` — disabled. Allowed `>= 0`. Hint: "Get notified about expiration date when reaching this threshold. (unit: day)".
Traffic Cap Notification (*Traffic Cap Notification*)	`trafficDiff`	`0`	The remaining-traffic threshold for the notification. Hint: "Get notified about traffic cap when reaching this threshold. (unit: GB)". Allowed `0–100`.
CPU Load Notification (*CPU Load Notification*)	`tgCpu`	`80`	Notify administrators if the CPU load exceeds the threshold (in **%**). Allowed `0–100`. Hint: "Get notified if CPU load exceeds this threshold. (unit: %)".

13.6. Date and time ("Date and Time" tab / *Date and Time*)

Time zone (*Time Zone*)

- **Key:** `timeLocation`
- **Default value:** `Local` (the server's system time zone).
- **Format:** a zone name from the IANA tz database (for example, `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Purpose:** the time zone in which the panel runs scheduled tasks (bot reports, traffic resets/checks, expirations). Hint: "Scheduled tasks will run based on this time zone.".
- **Validation:** when saving, the zone is checked — a non-existent zone is rejected. If an invalid value later ends up in the DB, the panel falls back to `Local` at runtime, and if that too is unavailable — to `UTC`.

13.7. External traffic and Xray behavior ("External Traffic" tab / *External Traffic*)

Field	Key	Default	Description
External Traffic Inform (*External Traffic Inform*)	`externalTrafficInformEnable`	`false`	Notify an external API on every traffic update. Hint: "Inform external API on every traffic update.".
External Traffic Inform URI (*External Traffic Inform URI*)	`externalTrafficInformURI`	`""`	The URL to which the panel sends traffic updates. Passes a URL validity check when saving. Hint: "Traffic updates are sent to this URI.".
Restart Xray After Auto Disable (*Restart Xray After Auto Disable*)	`restartXrayOnClientDisable`	`true`	Restart Xray when a client is automatically disabled due to expiration or exceeding the traffic limit. Hint: "When a client is automatically disabled due to expiration or traffic limit, restart Xray.". **The toggle is on the "Panel" (*General*) tab** — see section 13.2; it is listed here for completeness.

13.8. Other: Xray configuration template and test URL

Xray configuration template (*xrayTemplateConfig*)

- **Key:** `xrayTemplateConfig`
- **Default:** the embedded JSON template, shipped with the build.
- **Purpose:** the base JSON template for the Xray-core configuration, on top of which the panel builds the inbounds/outbounds. This value is **not returned** in the normal output of all settings and is edited on a separate Xray configuration page, not in the general list of panel settings fields. The standard default template is available via `GET /panel/setting/getDefaultJsonConfig`.

Outbound test URL (*xrayOutboundTestUrl*)

- **Key:** `xrayOutboundTestUrl`
- **Default:** `https://www.google.com/generate_204`

- **Purpose:** the URL used when testing the operability of outbound connections. When set, it is sanitized as an HTTP(S) URL.

13.9. Administrator account and API tokens

These parameters are on the adjacent tab ("Account" / *Authentication*) and are covered in detail in the security section; here is a brief summary of the keys.

- **Changing credentials** (the "Current Username", "Current Password", "New Username", "New Password" fields) is saved with a separate request `POST /panel/setting/updateUser`. The correct current username and password are required; the new username and password must not be empty. Messages: "You have successfully changed the credentials of the administrator." / "Wrong username or password".
- **Two-factor authentication (2FA)** — the keys `twoFactorEnable` (default `false`) and the secret `twoFactorToken`. The token is a secret: when 2FA is enabled, an empty field when saving does not overwrite the existing token. On the **first** enabling of 2FA, the panel invalidates the current sessions (the "login epoch" is raised).
- **API tokens** are managed by separate endpoints (`/panel/setting/apiTokens…`): list, create (`apiTokens/create`), delete, enable/disable. The token itself is shown **only once, at creation**, and is not stored in readable form: "Copy this token now. For security it is not stored in readable form and will not be shown again."

Details on 2FA, passwords, LDAP synchronization, and subscription formats (JSON/Clash, fragmentation, noises, mux) are moved to the corresponding separate sections of the manual.

13.10. API changes in 3.3.0 (important for integrations)

In version 3.3.0, the structure of the server API paths changed. If you have external integrations (scripts, bots, central panels, CI jobs) that access the panel over HTTP, they **need to be fixed**, otherwise they will stop working.

⚠ BREAKING CHANGE: the `/panel/setting/*` and `/panel/xray/*` endpoints moved under `/panel/api`

Previously, management of panel settings and the Xray configuration lived separately, under the paths `/panel/setting/*` and `/panel/xray/*`. Now both sets are registered inside the common API group `/panel/api`. The old paths are **completely removed** — a request to them will return 404.

Why this was done: the entire `/panel/api` group goes through the unified access check, that is, these endpoints now accept the same `Authorization: Bearer <token>` header as the rest of the API. An API token is full administrator access, and in this way the entire API surface became uniform.

What did NOT change: the web interface pages (SPA routes) `/panel/settings` and `/panel/xray` stayed in place — this is only about the server API endpoints.

Path mapping table (old → new)

The prefix for all paths below — `api/` was simply added after `/panel/`.

Was (≤ 3.2.x)	Now (3.3.0)	Method
`/panel/setting/all`	`/panel/api/setting/all`	POST
`/panel/setting/defaultSettings`	`/panel/api/setting/defaultSettings`	POST
`/panel/setting/update`	`/panel/api/setting/update`	POST
`/panel/setting/updateUser`	`/panel/api/setting/updateUser`	POST
`/panel/setting/restartPanel`	`/panel/api/setting/restartPanel`	POST
`/panel/setting/getDefaultJsonConfig`	`/panel/api/setting/getDefaultJsonConfig`	GET
`/panel/setting/apiTokens`	`/panel/api/setting/apiTokens`	GET
`/panel/setting/apiTokens/create`	`/panel/api/setting/apiTokens/create`	POST
`/panel/setting/apiTokens/delete/:id`	`/panel/api/setting/apiTokens/delete/:id`	POST
`/panel/setting/apiTokens/setEnabled/:id`	`/panel/api/setting/apiTokens/setEnabled/:id`	POST
`/panel/xray/`	`/panel/api/xray/`	POST
`/panel/xray/update`	`/panel/api/xray/update`	POST
`/panel/xray/getDefaultJsonConfig`	`/panel/api/xray/getDefaultJsonConfig`	GET
`/panel/xray/getXrayResult`	`/panel/api/xray/getXrayResult`	GET
`/panel/xray/getOutboundsTraffic`	`/panel/api/xray/getOutboundsTraffic`	GET
`/panel/xray/resetOutboundsTraffic`	`/panel/api/xray/resetOutboundsTraffic`	POST
`/panel/xray/testOutbound`	`/panel/api/xray/testOutbound`	POST
`/panel/xray/warp/:action`	`/panel/api/xray/warp/:action`	POST
`/panel/xray/nord/:action`	`/panel/api/xray/nord/:action`	POST
`/panel/xray/outbound-subs` (and `/outbound-subs/*`)	`/panel/api/xray/outbound-subs` (and `/outbound-subs/*`)	GET/POST/DELETE

The sub-path names themselves, the request bodies, and the response formats did not change — **only the prefix** changed.

How to fix existing integrations

1. Find all occurrences of `/panel/setting/` and `/panel/xray/` in your scripts/configs.
2. Replace the prefix: add `api/` right after `/panel/` (for example, `/panel/setting/all` → `/panel/api/setting/all`).
3. The request bodies, parameters, and response format do not need to be edited — only the URL changes.
4. Since settings and the Xray configuration are now under `/panel/api`, they can (and should) be accessed with the same API token `Authorization: Bearer <token>` as `/panel/api/inbounds/*` and the other endpoints. Don't forget the CSRF middleware, which is enabled for the entire `/panel/api` group.

Example: reading all settings via the API. Before (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
  -H "Authorization: Bearer <token>"
```

Now (3.3.0) — `api/` is added after `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
  -H "Authorization: Bearer <token>"
```

Likewise restarting the panel: `POST /panel/api/setting/restartPanel`. The old path `/panel/setting/restartPanel` now returns 404.

Typed API: schemas and documentation (Swagger / OpenAPI)

In 3.3.0, the OpenAPI specification became fully typed. Previously, typed responses were described by an empty object `{}`; now the components and schemas (`components.schemas`) are generated directly from the data models. Thanks to this:

- Swagger UI shows real data models, rather than faceless stubs.
- External generators (`openapi-generator`, etc.) can build ready-made clients in the desired language from the specification.
- A `$ref` to a concrete model is attached to each typed response, and response examples are included.

Where to look for the API documentation:

- **The built-in Swagger page.** In the panel menu — the **"API Documentation"** item (the SPA route `/panel/api-docs`). Here all endpoints are listed interactively, with descriptions, request bodies, and response examples.
- **The raw OpenAPI 3.0 specification** is served at `/panel/api/openapi.json`. This URL can be fed directly into Postman, Insomnia, or `openapi-generator`. The specification is embedded in the binary at build time; when the panel runs under a non-standard `webBasePath`, the `servers` field in the specification is automatically rewritten to the current base path, so that the "Try it out" button and external generators hit the correct prefix.

---

14. Telegram Bot

The 3X-UI panel includes a built-in Telegram bot through which you can receive notifications about server and client status, as well as manage individual clients directly from the messenger. The bot operates via long polling (continuous polling of Telegram), so it does not require an external domain or open port — outbound access to Telegram's servers is sufficient.

The bot distinguishes between two types of users:

- **Administrator** — a user whose Telegram User ID is specified in the bot settings (the "Bot Admin User ID" field). Has access to all features: server statistics, backup, client management, Xray restart.
- **Client** — any other user whose Telegram User ID is linked to a specific client of an inbound connection (the `tgId` field of the client). Can only see information about their own subscriptions.

Example: linking a client to Telegram. For a user to receive statistics about their subscription, their numeric Telegram User ID is written into the client's `tgId` field. In the client's JSON settings it looks like this:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

After this, the user with User ID `123456789` can request `/usage ivan` from the bot and see their statistics. The same ID can be set by the administrator via the "👤 Set Telegram User" button in the client card — no need to edit the JSON manually.

14.1. Enabling and configuring the bot

All bot parameters are set in the panel under **Settings** → **Telegram Bot**. After changing the settings, simply save them — the panel applies them immediately, no panel restart is required. If the enable flag (`tgBotEnable`), token, administrator User IDs, or API server address are changed, the panel automatically stops and restarts the bot with the new parameters. The old rule requiring a panel restart after changing the token no longer applies.

Field (UI)	Settings key	Default value	Description
Enable Telegram Bot	`tgBotEnable`	`false`	Main toggle. Hint: "Access panel features via Telegram Bot". While disabled, the bot does not start and notification tasks are not scheduled.
Telegram Token	`tgBotToken`	(empty)	Bot token. Hint: "You need to get the token from the Telegram bot manager @botfather". Without a non-empty token the bot will not start.

Field (UI)	Settings key	Default value	Description
SOCKS Proxy	`tgBotProxy`	(empty)	Proxy for connecting to Telegram. Hint: "If you need a Socks5 proxy to connect to Telegram, configure its parameters according to the guide".
Telegram API Server	`tgBotAPIServer`	(empty)	Alternative Telegram API server. Hint: "The Telegram API server to use. Leave empty to use the default server".
Bot Admin User ID	`tgBotChatId`	(empty)	One or more administrator Telegram User IDs separated by commas. Hint: "To get your User ID, use @userinfobot or the `/id` command in the bot".
Bot Notification Frequency for Admins	`tgRunTime`	`@daily`	Periodic report schedule in crontab format. Hint: "Specify the notification interval in Crontab format".
Database Backup	`tgBotBackup`	`false`	Hint: "Send a notification with the database backup file". Attaches the backup to the periodic report.
Login Notification	`tgBotLoginNotify`	`true`	Hint: "Displays the username, IP address, and time when someone attempts to log into your panel".
CPU Threshold for Notification	`tgCpu`	`80`	CPU load threshold in percent (validated 0–100). Hint: "Notify admins in Telegram if CPU load exceeds this threshold (value: %)". When set to 0, CPU checking is disabled.
Telegram Bot Language	—	—	Language in which the bot composes all messages.

Obtaining a token via @BotFather

1. Open a chat with **@BotFather** in Telegram.
2. Send the `/newbot` command and follow the instructions (bot name and a unique `username` ending in `bot`).
3. BotFather will issue a token in the form `123456789:AA...`. Copy it into the **Telegram Token** field.

Obtaining the administrator User ID

User ID is the numeric account identifier (not a username). You can find it in two ways:

- Message the **@userinfobot** bot.
- Start the already-configured bot and send it the `/id` command — it will return your ID.

Enter the resulting number in the **Bot Admin User ID** field. To assign multiple administrators, list their IDs separated by commas (e.g., `11111111,22222222`). Each ID is validated as an integer; an invalid value will cause the bot to fail to start.

Example: value of the "Bot Admin User ID" field. A single administrator — just a number:

```
123456789
```

Two administrators separated by a comma (spaces are optional):

```
123456789,987654321
```

Each value must be an integer. Values like `@username` or `123 456` (with a space inside the number) are not valid — the bot will not start.

Proxy

The schemes `socks5://`, `http://`, and `https://` are supported. If the proxy field is left empty, the bot attempts to use the panel's general proxy (if one is set and its scheme is supported). A URL with an unsupported scheme or invalid syntax is ignored — the bot connects directly. A proxy is useful when direct access to the Telegram API is blocked from the server.

Email notifications (SMTP)

In addition to Telegram, the same events can be received by email. The channel is configured under **Settings** → **Email** on the **SMTP Settings** tab:

Field (UI)	Settings key	Default value	Description
Enable Email Notifications	`smtpEnable`	`false`	Main toggle for email notifications via SMTP.
SMTP Host	`smtpHost`	(empty)	SMTP server host (e.g., `smtp.gmail.com`).
SMTP Port	`smtpPort`	`587`	SMTP server port.
SMTP Username	`smtpUsername`	(empty)	Username for SMTP authentication. Also used as the sender address (From).
SMTP Password	`smtpPassword`	(empty)	Password for SMTP authentication. Stored hidden; if a password is already set, the field shows a "configured" indicator, and it can be left empty to keep the current one.
Recipients	`smtpTo`	(empty)	Comma-separated list of recipients (e.g., `admin@example.com, ops@example.com`).
Encryption	`smtpEncryptionType`	`starttls`	Connection encryption type: `none` (no encryption), `starttls` (STARTTLS), or `tls` (implicit TLS).

The **Send Test Email** button sends a test message and shows the result step by step: **Connection**, **Authentication**, and **Send**. If something goes wrong, the diagnostics indicate at which step the error occurred (e.g., "Authentication failed — check username and password" or "Server requires STARTTLS — change encryption type"), making it easier to tune the parameters.

The second tab (**Notifications**) lets you select which events will trigger email messages — using the same grouped cards as for Telegram (see "Event bus and notification selection" in section 14.5).

Telegram API Server

By default the bot connects to the official Telegram API. In the **Telegram API Server** field you can specify the address of your own Bot API server (`telegram-bot-api`). The URL is checked for safety; a blocked or invalid address is discarded and the default server is used.

14.2. Main menu and buttons

The menu is invoked with the `/start` command. Buttons are an inline keyboard attached to the message; the set of buttons depends on whether you are an administrator or a client.

Administrator menu

Button	Action
Sorted Traffic Usage Report	Lists all clients sorted by traffic, with each client's usage; email entries with no data are marked "No results".
Server Status	Server summary (see section 14.5). The "Refresh" button redraws the data.
Reset All Traffic	Resets traffic counters for **all** clients. Asks for confirmation ("Are you sure?"), then reports "Success" or "Failed" for each client, and finally "Traffic reset completed for all clients".
DB Backup	Sends the database file and `config.json` (see section 14.6).
Ban Log	Sends the log files of IP addresses banned due to exceeding the IP limit.
Inbounds	Summary of all inbounds: Remark, port, traffic, number of clients, expiry date.
Expiring Soon	List of inbounds and clients whose traffic or expiry date is running out (see section 14.5).
Commands	Shows the administrator command reference.
Online	Count and list of clients currently online; clicking an email opens the client card. "Refresh" button.
All Clients	Opens inbound selection, then the list of its clients for viewing/management.
New Client	Starts the add-client wizard (select inbound → draft → confirm).
Subscription settings / individual links / QR code	Select an inbound and client to get a subscription link, individual links, or QR codes.

Client menu

Clients have access to a limited set of buttons:

Button	Action
Client Statistics	Shows data for all subscriptions linked to the client's Telegram User ID.
Commands	Shows the client command reference.
Subscription Settings	Select your client → subscription link.

Button	Action
Individual Links	Select your client → individual links.
QR Code	Select your client → QR codes.

If a user has no clients with their Telegram User ID, the bot responds: "❌ Your configuration not found! Please ask the administrator to use your Telegram User ID in the configuration. 🆔 Your User ID: ...". This ID should be passed to the administrator to enter in the client's field.

14.3. Bot commands

Four commands are registered with the bot, visible in the Telegram "/" menu:

Command	Description (from menu)	Access	What it does
`/start`	Show main menu	everyone	Greeting; additionally shows the administrator "🤖 Welcome to the management bot!" and the main menu.
`/help`	Bot help	everyone	Displays a general greeting and a prompt to select a menu item.
`/status`	Check bot status	everyone	Replies "✅ Bot is working normally".
`/id`	Show your Telegram ID	everyone	Returns "🆔 Your User ID: `...`". Convenient for finding out your own User ID.

In addition to the registered commands, three argument-based commands are also handled (they are not shown in the "/" menu but do work):

- **`/usage [Email]`** — search for a client by email.
  - For an **administrator** shows the full client card (with management buttons).
  - For a **client** shows only their own subscription with the specified email (matched by Telegram User ID binding). Without an argument the bot asks to specify an email: "❗ Please specify an email to search for".
- **`/inbound [connection name]`** — administrator only. Searches for an inbound by Remark and displays its parameters with statistics for all clients. Without an argument (or for a client) — "❗ Unknown command".
- **`/restart`** — administrator only. Restarts Xray Core. Possible responses: "✅ Xray core restarted successfully", "❗ Xray Core is not running" (if the core is not running), "❗ Error restarting Xray-core.". Any arguments after `/restart` result in an unknown command message with a `/restart` hint.

In group chats, a command in the form `/command@botusername` is only processed if the username matches the current bot's name.

Administrator help (the "Commands" button):

```
🔄 To restart Xray Core: /restart
🔎 To search for a client by email: /usage [Email]
📊 To search inbounds (with client statistics): /inbound [connection name]
🆔 Your Telegram User ID: /id
```

Client help:

```
To view your subscription information: /usage [Email]
Your Telegram User ID: /id
```

14.4. Client management (administrator only)

After opening a client card (via "All Clients", "Online", "Expiring Soon", or `/usage`), the administrator sees the client details (email, linked inbounds, "Active" status, connection status, expiry date, traffic usage) and inline management buttons:

Button	Purpose
Refresh	Reload the client card.
Reset Traffic	Reset the client's traffic counter. Requires confirmation " Confirm traffic reset?".
Traffic Limit	Set the traffic limit. Preset values: ♾ Unlimited (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB, or " Custom" — enter a number using the built-in numeric keyboard (buttons 0–9, " " to reset to 0, " " to delete the last digit, " Confirm: N"). The value is set in gigabytes.
Change Expiry Date	Preset options: ♾ Unlimited, " Custom", add 7/10/14/20 days, 1/3/6/12 months. A positive number extends the expiry (adds days to the current expiry date or to "now" if already expired); 0 removes the expiry limit.
IP Log	Shows the client's recorded IP addresses (with timestamps if available). From the log you can use " Refresh" and " Clear IPs" (with confirmation " Confirm IP clear?").
IP Limit	Limit on simultaneous IPs. Options: ♾ Unlimited (0), 1–10, or " Custom" (numeric keyboard).
Set Telegram User	Shows the currently linked Telegram User ID for the client; allows clearing the link (" Remove Telegram User" with confirmation). Linking a new user is done via the system Telegram contact picker.
Enable/ Disable	Enables or disables the client. Requires confirmation " Confirm enable/disable user?".

All operations that change the configuration (traffic/IP limit, expiry date, Telegram user link/unlink, enable/disable) flag Xray for a restart when necessary so the changes take effect. After a successful operation the bot displays a confirmation like " : ..." and shows the client card again.

Any numeric input in the wizards is limited to values < 999999.

14.5. Notifications and reports

Notifications are sent to all administrators (all User IDs from `tgBotChatId`).

Event bus and notification selection

Notifications are built on a unified event bus, with two delivery channels — **Telegram** and **email (SMTP)**. For each channel you separately choose which events to receive notifications for. In **Settings** → **Telegram** this is done on the **Notifications** tab; in **Settings** → **Email** — on the tab of the same name.

Events are grouped into cards; each group has a master toggle with a count of enabled events (n/total) and an intermediate state when only some are selected. Available groups:

- **Outbound** — "Down" (`outbound.down`) and "Up" (`outbound.up`): outbound going down and recovering.
- **Xray Core** — "Crash" (`xray.crash`): abnormal termination of the Xray core.
- **Nodes** — "Down" (`node.down`) and "Up" (`node.up`): a node became unavailable or recovered.
- **System** — "CPU high (%)" (`cpu.high`) and "Memory high (%)" (`memory.high`): high CPU and RAM load. Both events have an inline threshold field in percent next to them.
- **Security** — "Login attempt" (`login.attempt`): an attempt to log into the panel.

The set of enabled events is stored separately: for Telegram — in `tgEnabledEvents` , for Email — in `smtpEnabledEvents` . By default both channels have "Login attempt" and "CPU high" enabled (value `login.attempt,cpu.high`).

Panel login notification

Controlled by the **Login Notification** checkbox (`tgBotLoginNotify` , enabled by default). On every attempt to log in to the web panel, administrators receive a message:

- On success: "✅ Successful login to the panel." + host, username, IP, time.
- On failure: "❗ Panel login error." + host, **reason** (e.g., "2FA error" for an incorrect second factor), username, IP, time.

High CPU and memory load

Once per minute the panel checks CPU and RAM load. If the **`tgCpu`** threshold is > 0 and the one-minute average CPU load exceeds it, administrators receive: "🔴 CPU load is N%, which exceeds the threshold of M%". RAM load is checked similarly against the **`tgMemory`** threshold (80% by default) — the "Memory high (%)" event.

Both thresholds are set via inline fields next to the "CPU high (%)" and "Memory high (%)" events in the **System** group on the Notifications tab (see "Event bus and notification selection" below). For the Email channel, separate keys `smtpCpu` and `smtpMemory` apply. When a threshold is set to 0, the corresponding check is disabled.

Periodic report (scheduled)

Scheduled by the cron expression from the **Notification Frequency** field (`tgRunTime` , default `@daily`). If the value is empty or invalid, `@daily` is used. The report includes:

Schedule builder

The **Bot Notification Frequency for Admins** field is set not by typing a string manually, but through a schedule builder. First, select a mode from the dropdown:

- **`@every` — repeat at interval** — a number field and unit selector appear (**Seconds** / **Minutes** / **Hours**); the result is assembled into an expression like `@every 6h` .

- **`@hourly` — every hour**, **`@daily` — every day at 00:00**, **`@weekly` — every week**, **`@monthly` — every month** — ready presets that are saved as the corresponding macro (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Custom (crontab)** — a field for your own crontab expression. The panel's scheduler operates with seconds enabled, so a custom expression consists of **6 fields**: second, minute, hour, day of month, month, day of week (e.g., `0 30 8 * * *` — every day at 08:30:00). When switching to this mode the field is pre-filled with the crontab equivalent of the current selection, giving you a starting point.

Example: values of the "Notification Frequency" field (`tgRunTime`). Both ready-made shortcuts and full crontab format are supported:

Value	When it fires
`@daily`	once a day at midnight (default value)
`@hourly`	every hour
`@every 6h`	every 6 hours
`0 9 * * *`	every day at 09:00
`0 9 * * 1`	every Monday at 09:00
`0 */12 * * *`	every 12 hours (at 00:00 and 12:00)

Crontab field order: minute, hour, day of month, month, day of week.

1. The line "🗓 Scheduled reports: " and current date/time.
2. **Server status** (see below).
3. An "Expiring Soon" block for inbounds and clients.
4. Personal notifications for clients with a linked Telegram User ID — each non-admin client receives a list of their subscriptions with traffic or expiry running out (including disabled ones).
5. If **Database Backup** (`tgBotBackup`) is enabled — a DB backup sent to administrators.

Server status contains: host, 3X-UI and Xray version, IPv4/IPv6, uptime (in days), average load (Load1/2/3), RAM (current/total), number of online clients, TCP/UDP connection counts, total network traffic (↑/↓), and Xray status.

"Expiring Soon" shows:

- for inbounds: number of disabled and number of "expiring soon", followed by a list of those inbounds (Remark, port, traffic, expiry date);
- for clients: the same, plus client cards and buttons with their email (clicking opens the client card).

The "expiring soon" thresholds come from the panel's general settings: traffic reserve (in GB) and expiry reserve (in days). An inbound/client is considered "expiring" when its remaining traffic is less than the threshold OR its remaining time is less than the threshold.

14.6. Backup and logs

- **DB Backup** (the "📁 DB Backup" button or the checkbox in the periodic report): the bot sends the backup time, the database file (`x-ui.db` , or `x-ui.dump` for PostgreSQL), and the Xray configuration file `config.json` .

The name of the backup file sent by the bot is derived from the server's address: the value of **webDomain** is used, and if it is not set — the server's public IP. This helps identify which server the file came from when backups are collected from multiple panels. If the address cannot be determined, a generic name is used.

- **Ban Log** (the " Ban Log" button): sends the current and previous log files of IP addresses banned for exceeding the IP limit. Empty files are not sent.

14.7. Operational notes

- **Long messages** are split into parts (threshold ~2000 characters); the inline keyboard is attached to the last part.
- **Concurrency**: commands and button presses are handled concurrently (pool of up to 10 simultaneous handlers).
- **Delivery reliability**: on connection errors, messages are retried with exponential backoff (1s/2s/4s, up to 3 attempts).
- **Caching**: "Server Status" data is cached to prevent frequent "Refresh" presses from overloading the system.
- **Bot restart**: when settings affecting the bot are saved (enable flag, token, administrator User IDs, or API server address), the panel automatically stops the previous polling loop and starts a new one with the updated parameters — no panel reload is needed. Only one update-receiving instance runs at a time.

---

15. Geo databases (geoip / geosite and custom)

Geo databases are binary `.dat` files that Xray-core uses to route and filter traffic by country membership (IP ranges) or by domain category. The panel can download and update both the standard set of geo files and arbitrary user-defined sources specified by URL. All files are stored in the `bin` directory next to the Xray binary (the default path is `bin`, overridden by the `XUI_BIN_FOLDER` environment variable).

15.1. What geoip.dat and geosite.dat are

- **geoip.dat** — a database mapping "IP address → country/region code". It is used in routing rules as `geoip:<code>`, for example `geoip:ru`, `geoip:cn`, as well as for special labels such as `geoip:private` (private/local networks). In essence it answers the question "which country is this IP located in".
- **geosite.dat** — a database mapping "domain → category/list". It is used as `geosite:<category>`, for example `geosite:category-ads-all` (advertising domains), `geosite:google`, `geosite:ru`. In essence these are grouped lists of domains.

These files are needed to build rules such as "all traffic to Russian IPs/domains goes directly, everything else goes through an outbound" and similar. The rules themselves are defined in the Xray routing section; the geo databases merely supply the data for them. Without up-to-date geo files, rules that reference `geoip:` / `geosite:` will not work or will rely on outdated lists.

Example: a "Russian domains and IPs go directly" rule. Such a rule in the routing section sends all traffic to Russian resources to the outbound tagged `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Standard geo files and their update

The panel contains a fixed allowlist of six standard files with hard-coded download sources. The update is performed via `POST /panel/api/server/updateGeofile/:fileName` (or without a file name — to update all of them at once).

Example: updating a single file and all of them via the API. Update only `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
  -H 'Cookie: 3x-ui=<session-cookie>'
```

Update all six standard files in a single request (no file name):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
  -H 'Cookie: 3x-ui=<session-cookie>'
```

A successful response:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

File name	Source (releases repository)
`geoip.dat`	`github.com/Loyalsoldier/v2ray-rules-dat` (geoip.dat)
`geosite.dat`	`github.com/Loyalsoldier/v2ray-rules-dat` (geosite.dat)
`geoip_IR.dat`	`github.com/chocolate4u/Iran-v2ray-rules` (geoip.dat)
`geosite_IR.dat`	`github.com/chocolate4u/Iran-v2ray-rules` (geosite.dat)
`geoip_RU.dat`	`github.com/runetfreedom/russia-v2ray-rules-dat` (geoip.dat)
`geosite_RU.dat`	`github.com/runetfreedom/russia-v2ray-rules-dat` (geosite.dat)

Specifics of updating the standard files:

- **Button to update a single file.** Before the download a confirmation is shown: *Do you really want to update the geofile?* with the explanation *This will update the #filename# file.* On success a notification pops up: *Geofile updated successfully.*
- **The "Update all" button** (*Update all*) downloads all six files. Confirmation: *This will update all geofiles.*
- **Conditional download.** If the local file already exists, the `If-Modified-Since` header with the file's modification time is added to the request. A `304 Not Modified` server response means the file has not changed — it is not downloaded again, only the file's timestamp is updated.
- **File name safety.** Only names from the allowlist are accepted; the name is checked to contain no `..`, no path separators `/` or `\`, no absolute paths, and must match the pattern `^[a-zA-Z0-9._-]+\.dat$`. Any name outside the list is rejected with the error "Invalid geofile name".
- **Xray restart.** After the geo files are downloaded, Xray-core is restarted so that it re-reads the updated databases. If the restart fails, a corresponding line is added to the error message.

Updating geo databases from the command line (x-ui)

Geo databases can also be updated without the panel — through the interactive `x-ui` menu (the geo-files update item) or with the non-interactive `x-ui update-all-geofiles` command. Each file in the set (geoip/geosite, including the IR and RU sets) reports its own status: "updated", "already up to date", or "download failed". On a failed download no false success message is printed. Xray is restarted (which drops active connections) only if at least one file was actually updated; if no file changed (all returned `304 Not Modified`), neither the panel nor Xray is restarted.

15.3. Geodata auto-update via Xray (Geodata Auto-Update)

Additional `.dat` sources at an arbitrary URL are added not by the panel itself but through Xray-core's native `geodata` section. The corresponding screen lives in the Xray Updates modal (Dashboard → Xray updates, `xrayUpdates`) as the "Geodata Auto-Update" tab. Here the panel only edits the `geodata` key in the Xray config template; downloading, validating, and hot-reloading the files is handled by the Xray core itself.

At the top of the section a hint is shown: *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*

Section fields

- **Schedule (cron)** (*Schedule (cron)*) — a 5-field cron string; the default value is `0 4 * * *` (daily at 04:00). On save the string is checked to contain exactly 5 fields, otherwise the error *Cron must have 5 fields, e.g. 0 4 * * ** is shown.
- **Download through outbound (optional)** (*Download through outbound (optional)*) — a dropdown with the tags of available outbounds (plus subscription outbounds) through which Xray will download the files; outbounds with the `blackhole` protocol are excluded. The field may be left empty — then a direct connection is used. This choice is independent of the outbound used for the panel's own requests (see §11): geodata auto-update has its own separate download outbound.
- **File list** — each row defines a "URL + File name" pair (*File name*). The URL must start with `https://` (otherwise *Each file needs an HTTPS URL.*). The file name must be plain, with no paths or separators — only the characters `^[A-Za-z0-9._-]+$` (otherwise *File names must be plain names like geosite_custom.dat (no paths).*). When a URL is entered, the panel tries to fill in the file name automatically from the last path segment. The "Add file" button (*Add file*) adds a row, and the trash button removes it.

If the list is empty, a hint is shown: *No files configured. Reference files in routing rules as ext:geosite_custom.dat:category.*

Saving

The "Save & Restart Xray" button (*Save & Restart Xray*) shows a confirmation *Save geodata settings?* with the explanation *This updates the Xray config template and restarts Xray.* After saving, the `geodata` key is written into the config template (`POST /panel/api/xray/update`) and Xray is restarted (`POST /panel/api/server/restartXrayService`). If the file list is empty, the `geodata` key is removed from the template.

Important specifics:

- **The file must already exist in `bin`.** Xray only updates `.dat` files that are already present in the `bin` folder at startup. So a new custom file is first placed into `bin` manually (or at least an empty/outdated version is created there under the desired name), and only then does Xray keep it up to date on schedule.
- **Hot reload.** After a scheduled download Xray re-reads the updated databases without a full restart of the process.
- **Compatibility.** Previously downloaded geo files (both standard and custom) keep working in routing rules with the `ext:` syntax unchanged.

If the list is empty, a hint is displayed: *No custom geo sources yet — click Add to create one.*

Table columns and source fields

Field (UI)	JSON	Default value	Description
Type (*Type*)	`type`	— (required)	Resource type: only `geosite` or `geoip`. Determines the name of the resulting file.

Field (UI)	JSON	Default value	Description
Alias (*Alias*)	`alias`	— (required)	A short identifier of the source. The file name is built from it and the type.
URL (*URL*)	`url`	— (required)	A direct link to the `.dat` file (http/https).
Enabled (*Enabled*)	—	—	Flag indicating whether the source is active in the list.
Last updated (*Last updated*)	`lastUpdatedAt`	`0`	Time of the last successful update (Unix time; `0` — not updated yet).
Routing (ext:...) (*Routing (ext:...)*)	—	—	A ready-made string for routing rules: `ext:<file.dat>:tag`.
Actions (*Actions*)	—	—	The "Edit", "Delete", "Update now" buttons.

Additionally, the database stores service fields: `localPath` (the actual path to the file in the `bin` directory), `lastModified` (the value of the `Last-Modified` header from the server, used for conditional download), `createdAt` and `updatedAt`.

File naming

The name of the resulting file is generated automatically from the type and the alias:

- type `geoip` → `geoip_<alias>.dat`;
- type `geosite` → `geosite_<alias>.dat`.

For example, a source with type `geosite` and alias `myads` will create the file `geosite_myads.dat`.

Example: adding a source via the API. Add your own list of advertising domains as a `geosite` resource with the alias `myads`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
  -H 'Cookie: 3x-ui=<session-cookie>' \
  -H 'Content-Type: application/json' \
  -d '{
    "type": "geosite",
    "alias": "myads",
    "url": "https://example.com/lists/myads.dat"
  }'
```

The panel downloads the file into the `bin` directory as `geosite_myads.dat`, saves the record, and restarts Xray.

Buttons and actions

- **Add** (*Add*) — opens the "Add custom geo" form (*Add custom geo*). The save button is "Save" (*Save*). API: `POST /add`.

- **Edit** (*Edit*) — the "Edit custom geo" form (*Edit custom geo*). API: `POST /update/:id`. When the type or alias is changed, the old file is deleted and a new one is downloaded again.
- **Delete** (*Delete*) — confirmation *Delete this custom geo source?* Deletes the record from the database and the `.dat` file itself. API: `POST /delete/:id`. On success: *Custom geo file "" deleted*.
- **Update now** (*Update now*) — re-downloads a specific source and updates its timestamp. API: `POST /download/:id`. On success: *Geofile "" updated*.
- **Update all** — updates all custom sources at once. API: `POST /update-all`. On full success: *All custom geo sources updated*. If at least one source fails to update, the operation is considered partially unsuccessful with the message *One or more custom geo sources failed to update*, and the response lists the successful and unsuccessful sources.

After any of these actions (add, edit, delete, update, update all when there are successes), Xray-core is restarted.

Step by step: adding a source

1. Click "Add".
2. In the "Type" field select `geosite` or `geoip`.
3. In the "Alias" field enter an identifier (only lowercase Latin letters, digits, `-` and `_`; placeholder hint: `a-z 0-9 _ -`).
4. In the "URL" field specify a direct link to the `.dat` file (it must start with `http://` or `https://`).
5. Click "Save". The panel will immediately download the file into the `bin` directory, save the record, and restart Xray.

15.4. Validation and constraints

Strict checks are performed when a source is created and edited. Error messages:

Condition	Message
Type is not `geosite` / `geoip`	*Type must be geosite or geoip*
Empty alias	*Alias is required*
Invalid characters in the alias (not `^[a-z0-9_-]+$`)	*Alias must match allowed characters*
Alias is reserved	*This alias is reserved*
Empty URL	*URL is required*
URL does not parse	*URL is invalid*
Scheme is not http/https	*URL must use http or https*
Empty/invalid host, or blocked by SSRF protection	*URL host is invalid*
Duplicate "type + alias"	*This alias is already used for this type*
Source not found	*Custom geo source not found*
Download error	*Download failed*

Hints in the form (client-side validation): *Alias may only contain lowercase letters, digits, - and* _ and *URL must start with http:// or https://*.

Additional technical constraints:

- **Reserved aliases.** You cannot use aliases that conflict with the standard files. Reserved (comparison is case-insensitive, a hyphen is treated as an underscore): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. For example, `geosite-ru` will be rejected as `geosite_ru`.
- **SSRF protection.** The URL host is resolved into an IP, and if it points to a private/internal address, the download is blocked (the user sees *URL host is invalid*). This prevents using the panel to reach internal services.
- **Path traversal protection.** The final file path must reside inside the `bin` directory (with symlinks resolved); an attempt to escape it is rejected.
- **Minimum file size.** A downloaded file is considered valid only if it is at least 64 bytes; a file that is too small is rejected with a download error.
- **Proxy and conditional download.** If a proxy is configured in the panel settings, the download goes through it; otherwise a direct connection with an SSRF-safe transport is used. As with the standard files, `If-Modified-Since` / `304 Not Modified` is applied (an unchanged file is not downloaded again). The download timeout is 10 minutes, and the URL availability probe (HEAD, and a partial GET when needed) is 12 seconds.

15.5. Auto-check at panel startup

At startup the panel iterates over all custom sources and, for each one, checks the presence and integrity of the local file (the file is missing, is a directory, or is smaller than 64 bytes). If the file is missing or corrupted, the source is probed and a re-download is attempted. This guarantees that after a reinstallation or loss of the `bin` directory the custom geo files will be restored automatically.

15.6. Using geo databases in routing rules

In Xray routing rules geo databases are used in fields such as `domain` / `ip` via prefixes:

- **geoip:** for IP databases — `geoip:<code>`. Examples: `geoip:ru`, `geoip:cn`, `geoip:private`. Taken from `geoip.dat` (or `geoip_RU.dat`, etc., if the rule points to a specific file).
- **geosite:** for domain databases — `geosite:<category>`. Examples: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Taken from `geosite.dat`.

Example: blocking ads via geosite. A rule that sends all advertising domains into a black hole (assuming an outbound tagged `blocked` with the `blackhole` protocol):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

For **custom** files the external-file syntax `ext:` is used. The hint in the UI: *In routing rules use the value column as ext:file.dat:tag (replace tag).* Format:

```
ext:<file_name.dat>:<tag>
```

where `<file_name.dat>` is `geoip_<alias>.dat` or `geosite_<alias>.dat`, and `<tag>` is a specific list/category inside the file. In the "Routing (ext:...)" column the panel suggests a ready-made template like `ext:geosite_myads.dat:tag` — you just need to replace `tag` with the desired tag. The name of such a file is set in the "Geodata Auto-Update" section (see §15.3) in the "File name" field — for example `geosite_custom.dat`; rules reference it as `ext:geosite_custom.dat:category`.

Example: a rule based on a custom file. If a source of type `geosite` with the alias `myads` has been added, and inside the `.dat` file the list is labeled with the tag `ads`, the routing rule looks like this:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

For an IP source (type `geoip`, alias `mycorp`, tag `office`) the field would be `"ip": ["ext:geoip_mycorp.dat:office"]`.

---

16. Operations: backups, logs, updating, CLI

This section covers day-to-day panel maintenance: creating and restoring database backups, viewing panel and Xray logs, restarting and stopping services, updating Xray and the panel itself, scheduled tasks (cron), and uninstalling the panel. Some operations are performed from the web interface (tabs on the "Dashboard" and "Panel Settings" pages), others from the `x-ui` console menu on the server.

16.1. Database backup and restore

All panel data (inbounds, clients, groups, nodes, settings) is stored in a single database. Backup management is available on the **"Dashboard"** page under the **"Backup"** tab, with the block heading **"Backup & Restore"**.

The panel supports two database engines, and backup behavior depends on which one is in use:

- **SQLite** (the default) — data is stored in the file `x-ui.db`.
- **PostgreSQL** — if the panel is configured to use PostgreSQL, the block displays a note:

> "This panel is running on PostgreSQL. 'Backup' downloads a pg_dump archive (.dump), and 'Restore' uploads it back via pg_restore. PostgreSQL client tools (pg_dump and pg_restore) must be installed on the server."

Export (creating a backup)

The **"Export Database"** button (`Back Up`) downloads a backup file to your device.

DB engine	File name	What happens on the server
SQLite	`{host}_YYYY-MM-DD_HHMMSS.db`	A WAL checkpoint is performed first to ensure the file contains the latest records, then the file is read in full and sent for download
PostgreSQL	`{host}_YYYY-MM-DD_HHMMSS.dump`	`pg_dump` is run, and the archive is sent for download

As of version 3.4.2 the name of the downloaded backup file contains the address at which you opened the panel and a **date-time**: `{host}_YYYY-MM-DD_HHMMSS` with the `.db` (SQLite) or `.dump` (PostgreSQL) extension — for example `panel.example.com_2026-06-27_000000.db`. This way the files are grouped by server and sorted by time, and several copies made on the same day do not overwrite each other. The same name is used in the backups sent by the Telegram bot (`host` is taken from the panel domain/IP, falling back to `x-ui`).

Interface hints:

- SQLite: "Click to download the .db file containing a backup of your current database to your device."
- PostgreSQL: "Click to download a PostgreSQL dump (.dump) of the current database to your device."

Technically the export is a `GET /panel/api/server/getDb` request. The attachment name is set by the server (`Content-Disposition`) depending on the engine.

The backup file name is derived from the server address rather than being a fixed `x-ui.db` / `x-ui.dump`. When downloading from a browser it is taken from the panel address in the address bar (the request host); otherwise it comes from the configured web domain, and if that is absent — from the server's public IP (IPv4 first, then IPv6), falling back to `x-ui`. This makes it easy to tell backups from different servers apart. The extension remains `.db` for SQLite and `.dump` for PostgreSQL; Telegram backups are also named after the same domain/IP.

Example: downloading a backup via API. The same export can be obtained with a console request — for example, in an automated backup script. An authenticated session (login cookie) is required:

```
# 1) Log in and save the session cookie
curl -s -c cookies.txt \
     -d 'username=admin&password=admin' \
     https://panel.example.com:2053/panel/login

# 2) Download the database file (the server sets the name: x-ui.db or x-ui.dump)
curl -s -b cookies.txt -OJ \
     https://panel.example.com:2053/panel/api/server/getDb
```

If the panel is served under a base path (Web Base Path), it must be added to the URL: `…:2053/<base_path>/panel/api/server/getDb`.

Import (restore)

The **"Import Database"** button (`Restore`) opens a file picker and uploads the selected file to the server for restore (`POST /panel/api/server/importDB`, form field `db`).

Interface hints:

- SQLite: "Click to select and upload a .db file from your device to restore the database from a backup."
- PostgreSQL: "Click to select and upload a .dump file to restore the PostgreSQL database. This will replace all current data."

Import process for SQLite (important — it is atomic with rollback):

1. The uploaded file is validated as a proper SQLite database; otherwise the error "Invalid db file format" is returned.
2. The file is saved as a temporary `x-ui.db.temp` and undergoes an integrity check.
3. **Xray is stopped** before the database is replaced.
4. The current database is renamed to `x-ui.db.backup` (fallback).
5. The temporary file is moved into place as the working database, schema initialization and migrations are run, then inbound migration.
6. **If any step fails** — a rollback is performed: the previous database is restored from `x-ui.db.backup`, and Xray is restarted on the old data.
7. On success the fallback file is deleted, and **Xray is automatically restarted** on the restored data.

Interface result messages:

Result	Text
Success	"Database imported successfully"
Import error	"An error occurred while importing the database"
File read error	"An error occurred while reading the database"

> Restore completely replaces the current data. Because Xray is briefly stopped during the process, existing client connections are interrupted for the duration of the import.

Migration file between engines (SQLite ⇄ PostgreSQL)

Separate from a regular backup, there is a **"Download Migration File"** function (`Download Migration`, request `GET /panel/api/server/getMigration`). It generates a portable file for switching to a different database engine:

Current engine	What is downloaded	File name	Purpose
SQLite	Portable SQL dump (text)	`x-ui.dump`	Seed PostgreSQL with your data
PostgreSQL	An SQLite database assembled from PostgreSQL data	`x-ui.db`	Switch the panel back to SQLite

Hints:

- On SQLite: "Click to download a portable .dump export (SQL text) of your SQLite database."
- On PostgreSQL: "Click to download an SQLite database (.db) assembled from your PostgreSQL data and ready to run the panel on SQLite."

The `.db ⇄ .dump` conversion for SQLite can also be performed from the CLI with the `x-ui migrateDB [file]` command (see section 16.7).

Backup via Telegram bot

If a Telegram bot is configured (see the notifications section), it can send a backup directly to the administrator's chat. A Telegram backup includes **two files**: the database itself (`x-ui.db`, or `x-ui.dump` on PostgreSQL) and the Xray configuration `config.json`. The files are preceded by the message "🗓 Backup time: ...".

There are two ways to receive a backup in Telegram:

1. **On demand.** The **"📁 DB Backup"** button in the bot menu — the bot immediately sends the files to the current chat.
2. **Automatically with the report.** The bot settings have a toggle **"Database Backup"** with the description "Send a notification with the database backup file". When enabled, every time the periodic report is sent, the bot also sends the backup to all administrators after the report. The report sending period is set by the bot's cron schedule (see section 16.6). The bot introduces pauses between files and between administrators to stay within Telegram's rate limits.

> The bot backup is only sent if the bot is running; on PostgreSQL it also requires `pg_dump` to be present on the server.

16.2. Viewing logs

The panel has two independent log viewers, both accessible from the **"Logs"** tab on the "Dashboard". Each window can be refreshed (the refresh icon in the header) and its contents can be downloaded to a file named `x-ui.log` (the download icon button).

Panel logs (application / syslog)

The panel log window (`POST /panel/api/server/logs/{count}`). Controls:

Element	Default value	Description
Line count	`20`	Drop-down list: 20 / 50 / 100 / 500 / 1000
Level	`Info`	Minimum level: Debug / Info / Notice / Warning / Error
SysLog (checkbox)	off	Where to read logs from: the application buffer or the system journal
Auto Update (checkbox)	off	Re-read the log every 5 seconds (see below)

Behavior depends on the **SysLog** checkbox:

- **Off (default):** logs are taken from the panel's internal ring buffer, filtered by the selected level. Entries are displayed with their level (DEBUG / INFO / NOTICE / WARNING / ERROR) and source: `X-UI:` — messages from the panel itself, `XRAY:` — forwarded Xray messages.

> Simple notifications without a timestamp and level (for example, the system message "Syslog is not supported" on Windows) are now shown in full as-is. Only the strict format `YYYY/MM/DD LEVEL - body` is recognized; everything else is output without parsing, so such lines are no longer truncated (previously the first three words were incorrectly treated as date/time/level).

- **On:** the panel runs `journalctl -u x-ui --no-pager -n <count> -p <level>` on the server, showing the system journal of the `x-ui` service. The allowed line count is 1 to 10000; the level accepts syslog values (`emerg/0` , `alert/1` , `crit/2` , `err/3` , `warning/4` , `notice/5` , `info/6` , `debug/7`). On Windows, SysLog mode is not supported — a warning will be shown to uncheck the box and use application logs instead. If `systemd` /the service is unavailable, an error message about `journalctl` failing to start will appear.

Example: reading the same journal from the server console. When the panel is unavailable (for example, it won't start), the system journal can be read directly — this is exactly the command the panel runs in SysLog mode:

```
# last 100 lines at warning level and above
journalctl -u x-ui --no-pager -n 100 -p warning

# follow the journal in real time
journalctl -u x-ui -f
```

> The level in this window filters the **output**. The minimum level that is actually written to the console/syslog is determined by the panel's logging level (an environment variable, default `Info`; the panel always writes to file at `DEBUG` level).

Xray access logs (access log viewer)

A separate window for the Xray access log (`POST /panel/api/server/xraylogs/{count}`). It parses Xray access log lines and displays them as a table: **Date, From, To, Inbound, Outbound, Email**.

Starting from 3.4.1, this window and the button that opens it on the Xray status card are labeled **"Access Logs"** — previously they were simply called "Logs". The rename was made to distinguish the Xray access log viewer from the panel's own log viewer, which previously had the same name.

Element	Default value	Description
Line count	`20`	20 / 50 / 100 / 500 / 1000
Filter	empty	Substring text search (applied on Enter)
Auto Update (checkbox)	off	Re-read the log every 5 seconds (see below)
Direct (checkbox)	on	Show direct connections (traffic through the freedom outbound)
Blocked (checkbox)	on	Show blocked connections (traffic to the blackhole outbound)
Proxy (checkbox)	on	Show proxied traffic

The event type is determined automatically from the outbound tag in the log line: tags matching freedom → "DIRECT" (green), blackhole → "BLOCKED" (red), everything else → "PROXY" (blue). Lines `api -> api` and empty lines are skipped.

Auto Update. Both log windows ("Logs" and "Access Logs") have an **"Auto Update"** checkbox. When enabled, the log contents are automatically re-read every 5 seconds while preserving all current window settings — the selected line count, level/filter, and the Direct / Blocked / Proxy checkboxes. Polling stops as soon as the window is closed or the checkbox is unchecked.

> For this window to show records, Xray must have the **access log** enabled with a file path (not `none`) — see below. If the access log is disabled or the file is unavailable, the window will be empty ("No Record...").

16.3. Xray logging level and configuration

Xray's own logging parameters are configured on the **"Xray Configurations"** page in the **"Log"** block, which includes the warning:

> "Logs may slow down the server. Only enable the log types you need when necessary!"

Field	Default value	Description
Log Level (`logLevel`)	`warning`	Verbosity level of the Xray error log. Allowed values: `debug`, `info`, `notice`, `warning`, `error`. Hint: "The log level for error logs, indicating what information needs to be recorded."
Access Log (`accessLog`)	`none`	Path to the access log file. The special value `none` disables access logging. Hint: "Path to the access log file. The special value 'none' disables access logging."
Error Log (`errorLog`)	empty (default path)	Path to the error log file; `none` disables it. Hint: "Path to the error log file. The special value 'none' disables error logs."
DNS Log (`dnsLog`)	`false` (off)	Enable DNS request logging. Hint: "Enable DNS request logs."
Mask Address (`maskAddress`)	empty (off)	When active, real IP addresses are automatically replaced with a masked value in logs. Hint: "When active, the real IP address is replaced with a masked one in logs."

> Because **"Access Log" =** `none` by default, the "Xray Logs" window (section 16.2) is initially empty. To make it work, set an access log path here and restart Xray.

> Note: an empty access log only affects this window. The online clients list on the "Dashboard" and the IP count limit in the client form **do not depend** on the access log — the panel determines online clients and counts their IP addresses via Xray core's online-stats API (connection statistics). On older core versions where this API is unavailable, the panel automatically falls back to the previous method (reading the access log), and in that case the access log path is still required here for the IP limit to work.

> **IP limit and fail2ban.** The IP count limit per client (the "IP Limit" field in the client form and during bulk add) is enforced on the server only if **fail2ban** is installed — it is fail2ban that bans addresses exceeding the limit. The panel checks for fail2ban (`GET /panel/api/server/fail2banStatus`); if it is absent, the "IP Limit" field becomes unavailable with an explanatory hint (on Windows — a separate message), and previously set limits on such servers are automatically reset to zero since they had no effect anyway. The fail2ban block applies to both TCP and UDP. On regular servers, fail2ban is now installed automatically during panel installation and update (see section 16.5).

Example: a `log` block that will make the "Xray Logs" window start showing records. In Xray's JSON configuration this looks like:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

The key change is replacing `"access": "none"` with a file path (for example, `"./access.log"`). After saving, restart Xray and the table in the "Xray Logs" window will populate with records.

16.4. Managing Xray: stop and restart

Xray is controlled from the Xray card on the "Dashboard". The current state is shown as one of: **Running**, **Stopped**, **Unknown**, **Error**. On error, a tooltip "Error starting Xray" is available.

Button	Endpoint	Action
Stop	`POST /panel/api/server/stopXrayService`	Stops the Xray process. On success — a warning notification "Xray service has been stopped".
Restart	`POST /panel/api/server/restartXrayService`	Restarts (or starts) Xray with the current configuration applied. On success — a notification "Xray service has been restarted successfully".

After either operation the panel broadcasts the new state over WebSocket, so the status on the "Dashboard" updates without a page reload. If the operation fails, Xray's state becomes "Error" and the error text appears in the notification.

> In addition to manual restarts, the panel itself checks whether Xray needs to be restarted (a background task every 30 s) and whether the process has crashed (checked every second) — see section 16.6.

Tunnel health monitor (Xray auto-restart)

Version 3.4.1 introduced an optional **tunnel health monitor**. When enabled, the panel periodically checks the reachability of a given URL and, after several consecutive failed checks, automatically restarts the Xray core — this helps recover a tunnel that has stopped passing traffic. The monitor is **disabled** by default and is configured **only via service environment variables** (there are no settings for it in the web interface — this is by design).

The monitor is enabled by `XUI_TUNNEL_HEALTH_MONITOR=true`. `XUI_TUNNEL_HEALTH_PROXY` should point to a local xray inbound (for example `socks5://127.0.0.1:1080`) — in that case the probe goes through Xray itself and tests the tunnel specifically; without it only host connectivity is checked, and a restart will not fix the server's own network connectivity issue. The remaining variables control probe parameters:

Variable	Purpose	Default
`XUI_TUNNEL_HEALTH_MONITOR`	Enable the monitor (on/off)	`false`
`XUI_TUNNEL_HEALTH_PROXY`	Proxy through which the probe is sent (specify a local xray inbound)	empty
`XUI_TUNNEL_HEALTH_URL`	URL that is checked	`https://www.cloudflare.com/cdn-cgi/trace`
`XUI_TUNNEL_HEALTH_INTERVAL`	Interval between checks	`30s`
`XUI_TUNNEL_HEALTH_TIMEOUT`	Timeout for a single check	`10s`
`XUI_TUNNEL_HEALTH_FAILURES`	Number of consecutive failures before restart	`3`
`XUI_TUNNEL_HEALTH_COOLDOWN`	Minimum pause between restarts	`5m`

> Restarting Xray disconnects all connected clients, so it makes sense to keep the interval and failure threshold large enough that a single accidental probe failure does not trigger unnecessary restarts.

16.5. Restarting and updating the panel

Restarting the panel

The **"Panel Settings"** page has an action **"Restart Panel"** (`Restart Panel` , `POST /panel/api/setting/restartPanel`). When confirmed, the panel restarts **in 3 seconds**.

Messages:

- Confirmation: "Are you sure you want to restart the panel? Confirm and the restart will happen in 3 seconds. If the panel becomes unavailable, check the server log."
- Success: "Panel restarted successfully".

Technically on Linux the restart is performed by sending a `SIGHUP` signal to the panel process (or via a registered hook). Sending `SIGHUP` is not supported on Windows.

Panel self-update (Update Panel)

The "Dashboard" provides an **"Update Panel"** function — updating 3X-UI to the latest release directly from the web interface.

Before updating, the panel checks versions (`GET /panel/api/server/getPanelUpdateInfo`), querying the latest 3x-ui release from GitHub:

Field	Meaning
Current panel version	Currently installed version
Latest panel version	Latest available version

Field	Meaning
Panel is up to date / "Up to date"	Shown when no new version is available

Starting an update — `POST /panel/api/server/updatePanel`. Confirmation dialog:

- "Are you sure you want to update the panel?"
- "This will update 3X-UI to version #version# and restart the panel service."

After starting — a popup message "Panel update started"; if the version check fails — "Panel update check failed".

What happens on the server: self-update is supported **only on Linux** (on other operating systems the error "panel web update is supported only on Linux installations" is returned). The panel downloads the official `update.sh` script from GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) and runs it in a separate process: preferably via `systemd-run` in a dedicated unit (`x-ui-web-update-<timestamp>`), or as a detached process if systemd is not available. When finished, the script updates the components and restarts the panel service. `bash` is required to run it.

If during an update the script generates a new random Web Base Path for the panel, the `x-ui` service is restarted automatically so the new path takes effect immediately. (Without a restart, the server would keep serving the old path while the interface showed the new one, making the new address unreachable until a manual restart.)

Dev channel (rolling), as of 3.4.2. The panel version button on the "Dashboard" now always opens the update window (previously, on the latest stable build, it led to the GitHub releases page), and the **"Dev channel"** toggle (`devChannelEnable`) is always shown — even on a stable build. By enabling it, at the next update you can switch to the "rolling" dev channel (builds for every commit on `main`). When enabled, a warning is shown: "Dev builds track every commit on main and are not stable releases — there is no automatic rollback". Without an explicit action by the user, nothing is updated.

Dev update channel (rolling builds per commit)

In addition to a regular stable-release update, there is an optional **"Dev"** channel. The toggle appears in the panel update window **only on dev builds** (CI builds assembled from a specific commit); it is not visible on stable releases. When enabled, the panel will update to the `dev-latest` rolling build, which tracks every commit on the `main` branch and is not a stable release — a warning is shown that dev builds are unstable and there is no automatic rollback. In dev mode the window shows "Current commit" / "Latest commit" instead of version numbers. The feature is available only on Linux with systemd.

On dev builds the panel shows its version as `dev+<short-commit>` instead of a misleading stable number — in the sidebar badge, on the "Dashboard" card, in the update window, in the Telegram bot status report, and in the `x-ui -v` command output. On stable releases the version display is unchanged.

> On nodes, the same 3x-ui panel is updated centrally via `POST /panel/api/nodes/updatePanel` — see the nodes section.

Automatic fail2ban installation

To make the per-client IP count limit (section 16.3) work out of the box, `fail2ban` is now installed and configured automatically during panel installation and update on regular servers (previously this only happened in the Docker image). The behavior is controlled by the `XUI_ENABLE_FAIL2BAN` environment variable: setup is performed when the variable is not set or is equal to `true`. Manual setup is available with the `x-ui setup-fail2ban` command. A fail2ban setup failure does not abort the panel installation or update.

Installation and update on IPv6-only hosts

The `install.sh` and `update.sh` scripts now work correctly on servers with only IPv6: downloading the release, `x-ui.sh` script, and service files no longer forces IPv4 (`curl -4`) and instead uses whatever protocol is available. The panel can therefore be installed and updated on a host without an IPv4 address.

Overriding the panel port with `XUI_PORT`

The web panel's listening port can be overridden with the `XUI_PORT` environment variable — it takes effect only for the lifetime of the current process and **does not modify** the saved `webPort` value in the database. Valid values are `1` to `65535` ; an empty, invalid, or out-of-range value is ignored (falling back to `webPort`) with a warning in the log. This is convenient during deployment, primarily in Docker: when using a bridge network, the published container port must match `XUI_PORT` — for example, `XUI_PORT=8080` and `ports: "8080:8080"` .

Updating and switching the Xray-core version

From the same "Dashboard" you can manage the Xray-core version independently of the panel.

- **Xray Updates** / **Version** — a drop-down list of available versions. Hints: "Select the required version" and the warning "Important: older versions may not support current settings".
- Installing/switching a version — `POST /panel/api/server/installXray/{version}` . Dialog: "Switch Xray version" / "Are you sure you want to switch the Xray version?". On success — "Xray updated successfully".

Example: switching the Xray-core version via API. The version is specified as a release tag from XTLS/Xray-core (with the `v` prefix). For example, switching to `v1.8.24` :

```
curl -s -b cookies.txt -X POST \
     https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` — the cookie file from the example in section 16.1.) After installation, Xray restarts automatically with the selected version.

On the server, when switching versions, Xray is first stopped, the archive of the required version is downloaded from GitHub (XTLS/Xray-core), the binary is extracted and replaced, and then Xray is restarted with verification of archive/binary checksums.

16.6. Scheduled tasks (cron)

The panel registers a number of background tasks at startup. Their schedules are fixed (not configurable in the UI, with the exception of the Telegram report schedule and LDAP sync). Below are the tasks relevant to operations.

Task	Schedule	Purpose
Xray liveness check	every 1 s	Monitors that the Xray process is running
Xray restart check	every 30 s	Restarts if the configuration is marked as changed
Xray traffic collection	every 5 s (starting 5 s after launch)	Traffic accounting for inbounds/clients
Client IP check	every 10 s	IP limit enforcement from the log
Node heartbeat and traffic sync	every 5 s	Communication with nodes
Log cleanup	**daily** (`@daily`)	Clears IP-limit logs and persistent access-log by rotating the current log to `*.prev.log`
Traffic reset by period	`@hourly` , `@daily` , `@weekly` , `@monthly`	Resets traffic counters for inbounds (and their clients) that have the corresponding auto-reset period configured
Telegram report	set in bot settings (default `@daily`)	Sends a report to administrators; if the option is enabled — with the database backup attached (section 16.1)
Telegram hash store reset	every 2 m	Only when the bot is enabled
CPU load monitoring for Telegram	every 10 s	Only if a CPU threshold > 0 is set

Additional notes:

- **Periodic traffic reset** only fires for inbounds that have the corresponding auto-reset mode selected (hourly/daily/weekly/monthly). The task resets the traffic for the inbound itself and for all its clients.
- **Expiry and exhaustion check.** Disabling clients upon expiry or traffic limit exhaustion is performed as part of traffic accounting: clients with an expired `expiry_time` or exhausted quota are flagged and disabled; the next period is calculated if necessary (for cyclic limits and the "count from first use" mode). This is reflected on the "Dashboard" and in lists with the statuses "Expired" / "Exhausted" / "Expiring soon".
- **Automatic Telegram backup** is a side effect of the report task; there is no separate cron schedule for backup alone. Therefore the frequency of automatic backups equals the frequency of the bot's report.

16.7. Console menu and CLI (`x-ui`)

On the server, the panel is managed with the `x-ui` command. Without arguments it opens the interactive "3X-UI Panel Management Script" menu; with an argument it runs a specific subcommand. Menu items related to operations:

Menu #	Item	Action
1	Install	Install the panel (downloads and runs `install.sh`)
2	Update	Update all x-ui components to the latest version without data loss; auto-restart afterwards
3	Update to Dev Channel (latest commit)	Update to the `dev-latest` rolling build (latest commit of the `main` branch) with confirmation (see 16.5)
4	Update Menu	Update only the `x-ui` menu script itself
5	Legacy Version	Install a specified (older) panel version by entering its number (for example, `2.4.0`)
6	Uninstall	Completely remove the panel and Xray (see 16.8)
7	Reset Username & Password	Reset the administrator login and password
8	Reset Web Base Path	Reset the panel's web base path
9	Reset Settings	Reset settings to defaults
10	Change Port	Change the panel port
11	View Current Settings	View current settings
12–14	Start / Stop / Restart	Start, stop, restart the panel service
15	Restart Xray	Restart Xray only
16	Check Status	Current service status
17	Logs Management	View and clear logs (see below)
18–19	Enable / Disable Autostart	Enable/disable automatic service start on OS boot
27	Update Geo Files	Update geo files (GeoIP/GeoSite)
25	PostgreSQL Management	PostgreSQL management

> Menu item numbering changed in 3.4.1: the addition of item 3 "Update to Dev Channel" shifted all subsequent items by one. The total number of items is now 28, and selection is entered in the range `[0-28]`.

Log management in CLI (item 16)

The "Logs Management" submenu now opens from item **17** (previously 16):

- **Debug Log** — streaming view of the service journal: `journalctl -u x-ui -e --no-pager -f -p debug` (on Alpine — `grep` over `/var/log/messages`).
- **Clear All logs** — clear the system journal: `journalctl --rotate` + `journalctl --vacuum-time=1s`, after which the service is restarted. (Not available on Alpine.)

Direct `x-ui` subcommands

All available subcommands:

Command	Description
`x-ui`	Open the administration menu
`x-ui start`	Start the panel
`x-ui stop`	Stop the panel
`x-ui restart`	Restart the panel
`x-ui restart-xray`	Restart Xray
`x-ui status`	Current status
`x-ui settings`	Show current settings
`x-ui enable`	Enable start on OS boot
`x-ui disable`	Disable start on OS boot
`x-ui log`	View logs
`x-ui banlog`	View Fail2ban ban logs
`x-ui setup-fail2ban`	Install and configure fail2ban for IP limiting (see 16.5)
`x-ui update`	Update the panel

| `x-ui update-dev` | Update the panel to the dev channel (rolling build `dev-latest`) | | `x-ui update-all-geofiles` | Update all geo files (with subsequent restart) | | `x-ui migrateDB [file]` | Convert database `.db` ⇄ `.dump` (SQLite) | | `x-ui legacy` | Install a legacy version | | `x-ui install` | Install the panel | | `x-ui uninstall` | Uninstall the panel |

> The `x-ui update` command downloads and runs the official `update.sh` (the same script used by the web update in section 16.5), asking for confirmation: "This function will update all x-ui components to the latest version, and the data will not be lost." On completion, the panel restarts automatically.

> **`-webCert` / `-webCertKey` flags in the `setting` subcommand.** The paths to the web panel certificate and private key can be set directly in the `x-ui setting -webCert <path> -webCertKey <path>` subcommand — specifying either of these flags saves the corresponding path (as does the separate `cert` subcommand), and the panel immediately switches to HTTPS.

Obtaining an API token via CLI

The API token retrieval command via CLI (menu item / `x-ui` command) does not display a previously issued token. API tokens are stored only as hashes, so an existing token cannot be retrieved in plain text. If tokens are already configured, the command reports their count, recommends managing tokens in the panel (**Settings** →

API Tokens, see the API tokens section), and immediately generates a **new fallback token** with a name like `cli-fallback-<timestamp>` and prints it, so the CLI remains useful without logging into the interface.

16.8. Uninstalling the panel

Uninstall is performed from the CLI — menu item **6 (Uninstall)** or the command `x-ui uninstall`. A confirmation is requested before uninstalling (default answer is "no"): "Are you sure you want to uninstall the panel? xray will also uninstalled!".

On confirmation, the script:

1. Stops the service and disables its autostart (`systemctl stop/disable x-ui`, or on Alpine — `rc-service` / `rc-update`), removes the service unit file and reloads the systemd configuration.
2. Removes the data and application directories (`/etc/x-ui/`, the installation directory) and the service env file (`/etc/default/x-ui`, `/etc/conf.d/x-ui`, or `/etc/sysconfig/x-ui` — depending on the distribution).
3. Removes the `x-ui` script itself and prints "Uninstalled Successfully." along with the command for reinstallation.

If the panel was using PostgreSQL (`XUI_DB_TYPE=postgres` in the env file), after removing the panel files the script additionally asks whether the PostgreSQL server itself should be removed along with all its databases: "Also purge PostgreSQL and delete all of its data?". This requires an explicit confirmation (default is to decline) and is accompanied by the warning that the removal will affect **ALL** PostgreSQL databases on the machine, including those belonging to other applications, and is irreversible. If declined, PostgreSQL and its data are left untouched.

> Uninstall is irreversible: Xray and all data (including the database) are removed together with the panel. If you may need the data, export the database first (section 16.1).

16.9. The `x-ui migrateDB` command

Starting from version 3.3.0, the `x-ui.sh` management script gained a `migrateDB` subcommand — a wrapper around the built-in `x-ui` binary (`x-ui migrate-db`) for converting the panel's SQLite database between two formats: the binary `.db` and a portable text dump `.dump` (plain SQL text).

What the command does

The command works in two directions, with the direction determined **automatically** from the input file:

Direction	Name	What happens
`.db → .dump`	dump	the binary SQLite database is exported to a text SQL file
`.dump → .db`	restore	the binary SQLite database is reconstructed from the text SQL file

Under the hood, the script calls the panel binary:

- dump: `x-ui migrate-db --src <input> --dump <output>`

- restore: `x-ui migrate-db --restore <input> --out <output>`

Syntax

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **`[file.db|file.dump]`** — the input file (first argument). If not specified, the panel's default installed database is used: `/etc/x-ui/x-ui.db`.
- **`[output]`** — path to the output file (second argument). Optional: if absent, the name is chosen automatically next to the input file (see below).

Examples:

```
x-ui migrateDB                              # dump /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db      # reconstruct .db from a dump
```

How the direction is determined

The script looks at the input file extension:

- `*.db`, `*.sqlite`, `*.sqlite3` → **dump** mode (export to text);
- `*.dump`, `*.sql` → **restore** mode (reconstruct database).

If the extension is not recognized, the script reads the first 16 bytes of the file: the signature `SQLite format 3` indicates a binary database (dump mode); otherwise the file is treated as a dump (restore mode).

Output file name when the second argument is not provided:

- on dump — same name as the input with the `.dump` extension;
- on restore — same name with the `.db` extension.

Safety checks and behavior

- **Binary presence.** If the `x-ui` binary is not found or is not executable — the error "x-ui binary not found … Is the panel installed?" is printed.
- **Feature support in the build.** The script verifies that the binary supports `migrate-db --dump/--restore` (via `x-ui migrate-db -h`). If not — it suggests updating the panel first with `x-ui update`.
- **Input file existence.** If the input file is missing, an error and the usage line are printed.
- **Output overwrite.** If the output file already exists, confirmation is requested (default is no); without confirmation the operation is cancelled. On restore, the old output file is removed first.
- **Live database protection.** On restore to the default database `/etc/x-ui/x-ui.db` when the panel is running, the operation is rejected with a requirement to first stop the panel (`x-ui stop`) or choose a different output path. This prevents overwriting the working database of a running service.
- On failure to reconstruct the database, any partial output file is deleted.

Why this is useful

- **Backup.** A text `.dump` is human-readable, convenient for storage in version control systems, and for diffing database contents.
- **Migration.** A dump is portable between machines and resilient to differences in the SQLite file format version — a working `.db` can be built from it on a new server.
- **Diagnostics.** From a `.dump` you can visually inspect the panel's structure and data without SQLite tools at hand.

Interactive mode

In addition to direct invocation, the conversion is also available from the interactive menu. The PostgreSQL submenu (`x-ui` → PostgreSQL management section) has item **9. Convert SQLite `.db <-> .dump`** : it asks for the input file path (default `/etc/x-ui/x-ui.db`) and the output path (can be left empty for auto-naming), while the direction, as in CLI mode, is determined automatically.

This document was prepared from the 3X-UI source code. If any interface item in your version differs — the panel's own behavior and UI hints take precedence.